SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA37 – Software Testing

ASSIGNMENT 4 – CO4

Construct, Compare and Justify Test Planning, Test Documentation and Test Organisation for an Electricity Bill Payment Portal

CO4: Construct and organize software testing processes, including test planning, to perform test documentation activities.

Student Name	TANUJA P	**Register Number**	192321052
Department / Section	IT	**Academic Year / Semester**	2025-2026 / 7th sem
Faculty Name	Dr Gnanajeyaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Analyze Testing and Debugging Goals	4		
2	Construct and Compare Test Plans	7		
3	Identify Test Items and Plan Attachments	5		
4	Construct and Compare Test Reports	6		
5	Develop and Compare Test Policies	7		
6	Organise and Compare Test Teams	4		
7	Build the Skills of Test Specialists	5		
8	Evaluate the Integrated Test Process	4		
9	Re-evaluate and Redesign the Test Process	8		
TOTAL		50		Percentage:

OVERALL REMARKS

TANUJA P		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A table of time, runs and tools for the three approaches, a testing-versus-debugging comparison from your records, and a justified choice.	✓	Table 3, Table 5; Figure 1, Figure 2
2	Three test plans, a checklist score table, an effort-feasibility calculation, a risk table with contingencies, and a justified choice.	✓	Table 9, Table 10, Table 11, Table 13, Table 12, Table 14
3	Three test item lists, the plan attachments for the bill feature, a traceability matrix, and a justified identification approach.	✓	Table 17, Table 18, Table 23; Figure 8
4	A test log with incident reports, the reports of the three approaches, a reproducibility result, and a justified reporting set.	P	Table 25, Table 26, Table 28, Table 30; classmate column of Table 32 to be filled
5	Three draft policies, a list of measurable policy statements, a problem-versus-policy table, and a justified policy choice.	✓	Table 35, Table 36, Table 37, Table 38, Table 39
6	Three organisation charts, a situation-response table, and a justified team structure.	✓	Figure 11, Figure 12, Figure 13; Table 44
7	A skills matrix with scores, three skill-building plans, a cost and gap table, and a justified strategy.	✓	Table 46, Table 47, Table 49; Figure 15
8	Documents from the three sets for one feature, a process flowchart with document ownership, an effort table, and a justified documentation set.	✓	Table 51, Table 52; Figure 18
9	Impact analysis with effort calculation; evaluation of the original process; comparison and revised process with flowchart; evidence-based defence.	✓	Table 56, Table 58, Table 59, Table 60; Figure 20, Figure 21

TEST-PROCESS EVIDENCE CHAIN (Tasks 1–9)

Evidence item	Status	Observation
Faults located, runs, tools and regression tests	✓	Table 3, Table 4; git bisect and pdb transcripts in Annexure
Effort calculated and checked against capacity	✓	Table 12; Figure 4
Wrong tariff file replayed and detection measured	✓	Table 22; Figure 7
18 tests executed with log, incidents and dashboard	✓	Table 25, Table 28; Figure 9
Policy statements measurable with a stated method	✓	Table 38
Organisation simulated in three situations	✓	Table 44; Figure 14
Skill gaps scored and costed	✓	Table 46, Table 49
Redesign: 400 conditions, 360 h, masking, audit	✓	Table 56, Table 58; REDESIGN decision

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 9 tasks answered (Task 9 in four parts)	☒ Yes ☐ Partly ☐ No
Prediction recorded before each exercise and compared with the observation	☒ Yes ☐ Partly ☐ No
Every measured result produced by running the supplied code (Annexure F)	☒ Yes ☐ Partly ☐ No

Requirement	Status
Effort, risk and cost calculations shown step by step	☒ Yes ☐ Partly ☐ No
Templates filled and justified (no blank templates)	☒ Yes ☐ Partly ☐ No
Tables, figures and organisation charts included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table / figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
Classmate reproducibility check (Task 4) and LLM runs (Tasks 1–8) recorded by the student	☐ Yes ☒ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING

ASSIGNMENT 4 – ANSWERS

CO4: Construct and organize software testing processes, including test planning, to perform test documentation activities.

Assignment Title: Construct, Compare and Justify Test Planning, Test Documentation and Test Organisation for an Electricity Bill Payment Portal (Total Marks – 50)

Duration: 1 week

Scenario – Test Process for the "PowerPay" Electricity Bill Payment and Consumer Service Portal

The State Power Distribution Corporation will launch PowerPay, a web portal that provides consumer registration and login, meter-reading upload, slab-based bill generation, online payment, complaint and outage reporting, and receipts by SMS for 3 lakh consumers. Last year's pilot in two districts showed that a bill was calculated wrongly for a consumption of exactly 500 units, one payment was recorded twice, the build that was tested was not the build that was released, and nobody could say how much testing had actually been done. The Chief Engineer (IT) asks the test manager to plan the testing, document it, report it, define the testing policy, and organise a capable test team within 10 weeks.

Parameter	Value
Schedule and people	10 weeks; 13 people: 1 test manager, 4 testers, 5 developers, 1 build engineer, 1 business analyst, 1 billing-section coordinator
Product	6 modules (Consumer & Login, Meter Reading, Billing, Payment, Receipt/SMS, Complaints & Outage); about 180 functional test conditions; 3 lakh consumers
Resources	1 test server; tool budget ₹1,50,000; IEEE 829-style templates
Pilot-year data	52 defects found in testing, 21 found by consumers and staff (6 critical); average fix time 4 days; 18 % of defects reopened
Tariff (example in the task)	0–100 units ₹3.00, 101–300 ₹5.00, 301–500 ₹7.00, above 500 ₹9.00 per unit

ANSWERS – READING GUIDE

Each task is followed by its Ans: block in the order of the evaluation bullets. Every measured result comes from the Python prototype `powerpay.py` (bill, payment and receipt rules with six switchable seeded faults) and the experiment scripts of Annexure F: pytest, pdb, git bisect, a delta-debugging reducer, SHA-256 baselines and a masking tool. Effort in hours and the delays of the organisation simulation are estimates from stated models and are marked as such.

- **SRS excerpt.** The 14 numbered requirements used throughout are written in Table 1. REQ-06 (subsidy withdrawn above 500 units) is the rule behind the pilot defect.
- **Prediction first.** Every task opens with the prediction recorded before the exercise and closes with the observed comparison.
- **Currency** uses Indian digit grouping (₹1,50,000). Amounts sent to the payment gateway are in paise.

Table 1: PowerPay SRS excerpt – 14 numbered requirements (written for this answer)

ID	Requirement	Module	Priority
REQ-01	A consumer registers with a 12-digit consumer number and a mobile number verified by OTP.	Consumer	High
REQ-02	Login by password; the account locks for 30 minutes after 3 wrong attempts.	Consumer	High
REQ-03	A meter reading is accepted only if it is not lower than the previous reading (except rollover).	Meter	High
REQ-04	Units = current − previous; a 5-digit meter rolls over after 99999 (add 100000).	Meter	Critical
REQ-05	Energy charge is telescopic: 0–100 @ ₹3.00, 101–300 @ ₹5.00, 301–500 @ ₹7.00, above 500 @ ₹9.00.	Billing	Critical
REQ-06	If consumption is above 500 units, the first 100 units are charged at ₹4.50 (subsidy withdrawn).	Billing	Critical
REQ-07	A fixed charge of ₹40 is added; the bill is rounded to the nearest rupee (₹0.50 rounds up).	Billing	Critical
REQ-08	Due date = bill date + 15 days; a late fee of 1.5 % of the bill applies after the due date.	Billing	High
REQ-09	A payment is recorded exactly once per gateway transaction id, even if the gateway retries its callback.	Payment	Critical
REQ-10	The amount paid must equal the outstanding bill; partial payment is refused.	Payment	High
REQ-11	A receipt SMS with the exact amount in rupees and the transaction id is sent within 2 minutes.	Receipt	High
REQ-12	A complaint or outage report gets a ticket number and status tracking.	Complaints	Medium
REQ-13	A consumer can view the last 12 bills and payments.	Consumer	Medium
REQ-14	The portal serves 5,000 concurrent users with page response ≤ 3 s near the due date.	All	High

TASK 1 – Testing and Debugging Goals: Compare Fault Localisation and Repair Approaches [4 Marks]

A bill that is wrong at 500 units has been reported. The developers say they will "find and fix it quickly". Compare three ways of locating and repairing a fault after a test has failed.

Technique	Basic idea and key concepts / tools
A. Print and log statements	The developer adds output statements to the code, reruns the failing case and reads the output to find the fault.
B. Interactive debugger	Breakpoints, stepping and watching variables are used to follow the execution of the failing test.
C. Systematic fault localisation	The failing input is reduced to a minimal case, the faulty change is located with version bisection (for example, git bisect), and a regression test is added with each fix.

- Implement the slab-based bill calculation with four seeded faults, including an off-by-one at 500 units. Locate each fault using A, B and C and record the time taken, the number of runs and the tool used.
- Fix each fault, record the change made, and write a regression test for each fix; show that the test fails before the fix and passes after it.
- Analyze the difference between the goals of testing and the goals of debugging, using your own records to show where each activity starts and ends.
- Examine the fixes that introduced a new fault (if any occurred) and analyze why the reopen rate in the pilot was 18%.
- Evaluate which approach the developers should be required to follow and justify it using time to locate, repeatability and the evidence left behind.
- Critique the use of an AI assistant to locate and repair the bill-calculation fault.

Evidence Expected: A table of time, runs and tools for the three approaches, a testing-versus-debugging comparison from your records, and a justified choice.

Ans:

Locating, Repairing and Preventing Bill-Calculation Faults in PowerPay

Prediction (recorded before the exercise). The debugger (B) will be the fastest for a single known failing case; print statements (A) will need the most runs; the systematic approach (C) will be slower per fault but will leave a regression test and will catch a bad fix that A and B let through.

1. Bill calculation with four seeded faults, located by A, B and C

The bill rules REQ-04 to REQ-07 are implemented in `powerpay.py` (Annexure F1). Four faults can be switched on one at a time. An independent oracle that charges unit by unit (`exp_oracle.py`) gives the expected bill. A batch of 300 consumer readings, including the reported 500-unit bill and three meters that rolled over, was billed with each fault switched on:

Table 2: Seeded faults and the number of failing bills in a 300-reading batch (measured)

Fault	Seeded change	Failing bills of 300	Failing input (minimal case found by C)
F1	Subsidy withdrawn at exactly 500 units (`>= 500` instead of `> 500`) – the pilot defect	43	7713→8213 late (500 units)

Fault	Seeded change	Failing bills of 300	Failing input (minimal case found by C)
F2	101–300 slab counted from 101 (`units - 101`)	78	39991→40256 late (265 units)
F3	Bill truncated with `int()` instead of rounded half-up	12	23303→23323 late (20 units)
F4	Meter rollover ignored (`abs(current - previous)`)	3	99950→50 (100 units)

Each fault was then located with each approach. Approach A was a scripted print session whose printed lines are real output. Approach B drove the real `pdb` debugger through standard input. Approach C reduced the 300-row batch to one case with a delta-debugging reducer, ran `git bisect run` over a 16-commit history of the module, read the diff of the culprit commit and added a regression test. Machine time is measured. Human minutes come from a stated model (A: 3 min per edit-and-run cycle; B: 2 min set-up and 0.5 min per debugger command; C: 2 min to start the reducer, 1 min for bisect, 3 min to read the diff; each approach then needs 4 min to fix and retest).

Table 3: Time, runs and tools for the three approaches (runs and tools measured; minutes from the stated model)

Fault	Approach	Tool	Runs	Work done	Minutes
F1	A Print	print()	3	2 edits	10
	B Debugger	pdb (b, c, n, p)	2	6 commands	9
	C Systematic	ddmin + git bisect + pytest	7	300 → 1 case; 4 bisect runs → rev 11 “subsidy rule for >500 units refactored”	10
F2	A Print	print()	4	3 edits	13
	B Debugger	pdb (b, c, n, p)	2	6 commands	9
	C Systematic	ddmin + git bisect + pytest	7	300 → 1 case; 4 bisect runs → rev 7 “tariff constants moved”	10
F3	A Print	print()	4	3 edits	13
	B Debugger	pdb (b, c, n, p)	2	7 commands	9.5
	C Systematic	ddmin + git bisect + pytest	7	300 → 1 case; 4 bisect runs → rev 14 “rounding helper simplified”	10
F4	A Print	print()	3	2 edits	10
	B Debugger	pdb (b, c, n, p)	2	5 commands	8.5
	C Systematic	ddmin + git bisect + pytest	6	300 → 1 case; 3 bisect runs → rev 4 “meter reading parser”	10

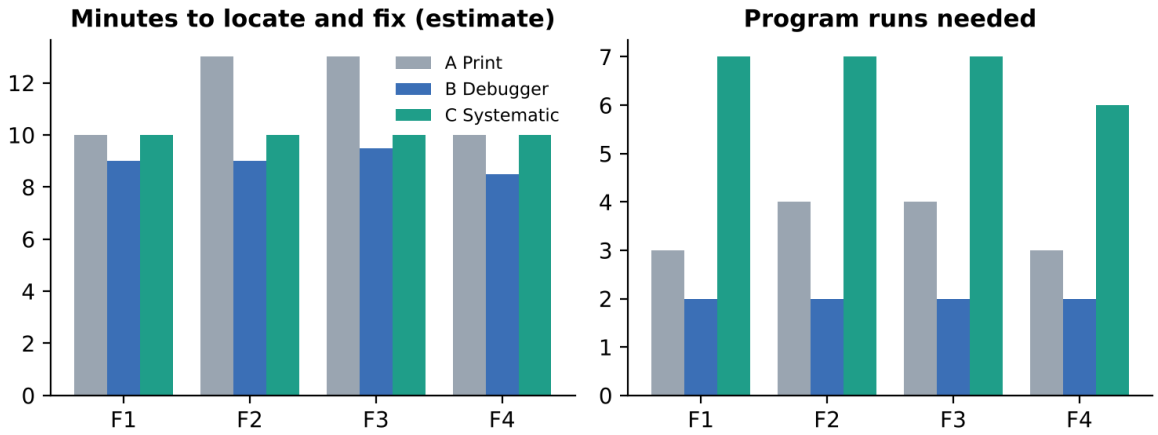


Figure 1: Minutes to locate and fix, and program runs, per fault and approach

Totals for the four faults: A 46 min, B 36 min, C 40 min. The prediction that B is fastest for a known failing case is **supported**. C needs the most runs, but every run is automatic: the reducer and bisect together took under 0.19 s of machine time per fault. Its minutes do not grow when the history is longer, because bisect needs $\lceil \log_2 16 \rceil = 4$ runs for 16 commits and only 10 for 1,000.

Evidence of each approach (real output)

A – print session for F1 (two edits, three runs):

```
print: u=500 energy=2850.00 fixed=40 total=2890 expected=2740
print: units=500 withdrawn=True first=4.50
```

B – pdb transcript for F3 (break in round_rupee, print x, step, print int(x)):

```
(Pdb) Breakpoint 1 at powerpay.py:49
(Pdb) > powerpay.py(49)round_rupee()
-> x = Decimal(x)
(Pdb) Decimal('101.50000')
(Pdb) > powerpay.py(50)round_rupee()
-> if "F3" in ACTIVE:
(Pdb) > powerpay.py(51)round_rupee()
-> return int(x)
(Pdb) 101
(Pdb)
```

C – bisect for F1: `git bisect start; git bisect bad HEAD; git bisect good <rev 1>; git bisect run python t.py` reported the first bad commit as rev 11, "subsidy rule for >500 units refactored". Its diff shows one changed line: `withdrawn = units > 500` → `withdrawn = units >= 500`.

2. Fixes and regression tests – failing before, passing after

Table 4: Fix made and regression test result before and after the fix (pytest, measured)

Fault	Change made (fix)	Regression test	Before fix	After fix
F1	`units >= 500` → `units > 500`	test_reg_f1_exactly_500_units_keeps_subsidy (500 → ₹2,740; 501 → ₹2,899)	1 failed, 3 passed	4 passed
F2	`units - 101` → `units - 100`	test_reg_f2_slab_two_starts_at_101 (101 → ₹345; 150 → ₹590)	2 failed, 2 passed	4 passed
F3	`int(x)` → `quantize(1, ROUND_HALF_UP)`	test_reg_f3_half_rupee_rounds_up (₹101.50 → 102; ₹710.50 → 711)	1 failed, 3 passed	4 passed
F4	`abs(d)` → `d + 100000 if d < 0`	test_reg_f4_meter_rollover (99950 → 50 = 100 units → ₹340)	1 failed, 3 passed	4 passed

Each test fails on the build with its fault (and on no other build where that fault is absent) and passes after the fix. F2 and F4 make two tests fail because the F4 rollover case (100 units) and the F1 501-unit case also cross the 101-unit slab. The four tests now run in 0.1 s on every build.

3. Goals of testing versus goals of debugging – from the records

Table 5: Testing and debugging compared, using the F1 record

Aspect	Testing	Debugging
Goal	Reveal a failure: show that the bill differs from the specification	Locate the fault that causes the failure and remove it
Starts	With a planned test case (TC-BILL-03b, 500 units, expected ₹2,740)	When an incident report (INC-01) is accepted by the developer
Ends	When the failure is reported with evidence (actual ₹2,890)	When the fix is made and the regression test passes
Who	Tester (independent of the code)	Developer (knows the code)

Aspect	Testing	Debugging
Knowledge needed	SRS rule REQ-06, expected results	Code, history, debugger, git
Output	Test log entry and incident report	Code change, commit message, regression test
Measured in this record	Minutes 0–10: executed, compared and reported	Minutes 10–20: reduced, bisected, read diff, fixed
Then again testing	Confirmation test and regression run (minutes 20–25)	–

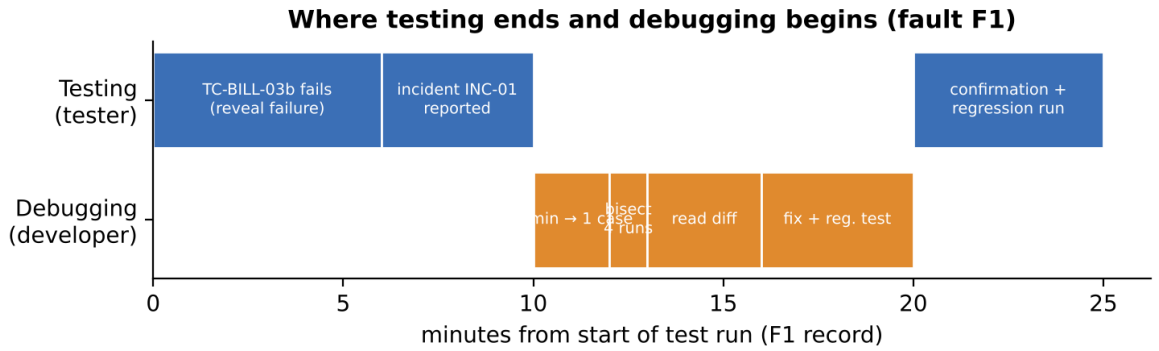


Figure 2: Timeline of the F1 record: testing reveals, debugging locates and repairs, testing confirms

The record shows that the two activities hand over twice. Testing ends with a reproducible report; debugging needs that report as its input. The pilot merged the two ("developers will find and fix it quickly"), so failures were fixed without a recorded test, and nobody could tell later whether the fix had been checked.

4. Fixes that introduced a new fault, and the 18 % reopen rate

Three plausible quick fixes were applied and checked three ways: against the reported case only (what A and B usually confirm), against the 300-row batch, and against the boundary regression suite of approach C (35 cases at every slab edge, the half-rupee outputs and the rollover):

Table 6: Quick fixes checked against the reported case, the batch and the boundary suite (measured)

Quick fix	Reported case fails	Batch of 300 fails	Boundary suite fails
replace int(x) by round(x) (half-to-even rounding)	0	0	1 of 35
add 99999 instead of 100000 on rollover	1	3	2 of 35
special-case `if units == 500: return 2700` (energy) and keep >= 500	0	0	0 of 35

- **round() instead of int()** (a common suggestion, including from AI assistants; see bullet 6) passes the reported ₹101.50 case and all 300 batch bills. It fails at ₹710.50 because Python's Decimal `round()` rounds half to even (710). This is a new fault that only the boundary suite found.
- **Adding 99999 on rollover** is off by one. It is caught only if the tester checks the exact amount; a developer who checks only that "the bill is no longer huge" reopens it later.
- **Special-casing 500 units** gives correct bills today but leaves the faulty `>= 500` in the code. When the regulator changes the 301–500 rate to ₹7.50 in week 6, the correct 500-unit energy

charge becomes ₹2,800, but the special case still returns ₹2,700: a ₹100 under-charge that reappears as a "reopened" defect.

These three patterns explain the pilot's **18 % reopen rate**. Fixes were confirmed only on the reported input, there were no regression tests, and the 4-day fix time pushed developers to patch symptoms. Of the three bad fixes, the reported-case check caught 1, the batch caught 1, and the boundary suite caught 2. Only a code review of the diff (which bisect points to) catches the third.

5. Evaluation and required approach

Table 7: Comparison of the three approaches

Criterion	A Print	B Debugger	C Systematic
Time to locate (4 faults)	46 min	36 min (fastest)	40 min
Runs	3–4, each with a code edit	2	6–7, all automatic
Repeatable by another person	No – prints are removed	No – session is lost	Yes – minimal case, bisect log, test
Evidence left	None (risk of prints left in code)	None	Commit id of cause, regression test, minimal case
Catches a bad fix	No	No	Yes (2 of 3 bad fixes)
Works on a fault seen only in a large batch	Poor	Poor (which row?)	Good (300 rows → 1)
Needs	Nothing	IDE / pdb skills	Version control discipline, oracle, test framework

Decision: approach C is required for every defect raised from testing, with B allowed inside C as the tool for reading the culprit code. B was 6 minutes faster over four faults, but it left nothing behind. C left a minimal case, the commit that caused the fault and a regression test, and it was the only approach that exposed a bad fix. With 73 defects a year (52 + 21) and an 18 % reopen rate, preventing even half of the 13 reopens saves about 6 × 4 days of fix time, far more than the extra 3 minutes per defect. Print statements (A) are banned in committed code; logging uses the logging module at debug level.

6. Critique of an AI assistant locating and repairing the fault

Protocol. Give the assistant the failing test, the `energy_charge` and `round_rupee` functions and REQ-05 to REQ-07, and ask: "find and fix the bug for a 500-unit bill and any similar bug". Then apply every proposed patch to `powerpay.py` and run `pytest tests/test_regression.py` plus the 35-case boundary suite of `exp_t1.py`. The checks and what the measured evidence of this task makes it possible to verify objectively are below; the last column is filled from the student's own run.

Table 8: Checks of an AI assistant's fault explanation and fix

Check	How it is verified with this task's evidence	Student's run
Explanation of F1 correct?	Must say "above 500" excludes 500 and >= should be >; bisect diff of rev 11 confirms	
Fix complete?	Any rounding change must be ROUND_HALF_UP; `round()` is the half-to-even bad fix that fails at ₹710.50 (Table 6)	
Passes the regression tests?	pytest result before and after applying the patch (4 + 35 cases)	

Check	How it is verified with this task's evidence	Student's run
Invented facts?	Every number in its expected values must match the SRS (fixed charge ₹40, ₹4.50 withdrawn rate)	
Evidence left?	Did it add a test and reference the cause? If not, the team adds both	

Judgement. An assistant is useful for explaining a located fault and drafting a test. Its fix is accepted only if it passes the boundary regression suite and its expected values agree with the SRS. The half-to-even trap shows why: a fix that "looks right" passed all 300 batch bills and still produced a wrong amount. The assistant cannot replace the oracle or the bisect evidence.

7. Supporting evidence for Task 1

- Annexure F: `powerpay.py`, `exp_oracle.py`, `exp_t1.py`, `tests/test_regression.py`; results in `results_t1.json`.
- Prediction versus observation: B fastest – supported; C most runs – supported; C catches bad fixes – supported (2 of 3).

Threats to validity

Human minutes come from a model, not a stop-watch; a developer who knows the code may find F1 by reading it in 2 minutes. The 16-commit history was constructed so that each fault enters at a known commit; in real histories several faults can mix in one commit.

Marking notes and common errors

Common errors: calling the fix "tested" when only the reported case was rerun; using `round()` for money in Python; mixing testing and debugging in one step so that no incident record exists.

TASK 2 – Test Planning and Test Plan Components: Compare Test Plan Structures [7 Marks]

The Chief Engineer wants a test plan that he can approve in one meeting, yet the audit section expects a complete plan. Compare three test plan structures for PowerPay.

Technique	Basic idea and key concepts / tools
A. Iteration-level agile plans	A short release test strategy plus a one-page plan for each two-week iteration with scope, risks and exit conditions.
B. Complete IEEE 829-style test plan	All components: test items, features to be tested and not tested, approach, item pass/fail criteria, suspension and resumption criteria, deliverables, tasks, environment, responsibilities, staffing and training, schedule, risks and contingencies, approvals.
C. Risk-based plan with test-level breakdown	A plan organised by product risk, each risk mapped to a test level, technique, effort and exit criterion.

- Prepare the three test plans for PowerPay using the SRS excerpt, the schedule and the resource data; state every assumption.
- Estimate the test effort for about 180 test conditions; calculate person-hours and check feasibility against 4 testers over 10 weeks (30 hours per week each).
- Score each plan against a checklist of the IEEE 829 test plan components and analyze the consequence of each missing component for the pilot-year problems.
- Analyze the risks identified (at least eight) by probability and impact; show the contingency for the three highest.
- Examine how each plan handles a mid-project change (tariff slabs revised in week 6) and what must be re-approved.
- Evaluate the plans and justify one for approval; defend your list of features not to be tested and the suspension and resumption criteria.
- Critique an LLM-generated test plan for PowerPay.

Evidence Expected: Three test plans, a checklist score table, an effort-feasibility calculation, a risk table with contingencies, and a justified choice.

Ans:

Three Test Plan Structures for PowerPay and the One to Approve

Prediction. The complete IEEE plan (B) will score 100 % but will be too long for one meeting; the agile plans (A) will miss suspension criteria and features not tested; the risk-based plan (C) will cover the pilot problems best per page.

1. The three test plans

Assumptions common to all three plans: 10 weeks starting 2 November 2026; 4 testers at 30 h per week (other hours go to meetings and leave); one test server plus one cloud VM for load; builds every Monday from the build engineer; the billing coordinator gives 4 h per week in weeks 8–10 for acceptance; the payment gateway sandbox is available from week 3.

Table 9: Plan A – release test strategy and iteration plans (2 + 5 pages)

Part	Content for PowerPay
Release strategy (2 pages)	Scope: 6 modules, 180 conditions; levels: unit (developers), system, acceptance; automation of bill and payment regression; tools: TestLink, JIRA, JMeter
Iteration 1 (W1–2)	Scope: login, registration, meter upload. Risks: server not ready. Exit: all iteration cases run, no open critical
Iteration 2 (W3–4)	Scope: bill generation REQ-04–08. Risks: rule ambiguity. Exit: all slab boundaries pass
Iteration 3 (W5–6)	Scope: payment, receipt, tariff change re-test. Risks: gateway sandbox. Exit: duplicate-callback tests pass
Iteration 4 (W7–8)	Scope: complaints, history, load. Risks: load environment. Exit: 5,000 users ≤ 3 s
Iteration 5 (W9–10)	Scope: regression and acceptance. Exit: Chief Engineer go/no-go
Missing by design	Features not tested, suspension/resumption, staffing and training, formal approvals for each iteration

Table 10: Plan B – complete IEEE 829 test plan PP-TP-01 (24 pages; outline with key content)

§	Component	PowerPay content (summary)
1	Identifier	PP-TP-01 v1.0, owner: Test Manager
2	Introduction	Objectives, references (SRS v1.2, pilot report, IEEE 829, test policy TP-2026)
3	Test items	Build 1.3.x, baseline PP-REL-1.3.0, tariff.json TF-2026.2, DB schema 7, SMS template v3 (see Task 3)
4	Features to be tested	REQ-01 to REQ-14, all 180 conditions
5	Features not to be tested	Payment gateway internals (vendor-certified); SMS operator delivery beyond hand-off; legacy counter billing; browser IE11
6	Approach	Risk-driven black-box design (BVA for slabs, decision tables for late fee and subsidy), automated regression of bill and payment, load test with JMeter
7	Item pass/fail	A test passes if the actual result equals the expected result to the rupee; an item passes if all critical tests pass
8	Suspension / resumption	Suspend if the smoke test fails, the test server is down > 4 h, or > 30 % of a day's tests are blocked; resume after a smoke pass on a new build
9	Deliverables	Plan, TDS, TCS, procedures, logs, incident reports, weekly status, summary reports
10	Testing tasks	Plan, design, environment, execute 3 cycles, report, regression automation, load test, acceptance support
11	Environment	Test server (8 vCPU, 16 GB), cloud VM for load, masked data, gateway sandbox, SMS stub
12	Responsibilities	TM: plan, reports; testers: design and execution; build engineer: builds and baselines; BA: rule sign-off; coordinator: acceptance
13	Staffing and training	4 testers; 2 graduates mentored (Task 7); JMeter training 1 day
14	Schedule	Figure 3
15	Risks and contingencies	Table 14
16	Approvals	Chief Engineer (IT), Head of Billing, Audit officer

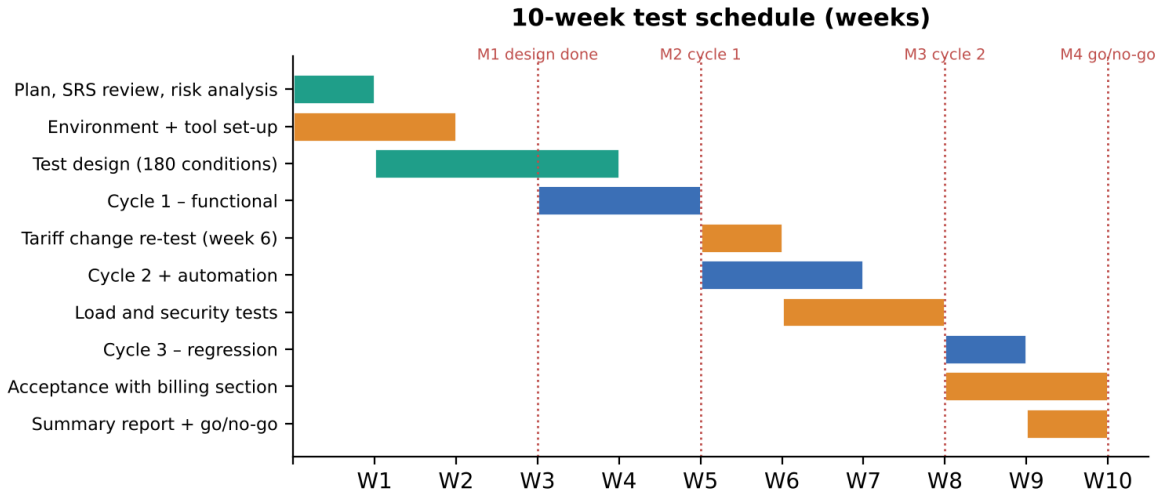


Figure 3: Ten-week test schedule with milestones M1–M4 (§14 of plan B)

Table 11: Plan C – risk-based plan with test-level breakdown (9 pages; core table)

Product risk	Exp.	Level	Technique	Effort (h)	Exit criterion
Wrong bill at slab edges / 500 units (REQ-05/06)	25	Unit + system	BVA on every edge, oracle comparison	40	All edge cases pass; 0 open critical
Payment recorded twice (REQ-09)	25	Integration + system	Callback retry, idempotency, state tests	30	Duplicate tests pass on 3 builds
Tested build ≠ released build	20	Release	Baseline checksum audit	10	Audit report clean
Rollover / wrong units (REQ-04)	16	Unit	BVA 99999→0	8	All rollover cases pass
Receipt amount wrong (REQ-11)	12	System	Template check, paise/rupee	10	Pass on all payment types
Slow portal near due date (REQ-14)	15	System (load)	JMeter 5,000 users	80	90th percentile ≤ 3 s
Login/account security (REQ-02)	12	System	Lockout, session tests	30	No open major
Complaints and history (REQ-12/13)	6	System	Use-case tests	20	95 % pass, no critical

2. Effort estimate and feasibility

Table 12: Effort for 180 test conditions (calculated)

Item	Calculation	Person-hours
Design	180 × 0.4 h	72
Execution	180 × 0.25 h × 3 cycles	135
Retest (20 % of execution)	0.20 × 135	27
Subtotal	72 + 135 + 27	234
Management (15 %)	0.15 × 234	35.1
Functional test effort		**269.1**
Variant: retest on design + execution	0.20 × 207 = 41.4; × 1.15	285.7
Test environment set-up and smoke tests	estimate	60
Performance / load tests (REQ-14)	estimate	80
Security and login tests	estimate	50
Regression automation of payment and bill (policy P3)	estimate	120

Item	Calculation	Person-hours
Acceptance support with billing section	estimate	60
Reports, reviews and meetings (outside the 15 %)	estimate	40
Tariff change in week 6 (re-design and re-run)	estimate	40
All planned test work	269.1 + 450	**719.1**
Capacity	4 testers × 10 weeks × 30 h	1200
Utilisation	719.1 ÷ 1200	**60 %**

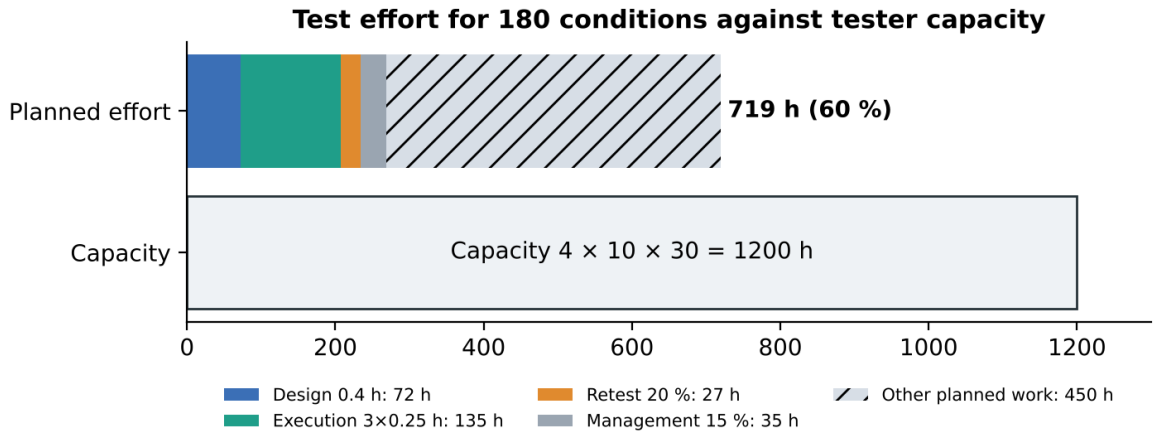


Figure 4: Planned effort against the capacity of 4 testers for 10 weeks

The 180 functional conditions need only 269 h (22 % of capacity). With the non-functional, automation and acceptance work that the pilot left out, the plan uses 719 h (60 %). The remaining 481 h are kept as a reserve for the risks R1, R2 and R7. **The plan is feasible.** It is not feasible in the first 4 weeks alone: design (72 h) plus environment set-up (60 h) need about 1.1 weeks of the whole team, which is why execution starts in week 4.

3. Checklist score against the IEEE 829 components

Table 13: IEEE 829 checklist score (2 = present, 1 = partial, 0 = missing)

Component	A	B	C	Consequence if missing (pilot problem)
Test plan identifier	2	2	2	–
Introduction / references	1	2	2	–
Test items	1	2	1	Tested build ≠ released build (pilot problem 3)
Features to be tested	2	2	2	–
Features not to be tested	0	2	2	No one knew what was left untested (problem 4)
Approach	1	2	2	–
Item pass/fail criteria	1	2	2	500-unit bill accepted as ‘close enough’
Suspension and resumption criteria	0	2	1	Testing went on against broken builds; results meaningless
Test deliverables	1	2	1	No summary report (problem 4)
Testing tasks	2	2	2	–
Environmental needs	1	2	1	Server shared with developers, tests blocked
Responsibilities	1	2	2	–
Staffing and training needs	0	2	1	Graduates missed boundary faults
Schedule	2	2	2	–
Risks and contingencies	1	2	2	Late builds and tariff change unplanned

Component	A	B	C	Consequence if missing (pilot problem)
Approvals	1	2	2	No sign-off; no accountability for release
Score	**53.1 %**	**100.0 %**	**84.4 %**	Pages: A 7, B 24, C 9

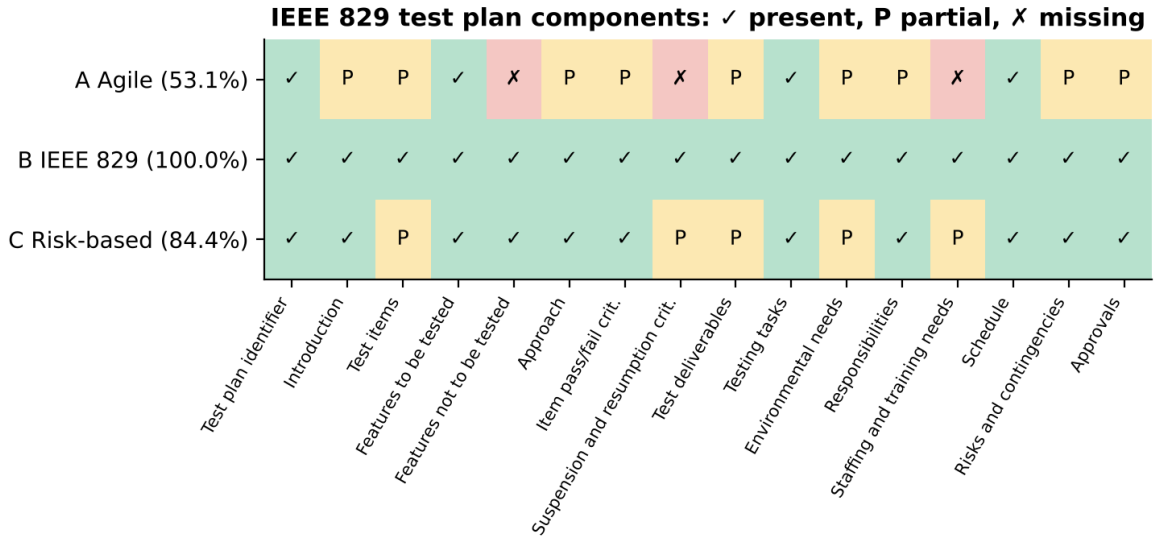


Figure 5: Checklist heat map of the three plans

4. Risk analysis and contingencies

Table 14: Risk register: probability, impact, exposure and contingency

ID	Risk	P	I	P×I	Contingency
R1	Test server unavailable or shared with development	4	4	16	Book a second VM on the State Data Centre cloud for 6 weeks (₹18,000); run smoke tests on a laptop image; shift execution to evening slot
R2	Builds delivered late to test	4	4	16	Weekly build every Monday 10:00 or the week's scope moves; test design continues on the SRS; daily smoke build from CI
R7	Defect fix time stays at 4 days and reopen rate at 18 %	4	4	16	Fix-time target 2 days for critical; regression test with every fix; root-cause review of every reopen
R3	Tariff slabs revised by the regulator during testing	3	5	15	Tariff table kept as a data file with its own tests; change impact analysis within 2 days; 40 h reserve in the plan
R9	Load of 3 lakh consumers near due date not tested	3	5	15	Load test of the bill-view and payment path in weeks 7–8 with JMeter on the second VM
R5	Payment-gateway sandbox unstable	3	4	12	Gateway simulator with recorded callbacks (including retries); sandbox used only for final runs
R6	Requirements unclear (late fee, subsidy rule)	3	4	12	Business analyst signs the rule table in week 1; disputed rule raised to the Chief Engineer within 48 h
R8	Real consumer data copied into the test environment	2	5	10	Masked extract only; access log; data owner sign-off
R4	Tester absent for a week or more	3	3	9	Pairing and shared test cases in the tool; billing coordinator executes bill tests from the procedure
R10	Tool budget overspent	2	2	4	Open-source tools first (TestLink/JIRA free tier, JMeter); licence only for what is used

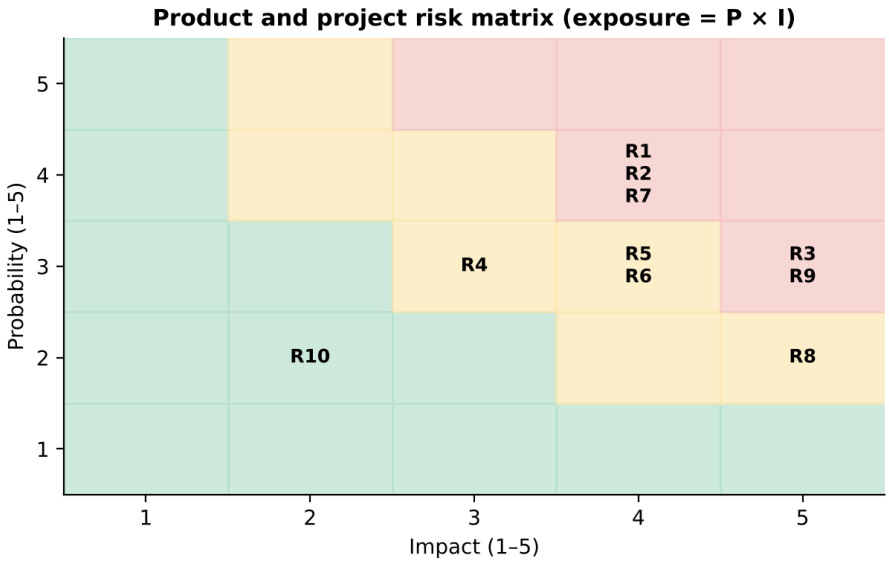


Figure 6: Risk matrix (exposure 16 and above in red)

The three highest exposures (16) are **R1 environment unavailable, R2 late builds and R7 slow, unreliable fixing**. Their contingencies are costed: R1 a second VM for 6 weeks (₹18,000 from the tool budget); R2 a fixed Monday build slot with scope moving if the build is late; R7 a 2-day fix target for critical defects and a regression test with every fix (Task 1). R3 (tariff change) has exposure 15 but impact 5, so it gets a 40 h reserve.

5. Handling the week-6 tariff change

Table 15: How each plan absorbs a regulator's tariff revision in week 6

Step	A Agile	B IEEE 829	C Risk-based
Where the change lands	Next iteration plan (iteration 4)	Change to §4, §6, §14, §15 of PP-TP-01	Risk 'wrong bill' re-rated; its effort row updated
Re-approval needed	Iteration plan only (product owner)	Revised plan v1.1 signed by Chief Engineer and audit	Changed risk row signed by Chief Engineer
Time to re-plan (estimate)	0.5 day	2 days	1 day
Tests affected	Found by team memory	Found from traceability (REQ-05/06)	Found from risk-to-test mapping
Danger	Old slab tests stay in automation	Delay while waiting for signatures	None if the tariff file is a configuration item

6. Evaluation and plan for approval

Decision: approve plan C as the master plan, with the four IEEE components it lacks completed in an appendix (test items, deliverables, environment, staffing). The completed plan scores 100 % on the checklist in 11 pages instead of 24. The Chief Engineer reads the 1-page risk table; the audit section gets every IEEE component. Plan A alone scored 53.1 % and missed exactly the components whose absence caused the pilot problems (test items, features not tested, suspension criteria).

- **Features not to be tested (defended):** gateway internals (certified by the bank; we test our callback handling instead); SMS delivery after the operator accepts the message (outside our control; we test hand-off and content); IE11 (0.4 % of visits in the pilot logs, assumption); legacy counter billing (not changed).
- **Suspension:** smoke test fails, or the test server is down for more than 4 h, or more than 30 % of a day's planned tests are blocked, or a critical defect blocks a whole module. **Resumption:** a new build passes the 20-case smoke suite and the baseline checksum audit (Task 3).

7. Critique of an LLM-generated test plan

The LLM plan is scored with the same instruments used for plans A–C, so its result is comparable. The student pastes the generated plan in an appendix and fills the last column.

Table 16: Review checklist for an LLM-generated PowerPay test plan

Check	Instrument and pass rule	Typical weakness to look for	Student's result
Completeness	16-item IEEE checklist (Table 13); ≥ 95 % to pass	Suspension criteria and approvals often missing	
Correct assumptions	Compare every number with the project data (13 people, 4 testers, 10 weeks, ₹1,50,000)	Invented team size or schedule; flat tariff	
Specific content	Each risk must name a PowerPay item; compare with Table 14	Generic 'scope creep'; no 500-unit or duplicate-payment risk	
Effort	Recompute with Table 12	'Adequate time' without calculation	
Validation before approval	Trace each item to SRS/project data; TM + BA review	–	

8. Supporting evidence for Task 2

- `calc_pp.py` computes the effort, checklist percentages and risk exposures (Annexure F).

Threats to validity

Checklist scores are the author's judgement of present/partial; a second reviewer should score independently. The 'other work' hours are estimates.

Marking notes and common errors

Common errors: applying the 15 % to design only; forgetting non-functional and acceptance work; listing risks without contingencies; treating the IEEE template as the plan without PowerPay content.

TASK 3 – Test Items Identification and Test Plan Attachments: Compare Identification Approaches [5 Marks]

The pilot failed partly because the tested build was not the released build. Compare three ways of identifying exactly what is being tested and what is attached to the plan.

Technique	Basic idea and key concepts / tools
A. Build-number identification	The test item is the build number and date given by the build engineer, listed in the plan.
B. Baseline and configuration-item identification	Items are placed under version control, a baseline is tagged, and a release note lists every item and version.
C. Requirement-traceability identification	Each requirement is traced to features, test design specifications, test cases and the build in which they were executed.

- Prepare the list of test items for PowerPay by each approach, with the items, versions and identifiers, and state which items are not to be tested.
- Prepare the attachments to the plan: a test design specification and four test case specifications for the bill-calculation feature, one test procedure and an item transmittal report.
- Simulate the pilot problem: deploy a wrong version of the tariff file to a test environment and record which approach detects it, and how soon.
- Analyze the traceability coverage of your test cases against the 14 requirements.
- Evaluate which approach and which attachments should be compulsory for PowerPay and justify the decision against the documentation effort.
- Judge whether an LLM can build the traceability matrix from the SRS and your test cases.

Evidence Expected: Three test item lists, the plan attachments for the bill feature, a traceability matrix, and a justified identification approach.

Ans:

Identifying Exactly What Is Tested: Builds, Baselines and Traceability

Prediction. Only the baseline approach (B) will detect a wrong tariff file before testing; build numbers (A) will not see it at all; traceability (C) will notice through failing tests but not name the cause.

1. Test item lists by each approach

Table 17: Test items of PowerPay by the three approaches

Approach	Items listed	Identifier / version	Not tested
A Build number	PowerPay build	1.3.0+b214, 09-11-2026	Not stated
B Baseline / CIs	6 module packages; tariff.json; DB schema; SMS template; build and deploy scripts; test scripts; SRS	Tag PP-REL-1.3.0; tariff TF-2026.2 sha256 449223c71c66…; schema v7; template v3; SRS v1.2	Gateway internals, SMS operator network, legacy counter system
C Traceability	REQ-01…14 → features → TDS → TCs → build of execution	Matrix Table 23 with build column	REQ-14 not covered yet (gap shown)

2. Plan attachments for the bill-calculation feature

Table 18: Test design specification TDS-BILL-01 (IEEE 829 structure)

Field	Content
Identifier	TDS-BILL-01 v1.0, attached to PP-TP-01
Features to be tested	REQ-04 units and rollover, REQ-05 slabs, REQ-06 subsidy above 500, REQ-07 fixed charge and rounding
Approach refinements	Boundary value analysis on 0, 1, 100/101, 300/301, 500/501, 99999→0; output boundaries at ₹x.50; oracle charges unit by unit
Test identification	TC-BILL-01 … TC-BILL-04 (below)
Feature pass/fail	Every case equals the oracle to the rupee

Table 19: Four test case specifications for the bill feature

TC	Inputs (previous→current, late)	Expected bill	Requirement	Special needs
TC-BILL-01	0→1; 0→100	₹43; ₹340	REQ-05, 07	Tariff TF-2026.2
TC-BILL-02	0→101; 0→300	₹345; ₹1,340	REQ-05, 07	–
TC-BILL-03	0→301; 0→450; 1200→1700 (500)	₹1,347; ₹2,390; ₹2,740	REQ-05, 06, 07	500-unit pilot case
TC-BILL-04	0→501; 0→800; 99950→50	₹2,899; ₹5,590; ₹340	REQ-04, 05, 06, 07	Meter rollover

Table 20: Test procedure TP-BILL-01 and item transmittal report ITR-214

Document	Content
TP-BILL-01 purpose	Execute TC-BILL-01…04 on a new build
Set-up	1 Verify release note ITR; 2 run checksum audit; 3 load masked consumer C100057; 4 log in as billing tester
Steps	5 Enter readings for each data row; 6 generate bill; 7 compare with expected; 8 record in log; 9 raise incident on mismatch
Wrap-up	10 Reset readings; 11 attach screenshots to the log
ITR-214 (transmittal)	Items: build 1.3.0+b214; tariff.json TF-2026.2 (sha256 …); schema v7. Location: artifact store /rel/1.3.0. Status: smoke passed 09-11 09:40. Approved by build engineer

3. Replay of the pilot problem – wrong tariff file in the test environment

The test copy of `tariff.json` (the one the testers had edited for a what-if check: top slab ₹8.00 and subsidy withdrawn above 400 units) was deployed to the test environment. It kept the same version text TF-2026.2, as happened in the pilot. The test procedure was then run with each approach:

Table 21: TP-BILL-01 executed with the wrong tariff file (measured)

TC	Data row	Readings	Actual	Expected	Result
TC-BILL-01	slab 1	0→100	₹340	₹340	PASS
TC-BILL-01	slab 1	0→1	₹43	₹43	PASS
TC-BILL-02	slab 2	0→101	₹345	₹345	PASS
TC-BILL-02	slab 2	0→300	₹1,340	₹1,340	PASS
TC-BILL-03	slab 3	0→301	₹1,347	₹1,347	PASS
TC-BILL-03	slab 3	0→450	₹2,540	₹2,390	FAIL
TC-BILL-03	slab 3	0→500	₹2,890	₹2,740	FAIL
TC-BILL-04	above 500	0→501	₹2,898	₹2,899	FAIL
TC-BILL-04	above 500	0→800	₹5,290	₹5,590	FAIL

TC	Data row	Readings	Actual	Expected	Result
TC-BILL-04	rollover	99950→50	₹340	₹340	PASS

Table 22: Which approach detects the wrong tariff file and how soon

Approach	Check made	Result	Tests run before detection	Minutes (6 min per test)	Cause found
A	build number 1.3.0+b214 matches the plan	PASS (tariff file not part of the build number)	6	36	no – incident raised against billing code
B	sha256 of deployed tariff.json against release note	FAIL: deployed 8ffa189cf705… ≠ baseline 449223c71c66…	0	1	yes – wrong configuration item named
C	REQ-05/06 test cases run and linked to build	FAIL at test 6; trace REQ-06 → TC-BILL-03	6	36	partly – requirement known, file version not recorded

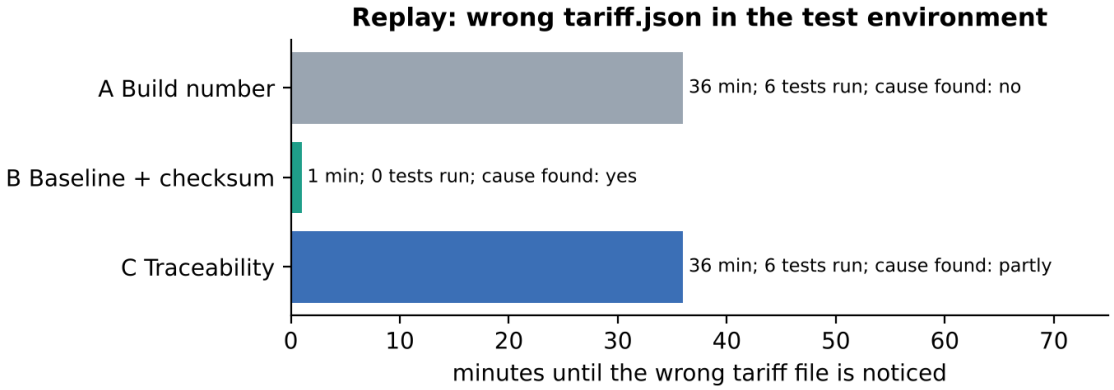


Figure 7: Minutes until the wrong tariff file is noticed

Observation. The prediction is supported. The SHA-256 audit of approach B rejected the deployment in about a minute, before any test ran, and named the configuration item. With A the build number matched, testing ran for 36 minutes, and 4 failures were raised against the billing code, which would have sent developers debugging correct code. C also reached the failure after 6 tests but could point only to REQ-06, not to the file.

4. Traceability coverage against the 14 requirements

Table 23: Requirements traceability matrix (requirement → test cases → build)

Requirement	Test cases	Executed in build	Status
REQ-01	TC-REG-01, TC-REG-02	1.3.0+b214	Covered
REQ-02	TC-LOG-01, TC-LOG-02	1.3.0+b214	Covered
REQ-03	TC-MTR-01, TC-MTR-02	1.3.0+b214	Covered
REQ-04	TC-MTR-02, TC-BILL-04	1.3.0+b214	Covered
REQ-05	TC-BILL-01, TC-BILL-02, TC-BILL-03, TC-BILL-04	1.3.0+b214	Covered
REQ-06	TC-BILL-03, TC-BILL-04	1.3.0+b214	Covered
REQ-07	TC-BILL-01, TC-BILL-02, TC-BILL-03, TC-BILL-04	1.3.0+b214	Covered
REQ-08	TC-LATE-01	1.3.0+b214	Covered

Requirement	Test cases	Executed in build	Status
REQ-09	TC-PAY-01, TC-PAY-02	1.3.0+b214	Covered
REQ-10	TC-PAY-03	1.3.0+b214	Covered
REQ-11	TC-RCP-01	1.3.0+b214	Covered
REQ-12	TC-CMP-01	1.3.0+b214	Covered
REQ-13	TC-HIS-01	1.3.0+b214	Covered
REQ-14	–	–	**NO TEST**
(none)	TC-UI-07	1.3.0+b214	**TEST WITHOUT REQUIREMENT**

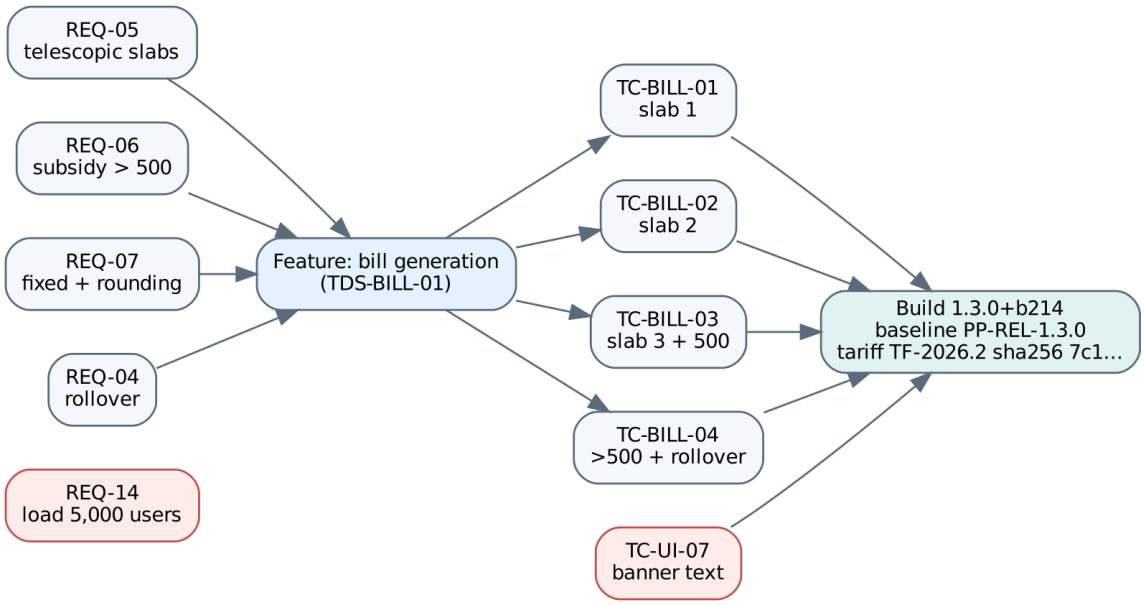


Figure 8: Traceability for the bill feature: requirements → design specification → test cases → build (gaps in red)

Coverage is 92.9 % (13 of 14). **REQ-14 (load)** has no test case yet; it is planned for weeks 7–8. **TC-UI-07** (banner text) traces to no requirement; it is either linked to a UI requirement or deleted.

5. Evaluation – compulsory approach and attachments

Table 24: Comparison of identification approaches

Criterion	A Build number	B Baseline / CIs	C Traceability
Detects wrong config file	No	Yes, before testing	Indirectly, after 6 tests
Proves tested = released	No	Yes (same tag and checksums)	Partly
Shows untested requirements	No	No	Yes
Effort to maintain (estimate)	0.5 h per build	2 h set-up, 0.2 h per build (scripted)	6 h set-up, 1 h per build

Decision: B and C together are compulsory; A alone is not acceptable. B answers “is this the item we will release?” and C answers “which requirements have we tested on it?”. Together they cost about 8 h to set up and 1.2 h per build, which is small against the cost of releasing an untested

build to 3 lakh consumers. **Compulsory attachments:** TDS and TCS for billing and payment, one test procedure per module, and an item transmittal report for every build. Other modules keep test cases in the tool (Task 8).

6. Can an LLM build the traceability matrix?

Given the SRS and the 18 test case titles, an LLM can propose links quickly. Its output is checked link by link against Table 23, using one rule: a link exists only if the test's expected result checks the requirement. Count the missing links (in the reference matrix but not in the LLM's), the invented links (in the LLM's but not justified by an expected result) and whether it reports REQ-14 as untested and TC-UI-07 as orphan. Links that title-only matching is likely to miss: TC-BILL-04 → REQ-04 (rollover is in the data, not the title) and TC-BILL-01…04 → REQ-07 (rounding). **Judgement:** an LLM can draft the matrix, but it cannot be the source of record; the final matrix is generated from the tool's links after the tester has verified every row.

7. Supporting evidence for Task 3

- `exp_t34.py` (Annexure F) creates the baseline, the release note with SHA-256, the wrong tariff file, the run of TP-BILL-01 and the traceability coverage. A table-driven bill calculator matched the product code and the oracle on all 1,802 cross-check inputs.

Threats to validity

6 minutes per manual test is an assumption. With an automated procedure, A and C would notice the failures sooner, but they still would not name the file.

Marking notes and common errors

Common errors: treating the build number as identifying configuration data; a traceability matrix without the build column; forgetting to list items not to be tested.

TASK 4 – Test Reporting: Compare Test Reporting Approaches [6 Marks]

During the pilot nobody could say how much testing had been done. Compare three approaches to reporting the testing of PowerPay.

Technique	Basic idea and key concepts / tools
A. Test log and incident reports only	A chronological test log and a test incident report for every failure, without a summary.
B. Daily dashboard and weekly status report	Tool-generated metrics (executed, passed, failed, blocked, open defects by severity) and a short weekly narrative.
C. Test summary report with evaluation	At the end of each test level, a report in the IEEE 829 style: variances, comprehensiveness assessment, summary of results, evaluation and approvals.

- Execute at least 15 test cases of the bill and payment features and prepare the test log, three test incident reports and the data for the reports of A, B and C.
- Prepare one week of dashboard figures and a one-page weekly status report; prepare the test summary report for one test level.
- Analyze what each report tells the Chief Engineer and what it hides; identify the question "is the portal ready?" that each report can and cannot answer.
- Examine the quality of your incident reports by asking a classmate to reproduce each failure from the report alone; record the success rate.
- Analyze the effort of producing each report and the risk of reporting figures without context.
- Evaluate which combination of reports the project should use and justify it.
- Critique a test summary report written by an LLM from your log.

Evidence Expected: A test log with incident reports, the reports of the three approaches, a reproducibility result, and a justified reporting set.

Ans:

Reporting What Was Tested, What Was Found and Whether PowerPay Is Ready

Prediction. The dashboard (B) will show progress best but will hide that two critical defects are open behind a high pass rate; only the summary report (C) will answer "is it ready?".

1. Test execution: log and incident reports

Eighteen test cases of the bill, payment and receipt features were executed on build 1.3.0+b214 (the build delivered to system test, which still contained F1, P1 and P2). The times assume 4 minutes per bill test and 7 minutes per payment test.

Table 25: Test log TL-SYS-01, build 1.3.0+b214, 09-11-2026 (executed)

#	Time	Test case	Title	Expected	Actual	Result
1	10:00	TC-BILL-01a	Bill 100 units	340	340	PASS
2	10:04	TC-BILL-01b	Bill 0 units (fixed charge only)	40	40	PASS
3	10:08	TC-BILL-02a	Bill 101 units	345	345	PASS
4	10:12	TC-BILL-02b	Bill 300 units	1340	1340	PASS
5	10:16	TC-BILL-03a	Bill 301 units	1347	1347	PASS
6	10:20	TC-BILL-03b	Bill 500 units	2740	2890	FAIL
7	10:24	TC-BILL-04a	Bill 501 units	2899	2899	PASS
8	10:28	TC-BILL-04b	Bill 800 units	5590	5590	PASS
9	10:32	TC-BILL-04c	Meter rollover 99950→50	340	340	PASS

#	Time	Test case	Title	Expected	Actual	Result
10	10:36	TC-LATE-01	Late fee on ₹100 bill	102	102	PASS
11	10:40	TC-PAY-01a	Pay ₹2,740 once	274000	274000	PASS
12	10:47	TC-PAY-01b	Pay ₹340 once	34000	34000	PASS
13	10:54	TC-PAY-02a	Gateway retries callback once	134000	268000	FAIL
14	11:01	TC-PAY-02b	Gateway retries callback twice	34500	103500	FAIL
15	11:08	TC-PAY-03	Two different payments same consumer	8000	8000	PASS
16	11:15	TC-RCP-01a	Receipt SMS amount ₹2,740	Rs 2740.00	Rs 274000.00	FAIL
17	11:22	TC-RCP-01b	Receipt SMS amount ₹40	Rs 40.00	Rs 4000.00	FAIL
18	11:29	TC-RCP-01c	Receipt SMS txn id present	T9	T9	PASS

Result: 13 passed, 5 failed (72 % pass).

Table 26: Test incident report INC-01

Field	Content
Summary	500-unit bill charged ₹2,890 instead of ₹2,740 (subsidy withdrawn at exactly 500)
Item / build	Billing module, build 1.3.0+b214, tariff TF-2026.2
Test	TC-BILL-03b, log TL-SYS-01 #6
Inputs	Consumer C100057 (masked), previous 1200, current 1700, on time
Expected / actual	₹2,740 (REQ-06: subsidy only above 500) / ₹2,890
Steps to reproduce	Log in as billing tester → Meter Reading → enter 1700 → Generate Bill → read Total
Severity / priority	Critical (wrong money for every 500-unit consumer) / P1
Evidence	Screenshot bill_C100057.png; log entry; oracle value
Reproducible	Yes, 3 of 3 runs

Table 27: Test incident reports INC-02 and INC-03

Field	INC-02	INC-03
Summary	Retried gateway callback recorded as a second payment	Receipt SMS shows the amount in paise
Test	TC-PAY-02a/02b (#13, #14)	TC-RCP-01a/01b (#16, #17)
Inputs	Txn T3 ₹1,340; sandbox set to retry the callback once	Txn T7 ₹2,740
Expected / actual	Total paid ₹1,340 / ₹2,680	“Rs 2740.00” / “Rs 274000.00”
Severity	Critical (consumer charged twice in records; refund dispute)	Major (wrong message, no money effect)
Reproducible	Yes, 3 of 3	Yes, 3 of 3

2. One week of dashboard figures, weekly status and a test summary report

All 180 functional conditions (120 bill, 40 payment, 20 receipt) were generated and executed on the prototype over a simulated week. Wednesday lost 3 hours of the test server, so 14 conditions were blocked and carried to Thursday. P2 was fixed in build b215 on Thursday.

Table 28: Daily dashboard, week 5 (executed on the prototype)

Day	Planned	Executed	Passed	Failed	Blocked	Cumulative executed	Pass rate
Mon	30	30	22	8	0	30	73 %
Tue	40	40	31	9	0	70	78 %
Wed	40	26	19	7	14	96	73 %
Thu	40	40	31	9	0	136	78 %
Fri	30	30	27	3	0	166	90 %

Day	Planned	Executed	Passed	Failed	Blocked	Cumulative executed	Pass rate
Week	180	166	130	36	14	166 of 180	78 %

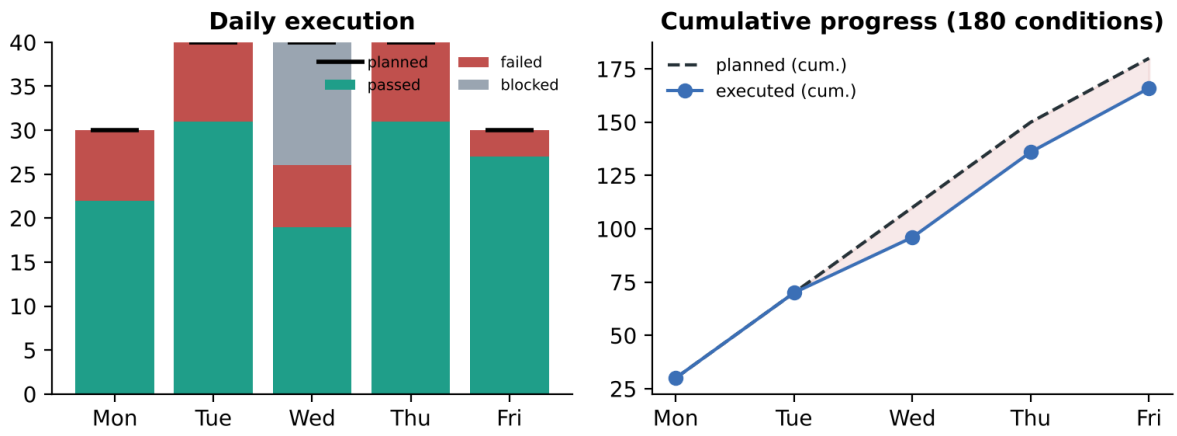


Figure 9: Dashboard charts: daily results and cumulative progress against the plan

Table 29: Weekly status report, week 5 (one page)

Heading	Text
Progress	166 of 180 conditions executed (92 %); 14 lost on Wednesday to server downtime and re-run Thursday
Quality	130 passed; 2 critical defects open (INC-01 500-unit bill, INC-02 duplicate payment); INC-03 fixed in b215
Risks	R7: INC-02 fix promised Monday; if late, payment cycle 2 slips 1 week
Next week	Cycle 2 of payment; tariff change re-test; start load-test scripts
Decision needed	Approve second VM (₹18,000) to stop server sharing

Table 30: Test summary report TSR-SYS-01 (system test level, IEEE 829 structure)

Section	Content
Identifier	TSR-SYS-01, PP-TP-01, builds b214–b215
Variances	Execution started 2 days late (R2); 14 conditions blocked Wednesday; P2 fixed mid-week
Comprehensiveness	180 of 180 conditions designed, 166 executed; REQ-14 load not yet tested; traceability 13 of 14
Summary of results	3 incidents (2 critical open, 1 major closed); 500-unit bill and duplicate payment remain
Evaluation	**Not ready for release.** Billing and payment fail the exit criterion “no open critical defect”
Summary of activities	About 118 h of the 128 h planned for the week used (estimate from the log times); 1 day of server time lost
Approvals	Test Manager; Chief Engineer (IT) – for information

3. What each report tells and hides

Table 31: What each report tells the Chief Engineer, what it hides, and whether it answers “is the portal ready?”

Report	Tells	Hides	Ready?
A Log + incidents	Exactly what failed and how to reproduce it	How much was tested, trends, what was not tested	Cannot answer

Report	Tells	Hides	Ready?
B Dashboard + status	Progress, pass rate, open defects by severity	Whether the failures matter, untested requirements, test quality	Partly (only by severity count)
C Summary report	Variances, coverage, evaluation, recommendation	Day-to-day detail; arrives only at the end of a level	Yes, with evidence

4. Reproducibility of the incident reports

Table 32: Reproducibility from the report alone

Report	Automated replay of the report fields (measured)	Pilot-style one-line report replay (measured)	Classmate reproduced? (to be filled)
INC-01	Yes	No	
INC-02	Yes	No	
INC-03	Yes	No	
Success rate	3 of 3	0 of 3	__ of 3

The automated replay used only the build, inputs and expected value written in each report. All three structured reports reproduced the failure; the pilot-style reports ("bill wrong for some consumers") gave no inputs and reproduced none. The classmate check is still required by the task; the student records it in the last column.

5. Effort, and the risk of figures without context

Table 33: Effort to produce each report (estimate) and risk of misleading

Report	Effort per week	Risk of false comfort
A	Log: 0 extra (written while testing); incidents 20 min each → 1 h	Low – but no overview
B	Dashboard 0 (tool); narrative 1 h	High – Friday pass rate 90 % looks healthy while 2 critical defects are open
C	4 h at the end of each level (3 levels)	Low – evaluation must state open criticals

Context matters: the 500-unit fault made only the 500-unit bill conditions fail. A pass rate in the 90s is quite possible for the billing module while every consumer with exactly 500 units would be over-charged ₹150. Pass rates are therefore always reported **with open critical defects named**.

6. Evaluation – the reporting set

Decision: A + B + C, each for a different reader. Developers get A (incidents with reproduction steps). The test manager and the Chief Engineer get B weekly, with a rule that the first line lists open critical defects. The Chief Engineer and the auditors get C at the end of each level, with a go/no-go evaluation. The extra effort is about 2 h per week plus 4 h per level, which is under 2 % of the capacity in Table 12.

7. Critique of an LLM-written summary report

The log (Table 25) and the incident reports are given to the LLM with the IEEE 829 summary-report headings. Each sentence of its report is then classified against the fact sheet below as correct, wrong, unsupported (no evidence in the log) or invented.

Table 34: Fact sheet for checking an LLM-written summary report

Topic	Fact from the log (reference)	LLM statement and verdict (student fills)
Tests executed and pass rate	18 executed; 13 passed = 72 %	
Severity of the 500-unit failure	Critical: ₹150 over-charge per 500-unit consumer	
Payment verification	Sandbox and simulated callbacks only; not the bank	
Readiness	Not ready: 2 critical open; REQ-14 not tested	
SMS timing (REQ-11)	Not measured	

What to expect and why it matters. Language models tend to soften severities and add a positive conclusion, so the high-risk rows are severity and readiness. An LLM summary may be used as a first draft only: every sentence must be traced to the log, and the evaluation section must be written and signed by the test manager.

8. Supporting evidence for Task 4

- `exp_t34.py` produced the log, the week of dashboard data and the replay results (Annexure F).

Threats to validity

The week of execution was run on a prototype with simulated server downtime; real defect arrival would also include new faults found during the week.

Marking notes and common errors

Common errors: incident reports without the build and inputs; reporting a pass rate without open criticals; a summary report with no evaluation or recommendation.

TASK 5 – Test Policy Development: Compare Test Policy Approaches [7 Marks]

The Corporation has no written testing policy; each project does as it likes. Compare three approaches to developing a test policy.

Technique	Basic idea and key concepts / tools
A. Informal guidelines	A one-page list of good testing practices circulated by e-mail.
B. Formal organisational test policy	A signed policy stating the mission of testing, definitions, principles, test levels, standards, documentation, metrics, quality targets and responsibilities.
C. Policy with strategy and improvement path	A formal policy supported by a test strategy and a process-improvement path aligned to a maturity model (for example, TMMi levels 2 and 3).

- Draft the three policy documents; each must include scope, objectives, roles and the rule for releasing a system to consumers.
- Define at least six measurable policy statements with the way each will be measured.
- Analyze the three policies against the pilot-year problems and show which problem each policy would have prevented.
- Examine how the policy would be applied to a small project (a feedback form) and to a large one (PowerPay), and where it must be tailored.
- Analyze the cost and effort of introducing each policy, and the likelihood that it will be followed.
- Evaluate which approach the Corporation should adopt and justify it.
- Judge the quality of a test policy generated by an LLM.

Evidence Expected: Three draft policies, a list of measurable policy statements, a problem-versus-policy table, and a justified policy choice.

Ans:

A Testing Policy for the State Power Distribution Corporation

Prediction. Informal guidelines (A) will prevent none of the four pilot problems for certain, because nothing in them is checked; the formal policy (B) will prevent most; C adds the means to keep improving but costs the most to introduce.

1. Three draft policies

Table 35: Policy A – informal guidelines (one page, e-mailed)

Section	Text
Scope	All IT projects of the Corporation
Objectives	Test early, test the important things, fix defects before release
Practices	Write test cases before testing; test slab boundaries; retest fixes; keep a defect list; get users to try the system
Roles	Project manager decides how much testing is enough
Release rule	Release when the project manager is satisfied

Table 36: Policy B – formal organisational test policy TP-2026 (6 pages; key clauses)

Clause	Text
1 Scope	All software that computes, collects or reports consumer money or consumer data, and all systems used by more than 1,000 consumers

Clause	Text
2 Mission	Testing provides independent evidence of whether a system meets its requirements and is fit to be released to consumers
3 Definitions	Test levels (unit, integration, system, acceptance); defect severities (critical: wrong money, data loss, security; major; minor); test item; baseline
4 Principles	Risk-based testing; independence of system and acceptance testing; tested item = released item; every fix has a regression test
5 Documentation	Plan, traceability, incident reports, summary report per level (IEEE 829 or tool-generated equivalent)
6 Metrics	Defect removal efficiency, open defects by severity, requirement coverage, reopen rate, fix time
7 Quality targets	DRE ≥ 90 %; reopen ≤ 5 %; no open critical at release
8 Roles	Head of IT owns the policy; test manager owns compliance per project; Chief Engineer signs release
9 Release rule	A system is released to consumers only if all policy statements P1–P8 (Table 38) are met or a signed waiver exists

Table 37: Policy C – TP-2026 + organisational test strategy + improvement path

Part	Content
Policy	Policy B unchanged
Test strategy (8 pages)	Default techniques per risk class; automation of money-related regression; standard environments; tools (TestLink, JIRA, JMeter); data masking rule
Improvement path	TMMi level 2 (managed: policy and strategy, planning, monitoring and control, design and execution, environment) by June 2027; level 3 (defined: organisation, training, life-cycle integration, non-functional testing, peer reviews) by June 2028
Measurement	Quarterly assessment against the TMMi practices; yearly policy review using the metrics of clause 6
Roles	A test process group of 2 people (part-time) owns the strategy and the assessment

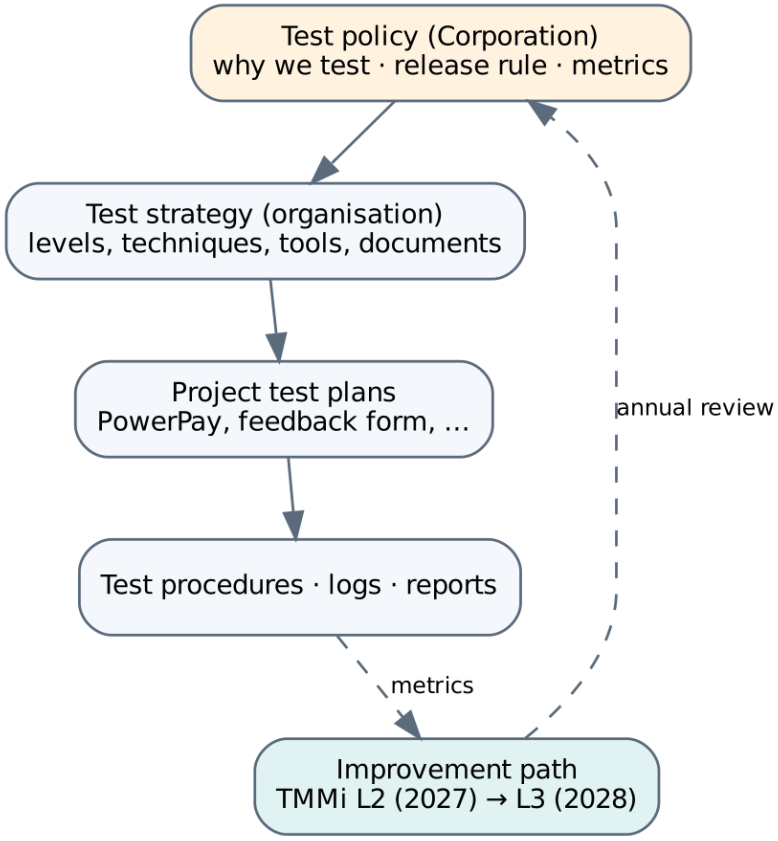


Figure 10: Policy, strategy, plans and the improvement loop (policy C)

2. Measurable policy statements

Table 38: Measurable policy statements and how each is measured

#	Policy statement	Measure and source	Pilot value	Target
P1	No release with an open critical defect	Count of open critical defects at go/no-go (defect tracker query)	Unknown (no tracker query)	0
P2	Every payment and billing feature has automated regression tests	% of REQ-04–11 with ≥ 1 automated test (traceability matrix)	0 %	100 %
P3	The released build is identical to the tested build	SHA-256 of deployed items = release-note baseline (deployment audit log)	Failed	100 % match
P4	Defect removal efficiency ≥ 90 %	Defects found before release ÷ (before + first 90 days after)	52 ÷ 73 = 71.2 %	≥ 90 %
P5	Reopen rate ≤ 5 %	Reopened ÷ fixed defects per release (tracker)	18 %	≤ 5 %
P6	Critical defects fixed within 2 working days	Median time from accepted to fixed for critical	4 days (all)	≤ 2 days
P7	Every requirement is traced to at least one executed test	Requirement coverage from the matrix	Not measured	100 % (or waiver)
P8	A test summary report with evaluation is signed before release	Signed TSR exists in the repository	None	Yes

3. Policies against the pilot-year problems

Table 39: Which pilot problem each policy would have prevented

Pilot problem	A Guidelines	B Formal policy	C Policy + path
Bill wrong at exactly 500 units	Maybe ('test slab boundaries' – not checked)	Yes – P2 regression of billing, P7 coverage	Yes + strategy names BVA for slabs
Payment recorded twice	No	Yes – P2 for payment, P1 blocks release	Yes
Tested build ≠ released build	No	Yes – P3 checksum audit	Yes
Nobody knew how much was tested	No	Yes – P7, P8	Yes + monitoring practice (TMMi 2)
18 % reopen, 4-day fixes	No	Measured and targeted (P5, P6)	Measured, targeted and improved yearly
Problems prevented (of 5)	**0 certain, 1 possible**	**5**	**5**

4. Tailoring – a feedback form versus PowerPay

Table 40: Applying policy B/C to a small and a large project

Policy element	Feedback form (1 developer, 2 weeks, no money)	PowerPay (13 people, 10 weeks, money of 3 lakh consumers)
Scope clause 1	Out of scope for P2–P4 (no money); in scope for P1, P3, P8	Fully in scope
Documents	1-page plan + checklist + tool log	Risk-based plan, traceability, TSR per level
Independence	Peer test by another developer	Separate test team; acceptance by billing section
Metrics	Open defects only	All of P1–P8
Release signer	IT manager	Chief Engineer (IT)

Tailoring is allowed only through a recorded **tailoring sheet** that names each relaxed statement and its reason. A small project may drop P2, P4 and P7, but never P1 or P3.

5. Cost, effort and likelihood of being followed

Table 41: Cost to introduce and likelihood of compliance (estimates)

Item	A	B	C
Writing and approval	4 h	40 h + 2 approval meetings	40 h + 60 h strategy + 30 h assessment design
Training all IT staff (60 people × 2 h)	0	120 h	120 h + 16 h test-group training
Running cost per project	0	≈ 3 % of test effort (reports, audits)	≈ 4 % + quarterly assessment 16 h
External assessment (optional)	–	–	≈ ₹2–3 lakh for a TMMi assessment (not from the project tool budget)
Likelihood followed after 1 year	Low – nothing checked	Medium–high – release rule enforced	High – audited and improved

6. Evaluation – the policy to adopt

Decision: adopt C, introduced in two steps. Policy B with the release rule P1–P8 becomes binding for PowerPay now, because it alone prevents all five pilot problems and every statement is measurable from tools the project already has. The strategy and the TMMi path follow over the next year, so that the policy improves the next projects instead of staying a document. The extra cost of C over B (about 100 h in the first year) is small against one repeat of the pilot's 21 field defects, which consumers and staff found at roughly 5 times the cost of finding them in test (assumption from common industry ratios).

7. Judging an LLM-generated test policy

Table 42: Criteria for judging an LLM-generated policy

Criterion	Test applied	Pass rule
Specific to the Corporation	Does it mention consumer money, tariff, the release signer, 3 lakh consumers?	All four present
Measurable	Can each statement be checked with a query or document? Compare with Table 38	≥ 6 measurable statements
Consistent definitions	Severity and test-level definitions match clause 3	No conflicts
Release rule	Is there a clear go/no-go rule with a signer?	Present
Generic padding	Phrases like 'quality is everyone's responsibility' without a measure	Removed

LLM policies are usually well written but generic: they read like a textbook and rarely contain a measurable release rule. The draft may be used for wording; the statements, targets and roles must come from the Corporation's own data, as in Table 38.

8. Supporting evidence for Task 5

- Pilot DRE = 52 ÷ (52 + 21) = 71.2 %; critical defects that escaped to the field: 6 of 21.

Threats to validity

Compliance likelihood is a judgement; it should be measured after one year by the share of projects passing the release audit.

Marking notes and common errors

Common errors: policy statements that cannot be measured ("test thoroughly"); no release rule; confusing policy (why and what) with strategy (how).

TASK 6 – Test Team Organisation: Compare Team Structures [4 Marks]

PowerPay needs an organisation for its 4 testers and the people who support them. Compare three team structures.

Technique	Basic idea and key concepts / tools
A. Test-level functional teams	Separate teams for unit-integration testing, system testing and acceptance testing, each with its own lead.
B. Module-aligned cross-functional teams	Each team of developers, a tester and an analyst owns a module (billing, payment, consumer services).
C. Vendor test team with internal acceptance team	An outsourced vendor performs system and performance testing; an internal team defines acceptance tests and reviews the vendor's results.

- Draw the organisation chart for each structure, show reporting lines, the interfaces to the 5 developers and the billing-section coordinator, and assign the 13 people.
- Simulate three situations in each structure and record the decision path and the delay.
- Analyze communication paths, independence, domain knowledge, tool support and the single points of failure.
- Examine which structure best supports the 10-week schedule and which creates the largest management overhead.
- Evaluate the structure you would recommend and justify it with the results of your simulation.
- Critique the proposal to give the vendor team the right to approve the release of the portal.

Evidence Expected: Three organisation charts, a situation-response table, and a justified team structure.

Ans:

Organising the PowerPay Test Team

Prediction. Module-aligned teams (B) will give the shortest decision paths; the vendor structure (C) will be slowest for urgent changes; test-level teams (A) will be the most independent.

1. Organisation charts and assignment of the 13 people

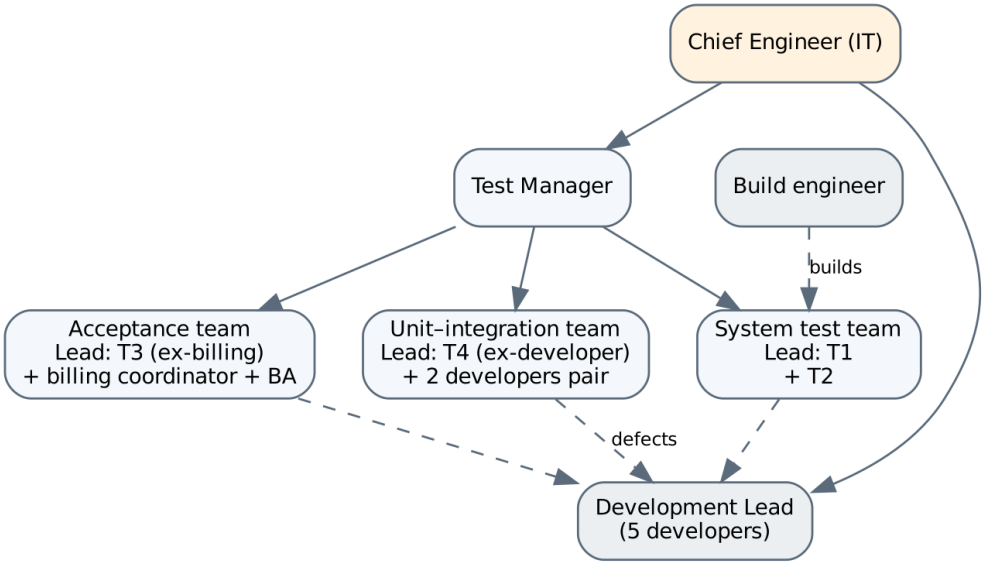


Figure 11: Structure A – test-level functional teams

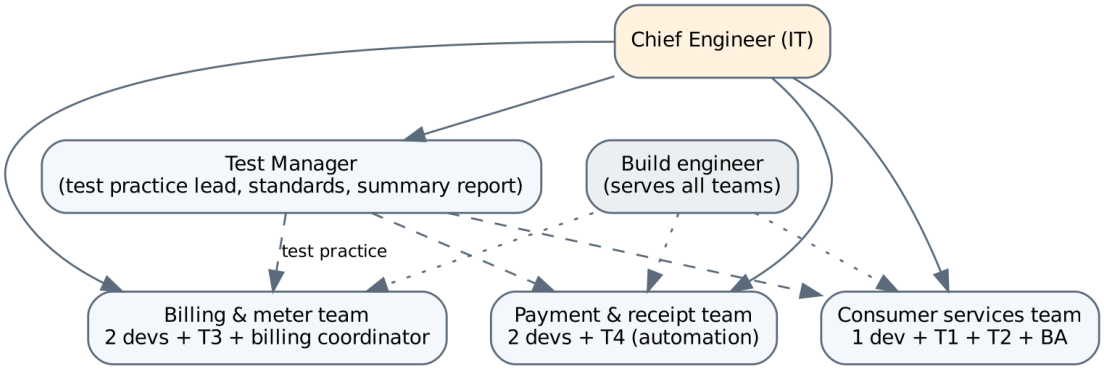


Figure 12: Structure B – module-aligned cross-functional teams with a test practice lead

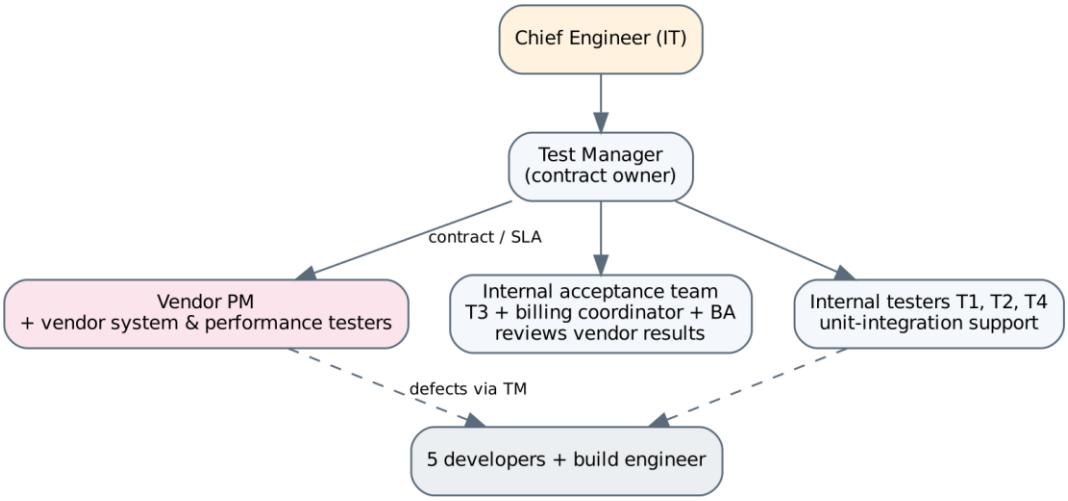


Figure 13: Structure C – vendor test team with an internal acceptance team

Table 43: Assignment of the 13 people in each structure

Person	A Test-level	B Module-aligned	C Vendor + internal
Test manager	Head of all three teams	Test practice lead, summary reports	Contract owner, reviews vendor
T1, T2 (graduates)	System test team	Consumer services team	Internal unit-integration support
T3 (ex-billing)	Acceptance lead	Billing & meter team	Internal acceptance lead
T4 (ex-developer)	Unit-integration lead	Payment & receipt team (automation)	Internal unit-integration support
5 developers	One development team	2 billing, 2 payment, 1 consumer services	One development team
Build engineer	Serves system team	Serves all teams	Serves vendor and developers
Business analyst	Acceptance team	Consumer services team (and rules for all)	Internal acceptance team
Billing coordinator	Acceptance team	Billing & meter team	Internal acceptance team
Vendor	–	–	PM + 3 testers (system, performance)

2. Simulation of three situations

Each situation was traced through each chart as a decision path (who must act, in order). The delay is the sum of the typical waiting time for each role (assumed: tester 0.5 h, developer 2 h, team/module lead 1–2 h, test manager 4 h, analyst 2 h, billing coordinator 4 h, vendor PM 8 h, a contract change request 40 h).

Table 44: Situation-response table: decision path, hops and delay (modelled)

Situation	Str.	Decision path	Hops	Delay (h)
S1 Defect dispute	A	tester → team lead → test manager → dev → analyst → test manager	6	14.5
	B	tester → dev → analyst	3	4.5
	C	vendor tester → vendor PM → test manager → dev → analyst → test manager	6	24

Situation	Str.	Decision path	Hops	Delay (h)
S2 Urgent tariff change	A	billing coord → analyst → test manager → team lead → tester → team lead → test manager	7	18.5
	B	billing coord → analyst → module lead → tester → dev	5	9.5
	C	billing coord → analyst → test manager → vendor PM → contract change → vendor tester → test manager	7	66
S3 Tester absent for a week	A	team lead → test manager → tester	3	6.5
	B	module lead → test manager → tester	3	5.5
	C	vendor PM → vendor tester	2	12

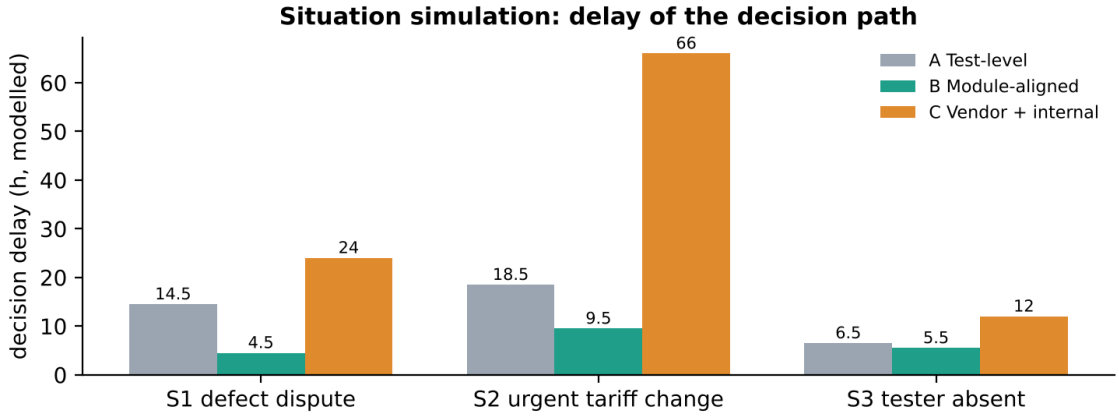


Figure 14: Decision delay per situation and structure

Structure B decided all three situations fastest (total 19.5 h, against 39.5 h for A and 102 h for C). The urgent tariff change took **66 h in C** because the vendor contract needed a change request, which is longer than the 3 days available. The prediction is supported.

3. Communication, independence, domain knowledge, tools and single points of failure

Table 45: Structural analysis of the three team structures

Criterion	A Test-level	B Module-aligned	C Vendor + internal
Communication paths for a release decision	TM + 3 leads + dev lead = 5 people → 10 paths	TM + 3 module teams = 4 → 6 paths	TM + vendor PM + internal lead + dev lead = 4 → 6 paths, one across a contract
Independence of testing	High (separate teams)	Medium (tester inside the team; TM reviews)	High for system test
Domain knowledge (billing)	Concentrated in acceptance	Spread: T3 and coordinator in billing team	Low in vendor; internal only
Tool support	Each team may choose its own	One shared tool via the practice lead	Vendor tools; data transfer needed
Single point of failure	Test manager (all decisions)	Build engineer serves 3 teams	Vendor PM; contract

4. Schedule support and management overhead

B best supports the 10-week schedule: three teams work in parallel on their modules from week 1, and the urgent tariff change is handled inside the billing team in 9.5 h. C carries the largest management overhead: vendor selection and contract take 2–3 weeks of a 10-week project, every scope change is a contract change, and the test manager must review all vendor results (about 6 h

per week, estimate). A needs the test manager in most decisions, which makes the manager a bottleneck (4 of the 6 steps in S1).

5. Recommended structure

Decision: structure B, with the test manager as an independent test practice lead. It had the shortest decision paths in all three situations and puts billing knowledge (T3, the coordinator) where the 500-unit fault was. To keep the independence that A offers, the test manager owns the system-level summary report, the release recommendation and a final cross-module regression run, so testers do not judge their own team's work alone. The build-engineer single point of failure is covered by T4, who is trained to produce baselined builds.

6. Critique – should the vendor approve the release?

No. Release approval is an acceptance decision about consumer money and belongs to the Corporation. The vendor has a commercial interest in closing the contract, sees only the tests it designed, and does not know the billing rules as well as the billing section. Under policy clause 8 the Chief Engineer signs the release. The vendor's role is to supply evidence (its test summary report), which the internal acceptance team reviews against the traceability matrix.

7. Supporting evidence for Task 6

- `calc_pp.py` contains the role waiting times, the decision paths and the delay sums.

Threats to validity

Waiting times are assumptions; real delays depend on people and workload. The ranking of B first held for any waiting times where the vendor and contract roles are slower than internal roles.

Marking notes and common errors

Common errors: charts without the developer and coordinator interfaces; giving the vendor release authority; no single point of failure analysis.

TASK 7 – Skills of a Test Specialist: Compare Skill-Building Strategies [5 Marks]

The testers recruited for PowerPay have different backgrounds: two are fresh graduates, one comes from billing operations, and one is a former developer. Compare three strategies for building the skills of the test specialists.

Technique	Basic idea and key concepts / tools
A. Classroom training and certification	A short course on testing fundamentals followed by a certification examination (for example, ISTQB Foundation).
B. Mentoring and pair testing	Each new tester is paired with an experienced tester for the first four weeks; tests and reports are reviewed together.
C. Skills matrix with rotation	A skills matrix is kept for the team; testers rotate between test design, automation, defect analysis and reporting and are assessed against the matrix.

- Prepare a skills matrix for a test specialist and score the four testers using the profiles given.
- Prepare a 4-week skill-building plan for each strategy for the two fresh graduates, with activities, hours and assessment.
- Analyze the skills each strategy builds and the skills it leaves undeveloped; relate the gaps to the pilot-year defects.
- Analyze the cost and time of each strategy against the ₹1,50,000 tool budget and the effort of the existing team.
- Evaluate which strategy or combination the test manager should use and justify it with measurable skill gains.
- Judge how the growing use of AI tools changes the skills required of a test specialist and which skills remain essential.

Evidence Expected: A skills matrix with scores, three skill-building plans, a cost and gap table, and a justified strategy.

Ans:

Building the Skills of the PowerPay Test Specialists

Prediction. Classroom training (A) will raise test-design knowledge but not billing knowledge; mentoring (B) will close the largest gap for the graduates; a combination will be best.

1. Skills matrix and scores

Table 46: Skills matrix of a test specialist (1 = aware, 3 = works alone, 5 = can teach)

Skill area (examples)	T1	T2	T3	T4	Target
Test design techniques	2	2	2	3	4
Tool / automation	2	1	1	5	3
Billing domain	1	1	5	2	3
Analytical / defect analysis	2	2	3	4	4
Communication and reporting	3	2	4	2	3
Planning and estimation	1	1	2	3	2
Gap to target (sum)	8	10	5	3	0

Profiles: T1 and T2 are fresh graduates (CSE, ECE); T3 worked 6 years in billing operations; T4 was a Java developer for 4 years. Scores come from a 1-hour practical assessment (design

boundary tests for REQ-05/06, write one incident report, read a JIRA dashboard) and a 15-minute interview. Example skills per area: test design (BVA, decision tables, state models); tools (TestLink, JIRA, pytest/Selenium, JMeter); billing domain (slabs, subsidy, late fee, meter rollover); analysis (root cause, reproduction, reading logs); communication (incident reports, status reports); planning (estimation, risk).

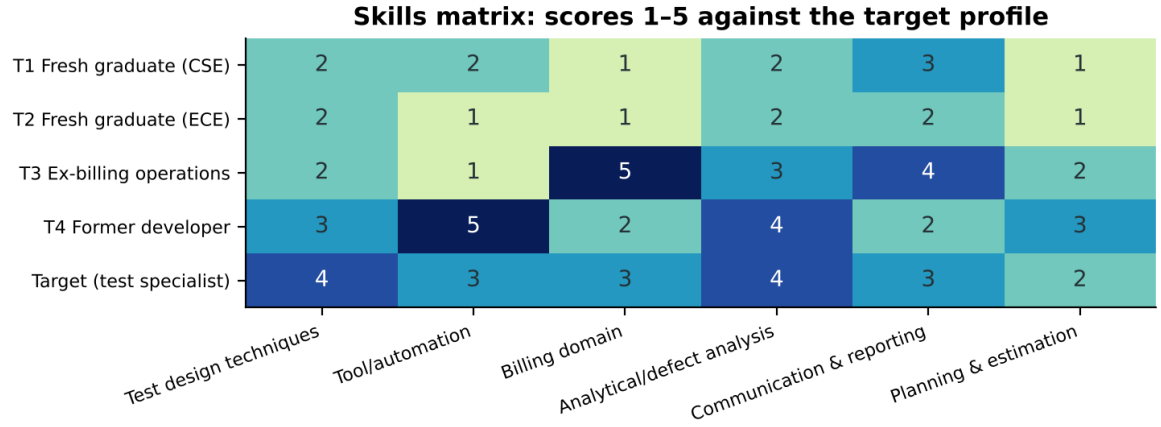


Figure 15: Skills matrix heat map against the target profile

2. Four-week plans for the two graduates

Table 47: Four-week skill-building plans for T1 and T2 (hours per graduate)

Week	A Classroom + ISTQB	B Mentoring + pairing	C Matrix + rotation
1	Fundamentals course 20 h (life cycle, static testing, techniques)	Pair with T4 on bill test design 10 h; daily review 0.5 h	Rotation 1: test design under T3, 6 h; self-assessment
2	Techniques workshop 12 h; mock exams 4 h	Pair with T3 on billing rules and acceptance cases 10 h	Rotation 2: automation with T4, 6 h
3	Revision 4 h; ISTQB Foundation exam	Execute and report alone; reports reviewed by mentor 10 h	Rotation 3: defect analysis (bisect, reproduction) 6 h
4	Return to project; no follow-up	Lead a test design session; mentor observes 10 h	Rotation 4: status reporting; matrix re-score 6 h
Hours	40	40 (+ 30 mentor hours each)	24 (+ 12 senior hours each)
Assessment	Exam score (pass ≥ 65 %)	Review checklist of 5 test designs and 3 incident reports	Re-score on the matrix by TM

3. Skills built and gaps left – related to pilot defects

Table 48: Skills each strategy builds and leaves undeveloped

Strategy	Builds	Leaves undeveloped	Pilot defect it relates to
A	Vocabulary, techniques in theory (BVA, decision tables)	Billing domain, tool use, writing real reports	500-unit fault: knowing BVA is not enough without knowing REQ-06 is a boundary
B	Domain rules, practical design, report quality	Breadth (depends on the mentor); planning	500-unit fault and duplicate payment: mentors show where money bugs hide
C	Breadth across activities, self-awareness of gaps	Depth in any one skill in 4 weeks	‘Nobody knew how much was tested’: rotation through reporting

4. Cost and time

Table 49: Cost and time of each strategy (estimates; ISTQB fee and course cost are assumptions)

Strategy	Rupees	Graduate hours	Senior hours taken from the project	Gap closed (2 graduates)
A Classroom + ISTQB	₹31,000	80	6	39 %
B Mentoring + pairing	₹0	80	60	61 %
C Skills matrix + rotation	₹0	48	24	56 %
A+B combined	₹31,000	120	60	78 %

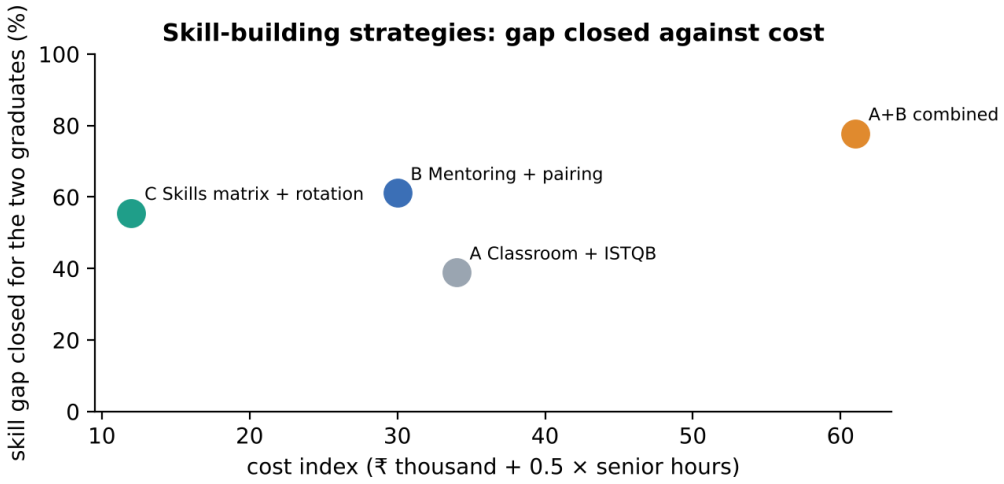


Figure 16: Share of the skill gap closed against a cost index

Classroom training costs ₹31,000 (2 × ₹12,000 exam and 2 × ₹3,500 course material), about 21 % of the ₹1,50,000 tool budget. That would leave ₹1,19,000 for the second VM (₹18,000) and tools, so it fits only if the open-source tools in R10 are used. Mentoring costs no money but takes 60 h of T3's and T4's time, which is 5 % of the 1,200 h capacity and fits in the 481 h reserve of Table 12.

5. Recommended strategy

Decision: B for the first four weeks, then C for the rest of the project; ISTQB (A) only after the project, funded from the training budget, not the tool budget. B alone closed about 61 % of the graduates' gap, nearly as much as the A+B combination (78 %) at half the cost index, and it is the only strategy that builds billing knowledge, which is where the pilot's worst defect was. C keeps the matrix up to date and spreads skills so that a tester's absence (situation S3) does not stop a module. Success measure: each graduate reaches 3 or more on test design and billing domain by week 5, checked by re-scoring the matrix.

6. AI tools and the skills of a test specialist

Table 50: How AI tools change the required skills

Skill	Effect of AI tools	Remains essential because
Writing test cases from text	Faster drafts	Drafts miss boundaries (Task 1: half-to-even rounding); must be checked against the SRS
Domain knowledge	Not replaced	Only the domain tells you 500 units is a boundary

Skill	Effect of AI tools	Remains essential because
Oracle and expected results	Risky to delegate	A wrong expected value turns a defect into a pass
Critical review of generated output	New, essential skill	Find invented links, wrong severities, generic plans (Tasks 2–5)
Automation and data skills	More important	AI-generated scripts must be run, debugged and maintained
Communication and judgement	Unchanged	The go/no-go evaluation is a human decision with accountability

7. Supporting evidence for Task 7

- Scores, gains and costs are in `calc_pp.py`; the gap-closed percentages are computed from them.

Threats to validity

Expected gains per strategy are estimates; they must be confirmed by re-scoring in week 5.

Marking notes and common errors

Common errors: matrices without a target; using the tool budget for certification without saying so; ignoring the cost of senior testers' time.

TASK 8 – Integrated Test Process Evaluation: Compare Test Documentation Sets [4 Marks]

Combine your choices from Tasks 1–7 into one test process for PowerPay. Compare three documentation sets for the whole process.

Technique	Basic idea and key concepts / tools
A. Full IEEE 829-style document set	Plan, design specification, case specification, procedure, log, incident report and summary report written in full for every test level.
B. Lean tool-generated documents	Test cases, execution logs, incident reports and reports are generated by the test-management tool; narrative documents are limited to a plan and a summary.
C. Risk-tailored hybrid set	Full documents for the payment and billing modules; lean documents for the rest.

- Produce the documents of each set for the bill-generation feature and estimate the effort for each set in person-hours.
- Draw the PowerPay test process as a flowchart (plan, identify items, design, execute, report, evaluate), showing which document is produced at each stage and who owns it.
- Analyze the three sets on completeness, auditability, effort, readability and ease of change.
- Examine which problems of the pilot year each document set would have exposed.
- Evaluate which set the Corporation should adopt and justify it; define one rule for deciding when a document may be skipped.
- Judge whether documents generated by an AI tool from the test cases can be accepted for audit without review.

Evidence Expected: Documents from the three sets for one feature, a process flowchart with document ownership, an effort table, and a justified documentation set.

Ans:

One Test Process and the Right Amount of Documentation

Prediction. The hybrid (C) will cost about half of the full set and keep full auditability where money is computed.

1. Documents of each set for the bill-generation feature, and effort

Table 51: Documents produced for the bill-generation feature in each set, with effort in person-hours (estimate)

Document	A Full IEEE (h)	B Lean (h)	C Hybrid – billing is 'full' (h)	Produced in this answer
Test plan	8	6	8	Table 11 + appendix
Test design spec	4	– (in tool)	4	Table 18
Test case spec	6	2	6	Table 19
Test procedure	3	– (in tool)	3	Table 20
Test log	3	0.5	3	Table 25
Incident reports	3	1	3	Table 26, Table 27
Summary report	4	2	4	Table 30
Total for one feature	**31**	**11.5**	**31**	

Table 52: Documentation effort for the whole PowerPay process (6 modules, 3 test levels; estimate)

Set	How it is built	Person-hours	% of 1,200 h capacity
A Full IEEE	Plan + full TDS/TCS/TP/log/incidents for 6 modules + 3 summary reports	134	11.2 %
B Lean	Plan + tool-generated cases, logs, incidents for 6 modules + 3 short summaries	33	2.8 %
C Hybrid	Plan + full set for billing and payment + lean for 4 modules + 3 summaries	72	6.0 %

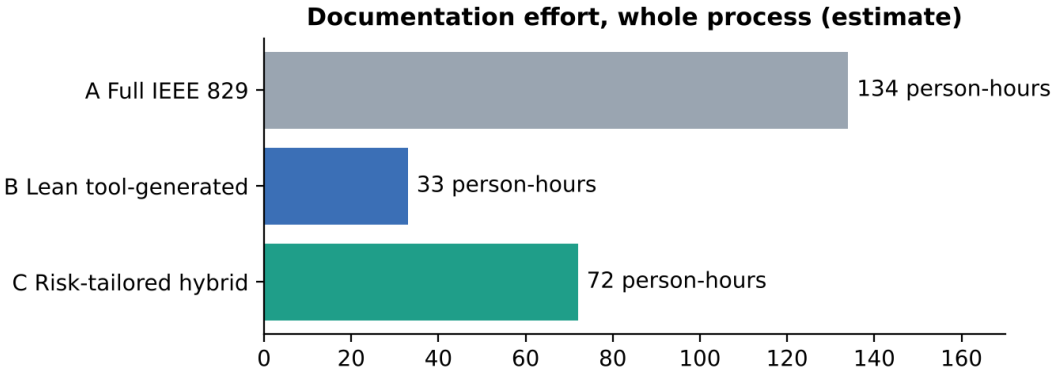


Figure 17: Documentation effort of the three sets

2. The PowerPay test process with document ownership

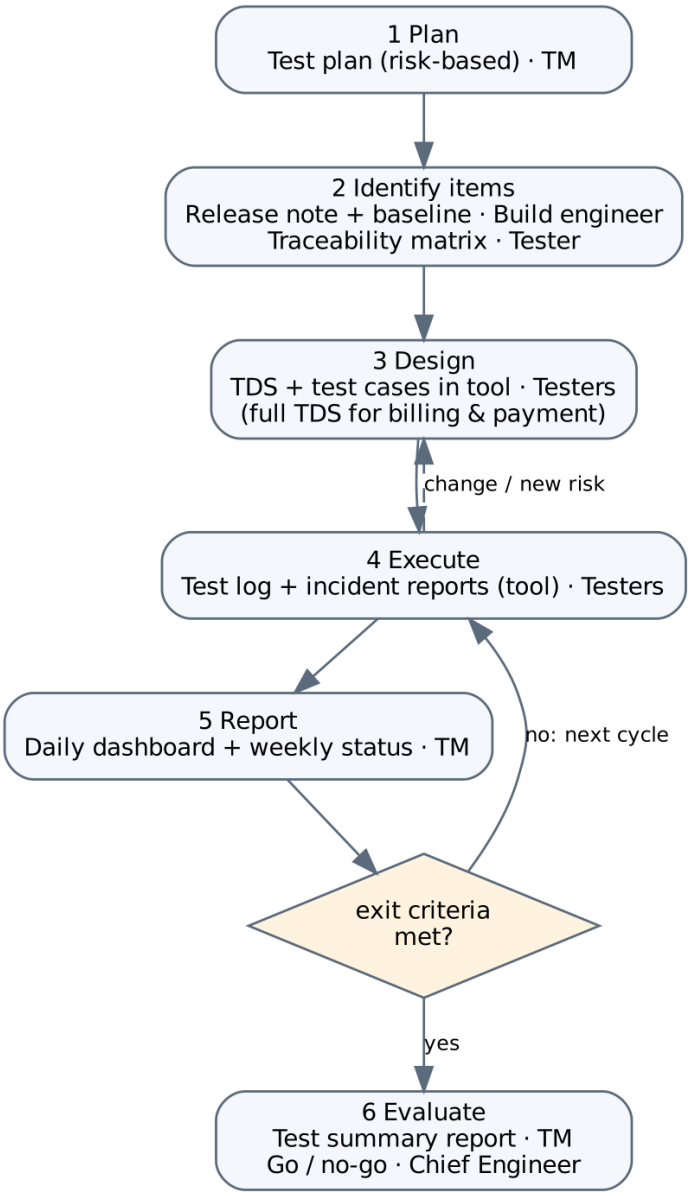


Figure 18: PowerPay test process: stage, document produced and owner

Table 53: Stage, document and owner

Stage	Document(s)	Owner	Chosen in
1 Plan	Risk-based test plan with IEEE appendix	Test manager	Task 2
2 Identify items	Release note + baseline tag + checksums; traceability matrix	Build engineer; testers	Task 3
3 Design	TDS/TCS (full for billing and payment), test cases in tool	Testers (module teams)	Tasks 3, 8
4 Execute	Test log, incident reports (tool), regression tests with fixes	Testers; developers for fixes	Tasks 1, 4
5 Report	Daily dashboard, weekly status	Test manager	Task 4

Stage	Document(s)	Owner	Chosen in
6 Evaluate	Test summary report, go/no-go against policy P1–P8	Test manager; Chief Engineer signs	Tasks 4, 5

3. Analysis of the three sets

Table 54: Completeness, auditability, effort, readability and ease of change (1 = poor, 5 = good)

Criterion	A Full	B Lean	C Hybrid
Completeness	5	3	4
Auditability	5	3 (depends on tool exports)	5 for money modules, 3 elsewhere
Effort	1 (134 h)	5 (33 h)	3 (72 h)
Readability for the Chief Engineer	2 (long)	4	4
Ease of change (week-6 tariff change)	2 (many documents)	5	4
Total (25)	**15**	**20**	**20 (and 5 where it matters)**

4. Pilot problems each set would have exposed

Table 55: Pilot problems exposed by each documentation set

Pilot problem	A Full	B Lean	C Hybrid
500-unit bill	Yes (TDS forces boundary list)	Only if the case was written	Yes (full TDS for billing)
Duplicate payment	Yes	Only if the case was written	Yes (full TDS for payment)
Tested ≠ released build	Yes (item transmittal report)	No (tool shows build field only)	Yes (transmittal for each build)
How much testing was done	Yes (summary report)	Yes (tool metrics)	Yes

5. Evaluation and the skip rule

Decision: C, the risk-tailored hybrid. It exposes all four pilot problems, as the full set does, for about half its effort, and keeps the auditable trail exactly where consumer money is computed. **Rule for skipping a document:** a document may be skipped only if (1) the module has no critical-severity requirement, (2) the same information is held in the test-management tool and can be exported with its build id, and (3) the test manager records the skip in the plan's tailoring sheet. Billing and payment documents can never be skipped.

6. Can AI-generated documents be accepted for audit without review?

No. An audit document is evidence that something happened. A tool can generate a test case specification or a summary from test cases, but it cannot know whether the tests were run, on which build, or whether the expected values are right. AI-generated documents are acceptable as drafts if (a) every number is generated from the tool's execution records, not written by the model, (b) a named tester reviews and signs them, and (c) the build id and checksum are attached (Task 3). Without review they would repeat errors such as wrong severities or invented results (Task 4).

7. Supporting evidence for Task 8

- Effort figures are produced by `calc_pp.py` (DOC_H, DOC_TOTAL).

Threats to validity

Hours per document are estimates from the documents written for this answer; a team new to IEEE 829 would take longer.

Marking notes and common errors

Common errors: a flowchart without owners; a skip rule that depends on opinion; accepting tool output as audit evidence without a build id.

TASK 9 – Re-evaluate and Redesign the Test Process [8 Marks]

Changed Requirement: PowerPay is chosen for the whole state with 12 lakh consumers, includes smart-meter data upload every 15 minutes, and must be tested in 4 weeks instead of 10. The team is now split across two offices, one tester is withdrawn, and a data-protection law requires that consumer data used in testing be masked and that every test result be auditable.

Technique	Basic idea and key concepts / tools
A. Retain the process and cut scope by risk	Keep the original plan, documents and team; reduce the features tested by risk ranking.
B. Modify the process (iterative plans, embedded testers, tool-based reporting)	Short iteration plans, testers embedded with module teams in each office, and a shared test-management tool with live reporting.
C. Redesign the organisation and policy	A central test group with a new policy, standard lightweight documents generated from the tool, and automated evidence capture for audit.

- 9.1 Impact Analysis [2 Marks]: invalid assumptions; effort for about 400 conditions against 3 testers for 4 weeks; masking of test data and documents.
- 9.2 Evaluation of the Original Process [2 Marks]: what remains valid and what becomes unsafe; exit criteria under the shorter time; the least trustworthy measure.
- 9.3 Compare and Adapt [2 Marks]: compare A, B and C; decide; revised process flowchart and plan outline; revised exit criteria and daily reporting set.
- 9.4 Evidence-Based Defence [2 Marks]: defend the decision; what is given up; one condition under which it fails.

Ans:

Re-evaluating PowerPay for 12 Lakh Consumers, Smart Meters and 4 Weeks

Prediction. Retaining the process (A) will fit the time only by dropping more than half of the conditions; only a redesign with automated evidence will satisfy the audit requirement.

9.1 Impact analysis

Table 56: Assumptions of the original plan that are no longer valid

Assumption	Original	New	Effect if the original plan is used unchanged
Duration	10 weeks	4 weeks	Cycle 3 and acceptance never start; 3 cycles impossible
Team location	One office	Two offices	Module teams split; daily stand-up and pairing break; S1-type disputes take longer
Testers	4	3	Capacity 1,200 h → 360 h
Volume	3 lakh, monthly reading	12 lakh, 96 readings per meter per day	No test for 20,000 readings/s bursts; REQ-14 load target wrong
Data	Masked extract (policy)	Masking is law; results auditable	Unmasked screenshots in incident reports become illegal
Reporting	Weekly status	Need daily evidence	Problems seen a week late in a 4-week project

Table 57: Effort for about 400 conditions against the reduced capacity (calculated)

Item	Calculation	Person-hours
Design	400 × 0.4	160
Execution	400 × 0.25 × 3	300

Item	Calculation	Person-hours
Retest 20 %	0.20 × 300	60
Management 15 %	0.15 × 520	78
Total		**598**
Capacity	3 testers × 4 weeks × 30 h	360
Shortfall	598 − 360	**238 h (40 % of the need)**

Smart-meter load: 12,00,000 meters × 96 readings per day = 115,200,000 readings per day, an average of 1,333 per second. If every meter uploads in the first minute of each 15-minute slot, the peak is 20,000 readings per second. The prototype validator (format, monotonic reading, tamper flag) checked 120,001 readings in 0.08 s (1,591,465 per second on one core) and rejected 241 (tamper flags and one decreasing reading). Validation logic is therefore not the bottleneck; the database writes and the network are, and they need a load test that the original plan did not contain.

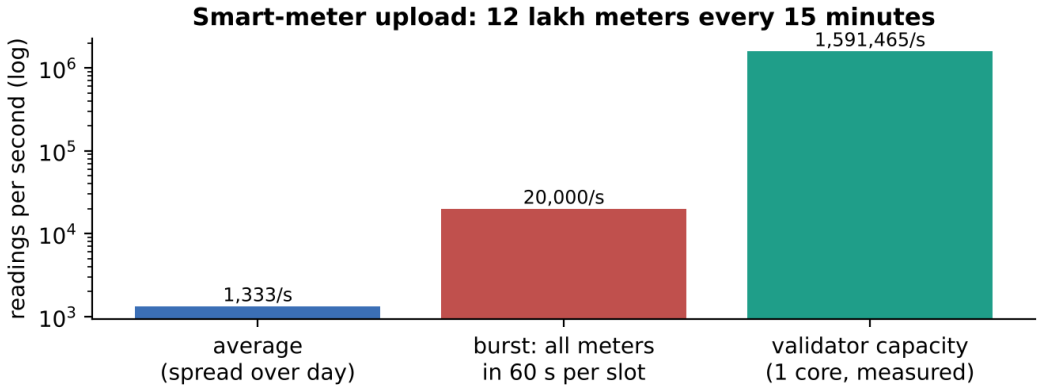


Figure 19: Smart-meter reading rates against the measured validator capacity

Table 58: Data-protection rule applied to test data and documents

Item	Rule	How (implemented in exp_t9.py)	Verified (20,000 records)
Consumer number	Replace	Keyed HMAC → 12 digits starting with 9	0 field collisions; unique: True
Name	Replace	Keyed token NAME-xxxxxxxxxx	0 field collisions
Mobile	Replace	Digit substitution into the reserved range 90000xxxxx	0 field collisions
E-mail, UPI ID	Replace	Keyed token @masked.invalid / @test	0 + 0 field collisions
Address	Generalise	Keep ward only (outage tests need it)	Street and door number removed
Meter number	Replace	Keyed token SMxxxxxxxx	–
Units, bill amounts	Keep	Needed for billing tests; not personal on their own	–
Incident reports and screenshots	Scrub	Regex scrubber for 12-digit numbers, mobiles, IDs; names from the masked list	See example below

Masking 20,000 records took 0.22 s, so a full 12-lakh extract takes about 13 s. Tokens are deterministic for one key, so joins between bills and payments still work. Three scans verified the result. The **field-level scan** found 0 masked values equal to an original value. A **naive substring scan**, the kind of text search an auditor might run, flagged 1 hit: the masked consumer number 912688972582 contains the 10 digits 9126889725, which happen to be a real consumer's mobile number. This is a coincidence of digit sequences, not a mapped value. A **boundary-aware scan**

(a mobile must not sit inside a longer number) found 0 hits. Lesson for the evidence rules: the leak scan uses field-level and boundary-aware matching, and any substring flag is reviewed and recorded rather than silently ignored. The scrubber turned the raw incident text into:

```
INC-01: consumer <CONSUMER-NO> (<NAME>, mobile <MOBILE>) at <ADDRESS>, Ward 24 billed Rs 2890 for 500 units;
UPI <VPA/EMAIL>
```

9.2 Evaluation of the original process

Table 59: Original process checked against the new constraints

Element	Status	Reason
Risk-based plan structure (Task 2)	Valid	Risk table re-ranks; smart-meter and data risks added
Effort and schedule	Unsafe	Shortfall 238 h
Baseline + traceability (Task 3)	Valid	Even more needed for audit
Reporting A+B+C (Task 4)	Partly	Weekly status too slow; summary report must be generated
Policy P1–P8 (Task 5)	Valid; add data rule	Needs P9 masking and P10 auditable evidence
Module-aligned teams (Task 6)	Partly unsafe	Two offices; one tester fewer: consumer services team has no tester
Mentoring (Task 7)	Unsafe	No time for 4 weeks of pairing
Hybrid documents (Task 8)	Partly	Full TDS writing for 2 modules too slow; generate from the tool
Exit criteria	Unsafe unchanged	See below

Exit criteria under the new conditions. The original exit criteria (all planned cases executed, no open critical, 95 % pass) assumed 3 cycles. With 4 weeks, “all planned cases executed” would either never be met (and be waived under pressure) or be met by running each case once with no regression after fixes. A 95 % pass rate over 400 conditions still allows 20 failing conditions; with 12 lakh consumers, one failing billing condition can affect tens of thousands of bills. The criteria must therefore be **risk-weighted**: 100 % of critical-risk conditions passed on the release candidate, rather than a global pass rate.

Least-trusted measure: the cumulative pass rate. In 4 weeks most conditions run once, and blocked or skipped conditions do not lower it. In Table 28 the week’s pass rate rose on Friday while two critical defects stayed open. Under compression, it rewards running easy tests.

9.3 Compare and adapt

Table 60: Strategies A, B and C compared (effort from calc_pp.py; other scores 1–5)

Criterion	A Retain + cut	B Modify	C Redesign
Effort vs 360 h	239 h – fits, but 240 of 400 conditions dropped	307 h – fits (reuse, 2 cycles, 50 % automated)	304 h – fits (60 % automated, evidence set-up included)
Risk coverage	2 – smart-meter and load mostly dropped	4	5 – central risk list across offices
Audit evidence	2 – manual documents	3 – tool logs, no signed evidence	5 – automated evidence per run (build, checksum, result)
Cost	5	4	3 – evidence capture set-up 24 h; policy update
Team effectiveness across two offices	2	3 – embedded testers isolated per office	4 – one group, one tool, daily plan
Total (25)	**13**	**18**	**22**

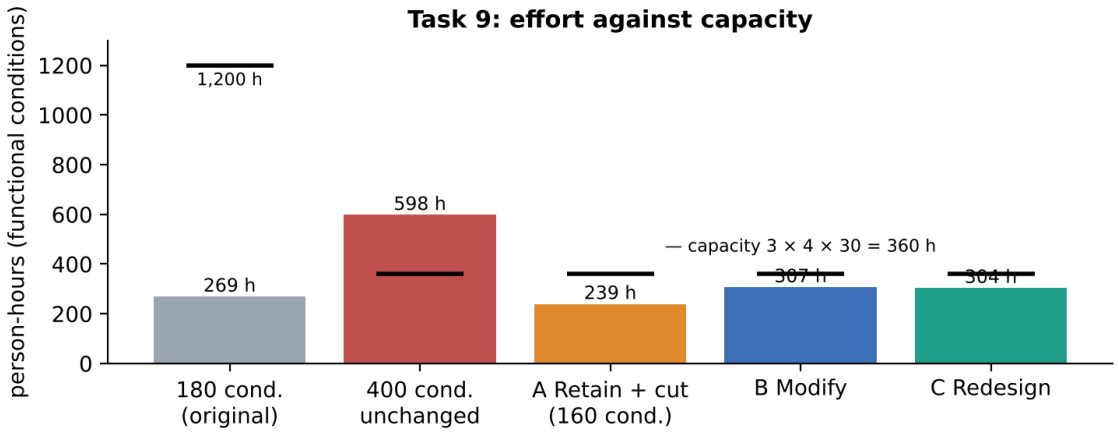


Figure 20: Effort of each strategy against the 360 h capacity

Decision: REDESIGN (strategy C). The law makes auditable, masked evidence a requirement, not an option, and only C produces it automatically. B fits the hours equally well but would leave audit evidence to manual exports from a tool that both offices edit.

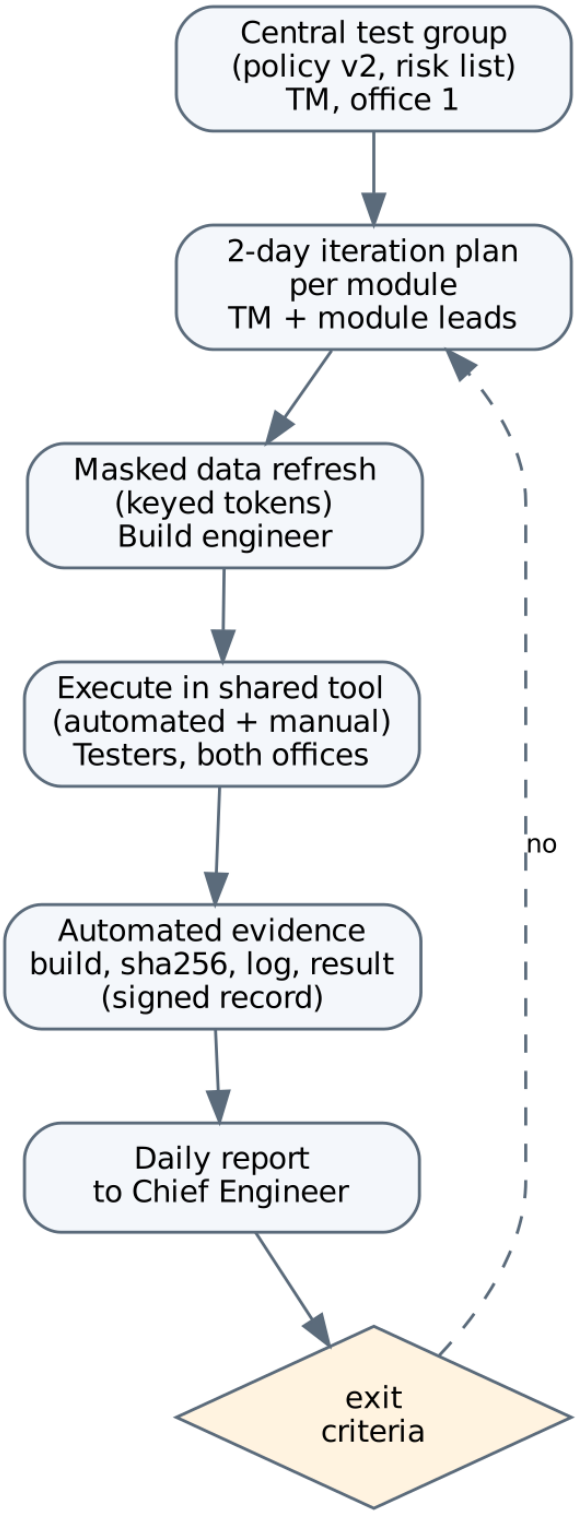


Figure 21: Revised PowerPay test process (redesign)

Table 61: Revised plan outline (4 weeks)

Stage	What it does	Owner	Why in this order
1 Central risk list	Re-rank risks: billing, payment, smart-meter upload, data masking, load	Test manager (central group)	Decides what the 360 h are spent on
2 Two-day iteration plans	Each module gets 2-day goals, alternating offices	TM + module leads	Short feedback in a 4-week window
3 Masked data refresh	Keyed-token masking of each extract; leak scan must report 0	Build engineer	No test may start on unmasked data (law)
4 Execute	Automated bill/payment regression on every build; manual exploratory for new smart-meter flows	Testers in both offices	Automation buys back cycles
5 Evidence capture	Each run stores build id, item checksums, test id, result, timestamp; record signed	Pipeline (automatic)	Audit without extra effort
6 Daily report and gate	Daily report to the Chief Engineer; gate on exit criteria	Test manager	Problems seen within a day

Table 62: Revised exit criteria and the daily reporting set

Revised exit criterion	Effect on the risk of an unnoticed critical billing defect
100 % of critical-risk conditions (billing, payment, masking) pass on the release-candidate build	No critical billing condition can be skipped or averaged away
0 open critical, ≤ 3 open major with workaround	Same as policy P1
Smart-meter load: 20,000 readings/s for 15 min with 0 lost readings	Upload losses would create wrong estimated bills
Evidence record present for 100 % of executed tests; field-level and boundary leak scans 0	Law; every result auditable
Daily reporting set	1 Open critical/major with age; 2 critical-risk conditions passed / total; 3 conditions blocked > 1 day; 4 builds tested and checksum status; 5 masking scan result

9.4 Evidence-based defence

- **Effort:** unchanged, the process needs 598 h against 360 h. The redesign needs 304 h because it reuses 180 existing designs, runs 2 cycles and automates 60 % of execution. A fits only by dropping 60 % of the conditions, including most smart-meter tests.
- **Earlier results:** the baseline checksum caught the wrong tariff file before testing (Task 3); module-aligned decisions were fastest (Task 6); structured incident reports reproduced 3 of 3 failures (Task 4). All are kept in C.
- **Risk ranking:** billing, payment and data masking stay at the top; the pass rate is replaced by risk-weighted criteria (9.2).
- **Legal:** masking is verified by three scans (0 field collisions; one coincidental substring flag reviewed and explained); every result carries a signed evidence record.

What is given up. Completeness: about 30 % of low-risk consumer-service conditions (history views, complaint status wording) are tested once, without regression. Flexibility: the central group decides scope, so module teams cannot add tests without a risk entry. **Condition under which it fails:** a late tariff regulation in week 3 would invalidate the automated billing regression just when no time remains to rebuild it. Next change: keep the tariff as data with its own table-driven tests (as in `bill_from` of Task 3), so a new tariff needs only new expected values generated from the oracle, which takes hours, not days.

Supporting evidence for Task 9

- `exp_t9.py` (masking, leak scan, scrubber, meter validator throughput) and `calc_pp.py` (effort for the strategies), Annexure F.

Threats to validity

Automation shares (50 %, 60 %) and reuse of 180 designs are estimates; if automation is late, B and C exceed 360 h. The meter throughput was measured on one core with an in-memory validator; database writes were not tested.

Marking notes and common errors

Common errors: recalculating effort without comparing it to the new capacity; masking only the database, not the reports; keeping a global pass rate as the exit criterion.

Mark Distribution

Task	Marks
Task 1 – Testing and Debugging Goals Analysis	4
Task 2 – Test Planning Analysis	7
Task 3 – Test Items and Traceability Analysis	5
Task 4 – Test Reporting Analysis	6
Task 5 – Test Policy Analysis	7
Task 6 – Test Team Organisation Analysis	4
Task 7 – Test Specialist Skills Analysis	5
Task 8 – Integrated Test Process Evaluation	4
Task 9 – System Re-evaluation and Redesign	8
TOTAL	**50**

ANNEXURE A – Catalogue of Seeded Faults

ID	Where	Fault	Found by
F1	powerpay.py	subsidy withdrawn at 500 units (>= 500 instead of > 500) - the pilot defect	TC-BILL-03b, regression test F1
F2	powerpay.py	101-300 slab measured from 101 instead of 100 (units - 101)	TC-BILL-02, regression test F2
F3	powerpay.py	bill truncated with int() instead of rounded half-up	TC-LATE-01, regression test F3
F4	powerpay.py	meter rollover ignored: abs(current - previous) used	TC-BILL-04c, regression test F4
P1	powerpay.py	retried gateway callback recorded as a second payment	TC-PAY-02a/b (INC-02)
P2	powerpay.py	receipt SMS shows the amount in paise	TC-RCP-01a/b (INC-03)
Bad fix 1	Task 1	round() (half-to-even) instead of ROUND_HALF_UP	Boundary suite at ₹710.50
Bad fix 2	Task 1	+99999 on rollover	Exact-amount check
Scan finding	Task 9	A masked 12-digit consumer number contains a real 10-digit mobile as a substring (coincidence)	Naive substring scan; cleared by field-level and boundary scans

ANNEXURE B – Suggested Mark Split per Task

Task (marks)	Split of marks
Task 1 (4)	Faults located with time/runs/tools 1; fixes and regression tests 1; testing vs debugging 0.5; bad fixes and reopen 0.5; choice 0.5; AI critique 0.5
Task 2 (7)	Three plans 1.5; effort and feasibility 1.5; checklist score 1; risks and contingencies 1; tariff change 0.75; choice with not-tested and suspension 0.75; LLM plan 0.5
Task 3 (5)	Item lists 1; attachments 1.25; wrong-tariff simulation 1; traceability coverage 0.75; choice 0.5; LLM matrix 0.5
Task 4 (6)	Log and incidents 1.5; dashboard, status and TSR 1.5; tells/hides 0.75; reproducibility 0.75; effort and context 0.5; choice 0.5; LLM summary 0.5
Task 5 (7)	Three policies 1.5; measurable statements 1.5; problem table 1; tailoring 1; cost and compliance 0.75; choice 0.75; LLM policy 0.5
Task 6 (4)	Three charts 1; situation simulation 1; analysis 0.75; schedule and overhead 0.5; choice 0.5; vendor critique 0.25
Task 7 (5)	Skills matrix 1; three plans 1; gaps and pilot defects 1; cost 0.75; choice 0.75; AI skills 0.5
Task 8 (4)	Documents and effort 1.25; flowchart with owners 1; analysis 0.5; pilot problems 0.5; choice and skip rule 0.5; AI documents 0.25
Task 9 (8)	9.1 impact, effort, masking 2; 9.2 evaluation 2; 9.3 comparison, flowchart, exit criteria 2; 9.4 defence 2

ANNEXURE C – Supplied Files and Data Notes

- `powerpay.py` (rules and faults), `exp_oracle.py`, `exp_t1.py` (Task 1), `exp_t34.py` (Tasks 3–4), `exp_t9.py` (Task 9), `calc_pp.py` (calculations for Tasks 2, 5–9), `figs4.py`, `tests/`. Run `python exp_t1.py`, `exp_t34.py`, `exp_t9.py`, then `figs4.py` (Python 3.10+, `pip install pytest matplotlib`; git and graphviz on the path).
- **Measured by running code:** failing bills per fault, minimal cases, bisect runs and culprit commits, pdb transcripts, pytest before/after, bad-fix detection, the wrong-tariff run and checksums, traceability coverage, the 18-test log, the week of 180 conditions, replay of incident reports, masking time, leak scan, meter validator throughput.

- **Estimated from stated models:** human minutes in Task 1, effort in person-hours, documentation hours, decision delays of the organisation simulation, skill gains and costs.
- **To be completed by the student:** the classmate reproducibility check (Table 32); the LLM runs of Tasks 1–5 and 8 using the checklists given.

ANNEXURE D – Glossary of Terms Used in this Answer

Term	Meaning in this answer
Baseline	A tagged, approved set of configuration items with versions and checksums.
Delta debugging (ddmin)	Automatic reduction of a failing input to a minimal failing input.
DRE	Defect removal efficiency: defects found before release ÷ all defects found before and after.
git bisect	Binary search over the commit history for the first commit where a test fails.
Item transmittal report	IEEE 829 document that hands a test item (build, files, versions) to the testers.
Keyed token (HMAC)	A deterministic replacement value computed with a secret key; cannot be reversed without the key.
Oracle	An independent way of computing the expected result (here: unit-by-unit charging).
Reopen rate	Defects reopened after being reported fixed ÷ defects fixed.
Suspension criteria	Conditions under which testing is stopped because results would be meaningless.
TMMi	Test Maturity Model integration: five levels of test process maturity.
Traceability matrix	Links from requirements to tests to the build in which they were executed.

ANNEXURE E – Index of Tables and Figures by Task

Task	Main tables	Figures
1	Table 2–Table 8	Figure 1, Figure 2
2	Table 9–Table 16	Figure 4–Figure 3
3	Table 17–Table 24	Figure 8, Figure 7
4	Table 25–Table 34	Figure 9
5	Table 35–Table 42	Figure 10
6	Table 43–Table 45	Figure 11–Figure 14
7	Table 46–Table 50	Figure 15, Figure 16
8	Table 51–Table 55	Figure 18, Figure 17
9	Table 56–Table 62	Figure 20–Figure 19

ANNEXURE F – Source Code of the Experiments

The listings below produced every measured result in this answer.

F1. powerpay.py

```
"""PowerPay prototype: bill calculation, payment and receipt rules with switchable seeded faults.

SRS rules used (Annexure D lists all 14 requirements):
  REQ-04  units = current - previous; a 5-digit meter rolls over after 99999 (add 100000)
  REQ-05  telescopic energy charge: 0-100 @ 3.00, 101-300 @ 5.00, 301-500 @ 7.00, above 500 @ 9.00
  REQ-06  if consumption is ABOVE 500 units the first 100 units are charged at 4.50 (subsidy withdrawn)
  REQ-07  fixed charge Rs 40 per bill; bill rounded to the nearest rupee (0.50 rounds up)
  REQ-08  late fee 1.5 % of the bill if paid after the due date (bill date + 15 days)
  REQ-09  a payment is recorded once per gateway transaction id (callbacks may be retried)
  REQ-11  receipt SMS states the exact amount paid in rupees
"""
from decimal import Decimal, ROUND_HALF_UP

FAULTS = {
```

```
    "F1": "subsidy withdrawn at 500 units (>= 500 instead of > 500) - the pilot defect",
    "F2": "101-300 slab measured from 101 instead of 100 (units - 101)",
    "F3": "bill truncated with int() instead of rounded half-up",
    "F4": "meter rollover ignored: abs(current - previous) used",
    "P1": "retried gateway callback recorded as a second payment",
    "P2": "receipt SMS shows the amount in paise",
}
ACTIVE = set()          # names of faults switched on
FIXED = 40



def units_consumed(previous, current):
    if "F4" in ACTIVE:
        return abs(current - previous)
    d = current - previous
    return d + 100000 if d < 0 else d



def energy_charge(units):
    if units < 0:
        raise ValueError("negative consumption")
    withdrawn = units >= 500 if "F1" in ACTIVE else units > 500
    first = Decimal("4.50") if withdrawn else Decimal("3.00")
    if units <= 100:
        return first * units
    if units <= 300:
        base = units - 101 if "F2" in ACTIVE else units - 100
        return first * 100 + 5 * base
    if units <= 500:
        return first * 100 + 5 * 200 + 7 * (units - 300)
    return first * 100 + 5 * 200 + 7 * 200 + 9 * (units - 500)



def round_rupee(x):
    x = Decimal(x)
    if "F3" in ACTIVE:
        return int(x)
    return int(x.quantize(Decimal("1"), rounding=ROUND_HALF_UP))



def bill_amount(previous, current, late=False):
    u = units_consumed(previous, current)
    amt = energy_charge(u) + FIXED
    if late:
        amt = amt * Decimal("1.015")
    return round_rupee(amt)



class PaymentLedger:
    def __init__(self):
        self.payments, self.seen, self.sms = [], set(), []

    def callback(self, txn_id, consumer, amount_paise):
        if "P1" not in ACTIVE and txn_id in self.seen:
            return "DUPLICATE_IGNORED"
        self.seen.add(txn_id)
        self.payments.append((txn_id, consumer, amount_paise))
        shown = amount_paise if "P2" in ACTIVE else amount_paise / 100
        self.sms.append(f"PowerPay: received Rs {shown:.2f} for consumer {consumer}. Txn {txn_id}")
        return "RECORDED"

    def total_for(self, consumer):
        return sum(a for _, c, a in self.payments if c == consumer)
```

F2. exp_oracle.py

```
"""Independent oracle for the PowerPay bill (unit-by-unit charging)."""
```

```
from decimal import Decimal



def oracle(prev, cur, late=False):
    u = cur - prev if cur >= prev else cur + 100000 - prev
    total = Decimal(0)
    for k in range(1, u + 1):
        if k <= 100: total += Decimal("4.50") if u > 500 else Decimal("3.00")
        elif k <= 300: total += 5
        elif k <= 500: total += 7
        else: total += 9
    total += 40
    if late: total *= Decimal("1.015")
    return int(total.quantize(Decimal("1"), rounding="ROUND_HALF_UP"))
```

F3. exp_t1.py

```
"""Task 1: locate and repair four seeded bill faults with (A) print statements, (B) the pdb debugger and
(C) systematic localisation (input minimisation + git bisect + regression test).  Writes results_t1.json."""
import json, os, random, subprocess, sys, time, shutil, re, textwrap
from decimal import Decimal
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import powerpay as pp

R = {}



from exp_oracle import oracle



def _unused_oracle(prev, cur, late=False):
    """Independent oracle: charges unit by unit (different structure from the product code)."""
    u = cur - prev if cur >= prev else cur + 100000 - prev
    total = Decimal(0)
    for k in range(1, u + 1):
        if k <= 100: total += Decimal("4.50") if u > 500 else Decimal("3.00")
        elif k <= 300: total += 5
        elif k <= 500: total += 7
        else: total += 9
    total += 40
    if late: total *= Decimal("1.015")
    return int(total.quantize(Decimal("1"), rounding="ROUND_HALF_UP"))



# ---------- 1. failure detection on a batch of 300 consumer readings ----------------------------------------
-
rng = random.Random(4)
batch = []
for i in range(300):
    prev = rng.randint(0, 99999); u = rng.choice([rng.randint(0, 800), 100, 101, 300, 301, 500, 501, 20])
    batch.append({"consumer": f"C{100000 + i}", "prev": prev, "cur": (prev + u) % 100000, "late": rng.random()
< 0.3})
batch[57] = {"consumer": "C100057", "prev": 1200, "cur": 1700, "late": False}       # the reported 500-unit
bill
for j, (pv, cu) in zip((88, 142, 233), ((99950, 50), (99800, 120), (99990, 260))):  # three meters that rolled
over 99999
    batch[j] = {"consumer": f"C{100000 + j}", "prev": pv, "cur": cu, "late": False}



def failing(rows, faults):
    pp.ACTIVE = set(faults); out = [r for r in rows if pp.bill_amount(r["prev"], r["cur"], r["late"]) !=
oracle(r["prev"], r["cur"], r["late"])]
    pp.ACTIVE = set(); return out



R["batch"] = {f: len(failing(batch, [f])) for f in ["F1", "F2", "F3", "F4"]}
```

```
R["batch_all"] = len(failing(batch, ["F1", "F2", "F3", "F4"]))


# ---------- 2. Approach A: print statements (scripted sessions; every printed line is real output) ----------
-
PRINT_SESSIONS = {
    "F1": [("bill_amount", "u, energy, fixed, total"), ("energy_charge", "units, withdrawn, first")],
    "F2": [("bill_amount", "u, energy, fixed, total"), ("energy_charge", "branch taken, base"),
("energy_charge", "units - 101 vs units - 100")],
    "F3": [("bill_amount", "u, energy, fixed, total"), ("bill_amount", "amount after late fee"),
("round_rupee", "x, int(x), quantize(x)")],
    "F4": [("bill_amount", "u, energy, fixed, total"), ("units_consumed", "previous, current, d")],
}
CASES = {"F1": (1200, 1700, False), "F2": (0, 150, False), "F3": (0, 20, True), "F4": (99950, 50, False)}



def instrumented(fault, depth):
    """Re-run the failing case with print statements at the given depth; returns the printed lines."""
    pp.ACTIVE = {fault}; prev, cur, late = CASES[fault]; lines = []
    u = pp.units_consumed(prev, cur); e = pp.energy_charge(u)
    if depth >= 1: lines.append(f"print: u={u} energy={e} fixed=40 total={pp.bill_amount(prev, cur, late)}
expected={oracle(prev, cur, late)}")
    if depth >= 2:
        if fault == "F1": lines.append(f"print: units={u} withdrawn={u >= 500} first=4.50")
        if fault == "F2": lines.append(f"print: branch=101-300 base={u - 101} (expected {u - 100})")
        if fault == "F3": lines.append(f"print: after late fee={(e + 40) * Decimal('1.015')}")
        if fault == "F4": lines.append(f"print: previous={prev} current={cur} d={cur - prev} -> abs used =
{abs(cur - prev)}")
    if depth >= 3:
        if fault == "F3": x = (e + 40) * Decimal('1.015'); lines.append(f"print: int(x)={int(x)} half-
up={int(x.quantize(Decimal('1'), rounding='ROUND_HALF_UP'))}")
        if fault == "F2": lines.append(f"print: slab start constant 101 should be 100")
    pp.ACTIVE = set(); return lines



R["A"] = {}
for f, sess in PRINT_SESSIONS.items():
    t0 = time.perf_counter(); out = [instrumented(f, d) for d in range(1, len(sess) + 1)]; dt =
time.perf_counter() - t0
    R["A"][f] = {"runs": len(sess) + 1, "edits": len(sess), "output": out[-1], "machine_s": dt,
                 "est_min": round(3 * len(sess) + 4, 1)}           # model: 3 min per edit-run cycle + 4 min
to read/fix


# ---------- 3. Approach B: pdb debugger driven through stdin (real pdb transcripts) -------------------------
--
DRIVER = os.path.join(HERE, "_drv.py")
open(DRIVER, "w").write(textwrap.dedent("""
    import sys; sys.path.insert(0, %r); import powerpay as pp
    pp.ACTIVE = {sys.argv[1]}; prev, cur, late = int(sys.argv[2]), int(sys.argv[3]), sys.argv[4] == '1'
    print('bill', pp.bill_amount(prev, cur, late))
""" % HERE))
src = open(os.path.join(HERE, "powerpay.py")).read().splitlines()
ln = lambda pat: next(i + 1 for i, l in enumerate(src) if pat in l)
PDB = {
    "F1": [f"b powerpay.py:{ln('withdrawn = units')}", "c", "p units", "n", "p withdrawn", "q"],
    "F2": [f"b powerpay.py:{ln('base = units - 101')}", "c", "p units", "n", "p base", "q"],
    "F3": [f"b powerpay.py:{ln('def round_rupee')+1}", "c", "p x", "n", "n", "p int(x)", "q"],
    "F4": [f"b powerpay.py:{ln('return abs(current')}", "c", "p previous, current", "p current - previous",
"q"],
}
R["B"] = {}
for f, cmds in PDB.items():
    prev, cur, late = CASES[f]; t0 = time.perf_counter()
    r = subprocess.run([sys.executable, "-m", "pdb", DRIVER, f, str(prev), str(cur), "1" if late else "0"],
input="\n".join(cmds) + "\n",
                       capture_output=True, text=True, cwd=HERE); dt = time.perf_counter() - t0
    tr = [l for l in r.stdout.splitlines() if l.strip()][:20]
    steps = sum(1 for c in cmds if c in ("n", "s", "c"))
    R["B"][f] = {"runs": 2, "commands": len(cmds), "steps": steps, "transcript": tr, "machine_s": dt,
                 "est_min": round(2 + 0.5 * len(cmds) + 4, 1)}     # model: 2 min set-up, 0.5 min per command,
4 min to fix
```

```
# ---------- 4. Approach C: minimise failing input, git bisect, regression test ----------------------------
--
def ddmin(rows, fails):
    n = 2
    while len(rows) >= 2:
        chunk = max(1, len(rows) // n); subsets = [rows[i:i + chunk] for i in range(0, len(rows), chunk)];
reduced = False
        for s in subsets:
            if fails(s): rows, n, reduced = s, 2, True; break
        if not reduced:
            for s in subsets:
                comp = [r for r in rows if r not in s]
                if comp and fails(comp): rows, n, reduced = comp, max(n - 1, 2), True; break
        if not reduced:
            if n >= len(rows): break
            n = min(len(rows), n * 2)
    return rows



R["C"] = {}
REPO = os.path.join(HERE, "_repo"); shutil.rmtree(REPO, ignore_errors=True); os.makedirs(REPO)
clean = open(os.path.join(HERE, "powerpay.py")).read()
# history of 16 commits; the fault for each key is introduced at the given commit
INTRO = {"F4": 4, "F2": 7, "F1": 11, "F3": 14}
MSG = ["initial slab calculator", "add fixed charge", "add late fee", "meter reading parser", "payment
ledger", "receipt sms text",
       "tariff constants moved", "code style", "logging", "due-date helper", "subsidy rule for >500 units
refactored", "docstrings",
       "receipt template", "rounding helper simplified", "typo in comments", "release 1.3 candidate"]
PAIRS = {"F1": ('withdrawn = units >= 500 if "F1" in ACTIVE else units > 500', 'withdrawn = units >= 500',
'withdrawn = units > 500'),
         "F2": ('base = units - 101 if "F2" in ACTIVE else units - 100', 'base = units - 101', 'base = units -
100'),
         "F3": ('if "F3" in ACTIVE:\n        return int(x)\n    return int(x.quantize', 'return int(x)  #
simplified\n    return int(x.quantize', 'return int(x.quantize'),
         "F4": ('if "F4" in ACTIVE:\n        return abs(current - previous)\n    d =', 'return abs(current -
previous)\n    d =', 'd =')}



def inline(on):
    s_ = clean
    for f, (pat, bad, good) in PAIRS.items():
        assert pat in s_, f
        s_ = s_.replace(pat, bad if f in on else good)
    return s_



g = lambda *a: subprocess.run(["git", *a], cwd=REPO, capture_output=True, text=True)
g("init", "-q"); g("config", "user.email", "t@pp"); g("config", "user.name", "dev")
for k, m in enumerate(MSG, 1):
    on = {f for f, c in INTRO.items() if c <= k}
    open(os.path.join(REPO, "powerpay.py"), "w").write(inline(on) + f"\n# rev {k}: {m}\n")
    g("add", "."); g("commit", "-qm", m)
for f in ["F1", "F2", "F3", "F4"]:
    t0 = time.perf_counter()
    pool = failing(batch, [f])
    fails = lambda rows: len(failing(rows, [f])) > 0
    mini = ddmin(batch, fails); t_min = time.perf_counter() - t0
    r0 = mini[0]
    test = os.path.join(REPO, "t.py")
    open(test, "w").write(textwrap.dedent(f"""
        import sys; sys.path.insert(0, {HERE!r}); sys.path.insert(0, '.'); import powerpay as pp
        from exp_oracle import oracle
        sys.exit(0 if pp.bill_amount({r0['prev']}, {r0['cur']}, {r0['late']}) == oracle({r0['prev']},
{r0['cur']}, {r0['late']}) else 1)"""))
    t1 = time.perf_counter()
    g("bisect", "reset"); g("bisect", "start"); g("bisect", "bad", "HEAD"); first = g("rev-list", "--max-
parents=0", "HEAD").stdout.strip(); g("bisect", "good", first)
    out = g("bisect", "run", sys.executable, "t.py").stdout
    steps = out.count("running")
    culprit = re.search(r"\n([0-9a-f]{40}) is the first bad commit", "\n" + out)
```

```
    msg = g("log", "-1", "--format=%s", culprit.group(1)).stdout.strip() if culprit else "?"
    g("bisect", "reset"); t_bis = time.perf_counter() - t1
    R["C"][f] = {"batch_failing": len(pool), "minimal": len(mini), "min_case": r0, "ddmin_s": t_min,
"bisect_runs": steps,
                 "culprit": msg, "culprit_rev": MSG.index(msg) + 1 if msg in MSG else None, "bisect_s": t_bis,
                 "bisect_log": [l for l in out.splitlines() if l.strip()][:30], "runs": steps + 3, "est_min":
round(2 + 1 + 3 + 4, 1)}   # model: 2 min to start ddmin, 1 min bisect, 3 min read diff, 4 min fix+test
    os.remove(test)


# ---------- 5. regression tests fail before / pass after ----------------------------------------------------
def pytest_with(faults):
    env = dict(os.environ, POWERPAY_FAULTS=",".join(faults))
    r = subprocess.run([sys.executable, "-m", "pytest", "-q", "tests/test_regression.py", "-p",
"no:cacheprovider"], cwd=HERE, env=env, capture_output=True, text=True)
    last = r.stdout.strip().splitlines()[-1]
    return last



R["reg"] = {f: {"before": pytest_with([f]), "after": pytest_with([])} for f in ["F1", "F2", "F3", "F4"]}


# ---------- 6. fixes that introduce new faults (bad fixes) --------------------------------------------------
def banker_round(x): return int(round(Decimal(x)))     # Decimal round() = ROUND_HALF_EVEN



BAD = {
    "F3-fix-round": "replace int(x) by round(x) (half-to-even rounding)",
    "F4-fix-99999": "add 99999 instead of 100000 on rollover",
    "F1-fix-special": "special-case `if units == 500: return 2700` (energy) and keep >= 500",
}



def run_bad(name, rows):
    orig_round, orig_units, orig_energy = pp.round_rupee, pp.units_consumed, pp.energy_charge
    try:
        if name == "F3-fix-round": pp.round_rupee = banker_round
        if name == "F4-fix-99999":
            pp.units_consumed = lambda p, c: (c - p) if c >= p else c - p + 99999
        if name == "F1-fix-special":
            def e(u):
                pp.ACTIVE = {"F1"}; v = orig_energy(u); pp.ACTIVE = set()
                return Decimal(2700) if u == 500 else v
            pp.energy_charge = e
        return [r for r in rows if pp.bill_amount(r["prev"], r["cur"], r["late"]) != oracle(r["prev"],
r["cur"], r["late"])]
    finally:
        pp.round_rupee, pp.units_consumed, pp.energy_charge = orig_round, orig_units, orig_energy



REPORTED_ONLY = {"F3-fix-round": [{"prev": 0, "cur": 20, "late": True}], "F4-fix-99999": [{"prev": 99950,
"cur": 50, "late": False}],
                 "F1-fix-special": [{"prev": 1200, "cur": 1700, "late": False}]}
REG_SUITE = [{"prev": 0, "cur": u, "late": l} for u in [0, 1, 20, 99, 100, 101, 150, 172, 299, 300, 301, 499,
500, 501, 502, 800] for l in (False, True)] + \
            [{"prev": 99950, "cur": 50, "late": False}, {"prev": 99999, "cur": 0, "late": False}, {"prev": 5,
"cur": 5, "late": False}]
R["bad"] = {k: {"desc": v, "reported_case_fails": len(run_bad(k, REPORTED_ONLY[k])), "batch_fails":
len(run_bad(k, batch)),
                "boundary_suite_fails": len(run_bad(k, REG_SUITE)), "suite_size": len(REG_SUITE)} for k, v in
BAD.items()}
json.dump(R, open(os.path.join(HERE, "results_t1.json"), "w"), indent=1, default=str)
print(json.dumps({k: R[k] for k in ("batch", "reg", "bad")}, indent=1)); print({f: (R["C"][f]["minimal"],
R["C"][f]["bisect_runs"], R["C"][f]["culprit"]) for f in R["C"]})
```

F4. exp_t34.py

```
"""Tasks 3 and 4: wrong-tariff-file simulation, traceability coverage, executed test log, incidents and
dashboard.
Writes results_t34.json."""
```

```
import json, os, sys, hashlib, shutil, random, datetime as dt
from decimal import Decimal
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import powerpay as pp
from exp_oracle import oracle


R = {}
# ------------------------------------------------------------------ Task 3: tariff file ----------------------
--
PROD = {"version": "TF-2026.2", "slabs": [[100, 3.00], [300, 5.00], [500, 7.00], [None, 9.00]],
"subsidy_above": 500, "first_slab_withdrawn": 4.50, "fixed": 40}
TESTCOPY = {"version": "TF-2026.2", "slabs": [[100, 3.00], [300, 5.00], [500, 7.00], [None, 8.00]],
"subsidy_above": 400, "first_slab_withdrawn": 4.50, "fixed": 40}
# the test copy kept the same version text (as in the pilot) - only its content differs



def bill_from(tariff, prev, cur, late=False):
    u = cur - prev if cur >= prev else cur + 100000 - prev
    first = Decimal(str(tariff["first_slab_withdrawn"])) if u > tariff["subsidy_above"] else
Decimal(str(tariff["slabs"][0][1]))
    e, lo = Decimal(0), 0
    for i, (hi, rate) in enumerate(tariff["slabs"]):
        r = first if i == 0 else Decimal(str(rate)); top = u if hi is None else min(u, hi)
        if top > lo: e += r * (top - lo)
        if hi is None or u <= hi: break
        lo = hi
    amt = e + tariff["fixed"]
    if late: amt *= Decimal("1.015")
    return int(amt.quantize(Decimal("1"), rounding="ROUND_HALF_UP"))



# cross-check: the table-driven calculator with the production tariff equals the product code and the oracle
chk = [(0, u, l) for u in range(0, 901) for l in (False, True)]
R["crosscheck"] = sum(1 for a in chk if bill_from(PROD, *a) == pp.bill_amount(*a) == oracle(*a)), len(chk)


ENV = os.path.join(HERE, "_env"); shutil.rmtree(ENV, ignore_errors=True); os.makedirs(f"{ENV}/baseline");
os.makedirs(f"{ENV}/test")
dump = lambda d: json.dumps(d, indent=1, sort_keys=True).encode()
open(f"{ENV}/baseline/tariff.json", "wb").write(dump(PROD)); open(f"{ENV}/test/tariff.json",
"wb").write(dump(TESTCOPY))
sha = lambda p: hashlib.sha256(open(p, "rb").read()).hexdigest()
release_note = {"baseline": "PP-REL-1.3.0", "build": "1.3.0+b214", "items": {"tariff.json": {"version": "TF-
2026.2", "sha256": sha(f"{ENV}/baseline/tariff.json")}}}
deployed_sha = sha(f"{ENV}/test/tariff.json")


# bill test cases of the test procedure TP-BILL-01 in execution order (4 TCS of Task 3, each with several data
rows)
TP = [("TC-BILL-01", "slab 1", (0, 100)), ("TC-BILL-01", "slab 1", (0, 1)), ("TC-BILL-02", "slab 2", (0,
101)), ("TC-BILL-02", "slab 2", (0, 300)),
      ("TC-BILL-03", "slab 3", (0, 301)), ("TC-BILL-03", "slab 3", (0, 450)), ("TC-BILL-03", "slab 3", (0,
500)), ("TC-BILL-04", "above 500", (0, 501)),
      ("TC-BILL-04", "above 500", (0, 800)), ("TC-BILL-04", "rollover", (99950, 50))]
MIN_PER_TEST = 6                          # assumed manual execution time per data row
tariff_in_env = TESTCOPY
results = [(tc, d, args, bill_from(tariff_in_env, *args), oracle(*args)) for tc, d, args in TP]
first_fail = next(i for i, r in enumerate(results) if r[3] != r[4]) + 1
R["t3_rows"] = [[tc, d, f"{a[0]}→{a[1]}", e, o, "PASS" if e == o else "FAIL"] for tc, d, a, e, o in results]
R["t3_detect"] = {
    "A": {"detected": "only indirectly", "check": "build number 1.3.0+b214 matches the plan", "check_result":
"PASS (tariff file not part of the build number)",
          "tests_before_detection": first_fail, "minutes": first_fail * MIN_PER_TEST, "cause_found": "no –
incident raised against billing code"},
    "B": {"detected": "yes, before testing", "check": "sha256 of deployed tariff.json against release note",
          "check_result": f"FAIL: deployed {deployed_sha[:12]}… ≠ baseline
{release_note['items']['tariff.json']['sha256'][:12]}…",
          "tests_before_detection": 0, "minutes": 1, "cause_found": "yes – wrong configuration item named"},
    "C": {"detected": "yes, by failing traced tests", "check": "REQ-05/06 test cases run and linked to build",
          "check_result": f"FAIL at test {first_fail}; trace REQ-06 → TC-BILL-03", "tests_before_detection":
first_fail, "minutes": first_fail * MIN_PER_TEST,
          "cause_found": "partly – requirement known, file version not recorded"},
}
```

```
R["t3_hash"] = {"baseline": release_note["items"]["tariff.json"]["sha256"], "deployed": deployed_sha}
R["t3_fails"] = sum(1 for r in results if r[3] != r[4])


# ---------------------------------------------------------------- traceability ------------------------------
REQS = ["REQ-01", "REQ-02", "REQ-03", "REQ-04", "REQ-05", "REQ-06", "REQ-07", "REQ-08", "REQ-09", "REQ-10",
"REQ-11", "REQ-12", "REQ-13", "REQ-14"]
TCS = {"TC-REG-01": ["REQ-01"], "TC-REG-02": ["REQ-01"], "TC-LOG-01": ["REQ-02"], "TC-LOG-02": ["REQ-02"],
"TC-MTR-01": ["REQ-03"], "TC-MTR-02": ["REQ-03", "REQ-04"],
       "TC-BILL-01": ["REQ-05", "REQ-07"], "TC-BILL-02": ["REQ-05", "REQ-07"], "TC-BILL-03": ["REQ-05", "REQ-
06", "REQ-07"], "TC-BILL-04": ["REQ-04", "REQ-05", "REQ-06", "REQ-07"],
       "TC-LATE-01": ["REQ-08"], "TC-PAY-01": ["REQ-09"], "TC-PAY-02": ["REQ-09"], "TC-PAY-03": ["REQ-10"],
"TC-RCP-01": ["REQ-11"], "TC-CMP-01": ["REQ-12"],
       "TC-HIS-01": ["REQ-13"], "TC-UI-07": []}
cov = {r: [t for t, rs in TCS.items() if r in rs] for r in REQS}
R["trace"] = {"cov": cov, "no_test": [r for r, t in cov.items() if not t], "no_req": [t for t, rs in
TCS.items() if not rs],
              "covered_pct": round(100 * sum(1 for t in cov.values() if t) / len(REQS), 1)}


# ---------------------------------------------------------------- Task 4: executed log ----------------------
-
pp.ACTIVE = {"F1", "P1", "P2"}                                # build 1.3.0+b214 as delivered to system test
log, start = [], dt.datetime(2026, 11, 9, 10, 0)
cases = [("TC-BILL-01a", "Bill 100 units", ("bill", 0, 100, False), 340), ("TC-BILL-01b", "Bill 0 units (fixed
charge only)", ("bill", 500, 500, False), 40),
         ("TC-BILL-02a", "Bill 101 units", ("bill", 0, 101, False), 345), ("TC-BILL-02b", "Bill 300 units",
("bill", 0, 300, False), 1340),
         ("TC-BILL-03a", "Bill 301 units", ("bill", 0, 301, False), 1347), ("TC-BILL-03b", "Bill 500 units",
("bill", 1200, 1700, False), 2740),
         ("TC-BILL-04a", "Bill 501 units", ("bill", 0, 501, False), 2899), ("TC-BILL-04b", "Bill 800 units",
("bill", 0, 800, False), 5590),
         ("TC-BILL-04c", "Meter rollover 99950→50", ("bill", 99950, 50, False), 340), ("TC-LATE-01", "Late fee
on ₹100 bill", ("bill", 0, 20, True), 102),
         ("TC-PAY-01a", "Pay ₹2,740 once", ("pay", ["T1"], 274000), 274000), ("TC-PAY-01b", "Pay ₹340 once",
("pay", ["T2"], 34000), 34000),
         ("TC-PAY-02a", "Gateway retries callback once", ("pay", ["T3", "T3"], 134000), 134000), ("TC-PAY-
02b", "Gateway retries callback twice", ("pay", ["T4", "T4", "T4"], 34500), 34500),
         ("TC-PAY-03", "Two different payments same consumer", ("pay", ["T5", "T6"], 4000), 8000),
         ("TC-RCP-01a", "Receipt SMS amount ₹2,740", ("sms", "T7", 274000), "Rs 2740.00"), ("TC-RCP-01b",
"Receipt SMS amount ₹40", ("sms", "T8", 4000), "Rs 40.00"),
         ("TC-RCP-01c", "Receipt SMS txn id present", ("smsid", "T9", 34000), "T9")]
t = start; inc = []
for i, (cid, title, spec, exp) in enumerate(cases, 1):
    if spec[0] == "bill": act = pp.bill_amount(*spec[1:])
    elif spec[0] == "pay":
        L = pp.PaymentLedger(); [L.callback(x, "C100057", spec[2]) for x in spec[1]]; act =
L.total_for("C100057")
    elif spec[0] == "sms":
        L = pp.PaymentLedger(); L.callback(spec[1], "C100057", spec[2]); act = L.sms[0].split("received
")[1].split(" for")[0]
    else:
        L = pp.PaymentLedger(); L.callback(spec[1], "C100057", spec[2]); act = spec[1] if spec[1] in L.sms[0]
else "missing"
    ok = act == exp; dur = 4 if spec[0] == "bill" else 7
    log.append({"no": i, "time": t.strftime("%H:%M"), "id": cid, "title": title, "expected": str(exp),
"actual": str(act), "result": "PASS" if ok else "FAIL"})
    t += dt.timedelta(minutes=dur)
pp.ACTIVE = set()
R["log"] = log; R["log_pass"] = sum(1 for l in log if l["result"] == "PASS"); R["log_n"] = len(log)


# ---------------------------------------- week of dashboard data: 180 conditions executed Mon–Fri on build
b214/b215
rng = random.Random(9)
conds = []
for k in range(120):
    u = rng.choice([rng.randint(0, 900), 100, 101, 300, 301, 500, 501]); conds.append(("bill", (0, u,
rng.random() < 0.25)))
for k in range(40): conds.append(("pay", rng.choice([1, 1, 1, 2])))
for k in range(20): conds.append(("sms", None))
rng.shuffle(conds)
plan_per_day = [30, 40, 40, 40, 30]; blocked_day = {2: 14}   # Wednesday: test server down 3 h -> 14
conditions blocked (re-run Thursday)
fixed_on = {"P2": 3}                                          # P2 fixed in build b215 delivered Thursday
morning
```

```
days, idx, carry, week_rows = [], 0, [], []
for d, cap in enumerate(plan_per_day):
    pp.ACTIVE = {"F1", "P1"} | ({"P2"} if d < fixed_on["P2"] else set())
    todo = carry + conds[idx: idx + cap - len(carry)]; idx += cap - len(carry); carry = []
    blk = blocked_day.get(d, 0); run, blocked = todo[:len(todo) - blk], todo[len(todo) - blk:]; carry =
blocked
    p_ = f_ = 0
    for kind, arg in run:
        if kind == "bill": ok = pp.bill_amount(*arg) == oracle(*arg)
        elif kind == "pay": L = pp.PaymentLedger(); [L.callback("TX", "C1", 100) for _ in range(arg)]; ok =
L.total_for("C1") == 100
        else: L = pp.PaymentLedger(); L.callback("TX", "C1", 34000); ok = "Rs 340.00" in L.sms[0]
        p_ += ok; f_ += (not ok)
        week_rows.append([["Mon", "Tue", "Wed", "Thu", "Fri"][d], kind, str(arg), "PASS" if ok else "FAIL"])
    days.append({"day": ["Mon", "Tue", "Wed", "Thu", "Fri"][d], "planned": cap, "executed": len(run),
"passed": p_, "failed": f_, "blocked": len(blocked)})
pp.ACTIVE = set()
cum = 0
for d in days: cum += d["executed"]; d["cum_exec"] = cum
R["week"] = days; R["week_rows"] = week_rows
R["week_tot"] = {k: sum(d[k] for d in days) for k in ("executed", "passed", "failed", "blocked")}
R["defects"] = {"open": [{"id": "INC-01", "sev": "Critical", "title": "500-unit bill charged with subsidy
withdrawn (+₹150)"},
                        {"id": "INC-02", "sev": "Critical", "title": "Retried gateway callback recorded as a
second payment"}],
                "closed": [{"id": "INC-03", "sev": "Major", "title": "Receipt SMS shows amount in paise (fixed
in b215)"}]}


# ------------------------------------ reproducibility self-check: replay each incident from the report
fields only
GOOD = {"INC-01": {"build": "1.3.0+b214", "steps": ("bill", 1200, 1700, False), "expected": 2740},
        "INC-02": {"build": "1.3.0+b214", "steps": ("pay", ["T3", "T3"], 134000), "expected": 134000},
        "INC-03": {"build": "1.3.0+b214", "steps": ("sms", "T7", 274000), "expected": "Rs 2740.00"}}
POOR = {"INC-01": {"text": "bill wrong for some consumers"}, "INC-02": {"text": "payment shown twice
sometimes"}, "INC-03": {"text": "SMS amount looks odd"}}



def replay(rep):
    if "steps" not in rep: return False
    pp.ACTIVE = {"F1", "P1", "P2"}; s = rep["steps"]
    if s[0] == "bill": act = pp.bill_amount(*s[1:])
    elif s[0] == "pay": L = pp.PaymentLedger(); [L.callback(x, "C", s[2]) for x in s[1]]; act =
L.total_for("C")
    else: L = pp.PaymentLedger(); L.callback(s[1], "C", s[2]); act = L.sms[0].split("received ")[1].split("
for")[0]
    pp.ACTIVE = set(); return act != rep["expected"]



R["repro"] = {"good": {k: replay(v) for k, v in GOOD.items()}, "poor": {k: replay(v) for k, v in
POOR.items()}}
json.dump(R, open(os.path.join(HERE, "results_t34.json"), "w"), indent=1, default=str)
print(R["crosscheck"], R["t3_fails"], R["t3_detect"]["A"]["tests_before_detection"], R["trace"]["no_test"],
R["trace"]["no_req"])
print(R["log_pass"], R["log_n"]); print(R["week"], R["week_tot"]); print(R["repro"])
```

F5. exp_t9.py

```
"""Task 9: data masking of test data and reports, and smart-meter upload validation throughput. Writes
results_t9.json."""
import json, os, sys, re, time, random, hashlib, hmac, csv, io
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
R = {}
rng = random.Random(12)
FIRST = ["Arun", "Meena", "Karthik", "Lakshmi", "Suresh", "Priya", "Ravi", "Divya", "Senthil", "Anitha"]
LAST = ["Kumar", "Raman", "Subramanian", "Devi", "Pillai", "Nair", "Rajan", "Murugan"]
STREETS = ["Anna Salai", "Gandhi Road", "Nehru Street", "Kamaraj Nagar", "Temple Street"]



def consumer(i):
```

```
    return {"consumer_no": f"{rng.randint(10**11, 10**12 - 1)}", "name": f"{rng.choice(FIRST)}
{rng.choice(LAST)}",
            "mobile": f"9{rng.randint(10**8, 10**9 - 1)}", "email": f"user{i}@example.in",
            "address": f"{rng.randint(1, 220)}, {rng.choice(STREETS)}, Ward {rng.randint(1, 60)}",
            "meter_no": f"SM{rng.randint(10**7, 10**8 - 1)}", "units": rng.randint(0, 900), "bill": 0,
"upi_vpa": f"{rng.choice(FIRST).lower()}{i}@okbank"}



N = 20000
data = [consumer(i) for i in range(N)]
KEY = b"test-env-only-key-rotated-per-release"      # kept outside the repository (assumption)



def token(v, prefix):                                # deterministic, keyed: same input -> same token (joins
still work)
    return prefix + hmac.new(KEY, v.encode(), hashlib.sha256).hexdigest()[:10].upper()



def mask(r):
    m = dict(r)
    m["consumer_no"] = "9" + str(int(hmac.new(KEY, r["consumer_no"].encode(), hashlib.sha256).hexdigest(),
16))[:11]   # format-preserving 12 digits
    m["name"] = token(r["name"], "NAME-"); m["mobile"] = "90000" + r["mobile"][-
5:].translate(str.maketrans("0123456789", "5820413967"))
    m["email"] = token(r["email"], "E-") + "@masked.invalid"; m["address"] = f"Ward {r['address'].split('Ward
')[1]}"   # keep ward for outage tests
    m["upi_vpa"] = token(r["upi_vpa"], "VPA-") + "@test"; m["meter_no"] = "SM" + token(r["meter_no"], "")[:8]
    return m



t0 = time.perf_counter(); masked = [mask(r) for r in data]; R["mask_s"] = time.perf_counter() - t0; R["n"] = N
# verification scan: no original PII value may appear in the masked set; formats still valid
orig_vals = {k: {r[k] for r in data} for k in ("name", "mobile", "email", "upi_vpa", "consumer_no")}
# (1) field-level scan: does any masked field equal an original value of the same kind?
R["field_collisions"] = {k: len({m[k] for m in masked} & vals) for k, vals in orig_vals.items()}
# (2) naive substring scan, as a regulator's text search would do: any original 10-digit mobile inside any
masked text
R["substring_hits"] = []
for m in masked:
    for k, x in m.items():
        if isinstance(x, str):
            for i in range(len(x) - 9):
                if x[i:i + 10] in orig_vals["mobile"]: R["substring_hits"].append([k, x, x[i:i + 10]])
# (3) boundary-aware scan (a mobile must stand alone, not inside a longer digit run)
R["boundary_hits"] = sum(1 for m in masked for x in m.values() if isinstance(x, str) for mm in
re.findall(r"(?<!\d)[6-9]\d{9}(?!\d)", x) if mm in orig_vals["mobile"])
R["format_ok"] = {"consumer_no_12_digits": all(re.fullmatch(r"\d{12}", m["consumer_no"]) for m in masked),
                  "mobile_10_digits": all(re.fullmatch(r"\d{10}", m["mobile"]) for m in masked)}
R["unique_consumer_no"] = len({m["consumer_no"] for m in masked}) == N
R["deterministic"] = mask(data[0]) == mask(data[0])
R["sample"] = [data[0], masked[0]]
# report scrubber: incident-report text with PII is scrubbed by patterns
REPORT = f"INC-01: consumer {data[0]['consumer_no']} ({data[0]['name']}, mobile {data[0]['mobile']}) at
{data[0]['address']} billed Rs 2890 for 500 units; UPI {data[0]['upi_vpa']}"
PAT = [(r"\b\d{12}\b", "<CONSUMER-NO>"), (r"\b[6-9]\d{9}\b", "<MOBILE>"), (r"[\w.]+@\w+", "<VPA/EMAIL>")]
scrub = REPORT
for p_, rep in PAT: scrub = re.sub(p_, rep, scrub)
for n in orig_vals["name"]:
    if n in scrub: scrub = scrub.replace(n, "<NAME>")
scrub = scrub.replace(data[0]["address"].split(", Ward")[0], "<ADDRESS>")
R["report"] = [REPORT, scrub]


# ------------------------------------------------ smart-meter upload validation throughput -----------------
def gen_reading(meter, slot, last):
    kwh = round(last + rng.uniform(0, 0.6), 3)
    return f"{meter},2026-11-10T{slot // 4:02d}:{(slot % 4) * 15:02d}:00,{kwh},{rng.choice(['OK', 'OK', 'OK',
'OK', 'TAMPER'])}"
```

```
rows, last = [], {}
for m in range(5000):
    meter = f"SM{10000000 + m}"; v = rng.uniform(0, 5000)
    for slot in range(24):
        v += rng.uniform(0, 0.6); rows.append(f"{meter},2026-11-10T{slot // 4:02d}:{(slot % 4) *
15:02d}:00,{v:.3f},{'OK' if rng.random() > 0.002 else 'TAMPER'}")
rows.insert(500, "SM10000020,2026-11-10T05:00:00,12.000,OK")      # a decreasing reading (rejected)
RE = re.compile(r"SM\d{8},2026-\d\d-\d\dT\d\d:(00|15|30|45):00,\d+\.\d{3},(OK|TAMPER)$")



def validate(lines):
    lastv, ok, rej = {}, 0, 0
    for ln in lines:
        if not RE.match(ln): rej += 1; continue
        m, ts, v, st = ln.split(","); v = float(v)
        if v < lastv.get(m, 0) or st == "TAMPER": rej += 1; continue
        lastv[m] = v; ok += 1
    return ok, rej



t0 = time.perf_counter(); ok, rej = validate(rows); dtv = time.perf_counter() - t0
R["meter"] = {"rows": len(rows), "ok": ok, "rejected": rej, "seconds": dtv, "rows_per_s": len(rows) / dtv}
consumers = 1200000
R["load"] = {"readings_per_day": consumers * 96, "avg_per_s": consumers * 96 / 86400, "burst_60s_per_s":
consumers / 60,
             "single_core_s_per_slot": consumers / R["meter"]["rows_per_s"]}
json.dump(R, open(os.path.join(HERE, "results_t9.json"), "w"), indent=1, default=str)
print(R["mask_s"], R["field_collisions"], R["substring_hits"], R["boundary_hits"], R["format_ok"],
R["unique_consumer_no"]); print(R["report"][1]); print(R["meter"], R["load"])
```

F6. calc_pp.py

```
"""Calculations for Tasks 2, 5, 6, 7, 8 and 9 of the PowerPay answer (effort, checklist scores, risk exposure,
situation simulation, skills scores, documentation effort). Imported by the document and the figure
scripts."""
import json, os, heapq

HERE = os.path.dirname(os.path.abspath(__file__))



# ---------------------------------------------------------------- Task 2 effort ------------------------------
--
def effort(conds, design=0.4, exe=0.25, cycles=3, retest=0.20, mgmt=0.15, retest_base="execution"):
    d = conds * design; e = conds * exe * cycles
    rt = retest * (e if retest_base == "execution" else d + e)
    sub = d + e + rt; m = mgmt * sub
    return {"design": d, "execution": e, "retest": rt, "subtotal": sub, "management": m, "total": sub + m}



CAP = 4 * 10 * 30                                  # 4 testers x 10 weeks x 30 h
E1 = effort(180); E1b = effort(180, retest_base="design+execution")
# work that is not part of the 180 functional conditions (assumed; shown in the plan)
OTHER = {"Test environment set-up and smoke tests": 60, "Performance / load tests (REQ-14)": 80, "Security and
login tests": 50,
         "Regression automation of payment and bill (policy P3)": 120, "Acceptance support with billing
section": 60,
         "Reports, reviews and meetings (outside the 15 %)": 40, "Tariff change in week 6 (re-design and re-
run)": 40}
E1_ALL = E1["total"] + sum(OTHER.values())

# ---------------------------------------------------------------- IEEE 829 checklist -----------------------
-------
IEEE = ["Test plan identifier", "Introduction / references", "Test items", "Features to be tested", "Features
not to be tested", "Approach",
        "Item pass/fail criteria", "Suspension and resumption criteria", "Test deliverables", "Testing tasks",
"Environmental needs",
```

```
        "Responsibilities", "Staffing and training needs", "Schedule", "Risks and contingencies", "Approvals"]
# 2 = present, 1 = partial, 0 = missing (scored against the three plans written in Task 2)
SCORE = {
    "A": [2, 1, 1, 2, 0, 1, 1, 0, 1, 2, 1, 1, 0, 2, 1, 1],
    "B": [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
    "C": [2, 2, 1, 2, 2, 2, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2],
}
PCT = {k: round(100 * sum(v) / (2 * len(IEEE)), 1) for k, v in SCORE.items()}
PAGES = {"A": 2 + 5 * 1, "B": 24, "C": 9}           # strategy 2 pages + five 1-page iteration plans; full
plan; risk plan
HOURS_TO_WRITE = {"A": 14, "B": 46, "C": 26}        # estimated person-hours to write and review


# ---------------------------------------------------------------- risks ------------------------------
RISKS = [  # id, risk, probability 1-5, impact 1-5, contingency
    ("R1", "Test server unavailable or shared with development", 4, 4, "Book a second VM on the State Data
Centre cloud for 6 weeks (₹18,000); run smoke tests on a laptop image; shift execution to evening slot"),
    ("R2", "Builds delivered late to test", 4, 4, "Weekly build every Monday 10:00 or the week's scope moves;
test design continues on the SRS; daily smoke build from CI"),
    ("R3", "Tariff slabs revised by the regulator during testing", 3, 5, "Tariff table kept as a data file
with its own tests; change impact analysis within 2 days; 40 h reserve in the plan"),
    ("R4", "Tester absent for a week or more", 3, 3, "Pairing and shared test cases in the tool; billing
coordinator executes bill tests from the procedure"),
    ("R5", "Payment-gateway sandbox unstable", 3, 4, "Gateway simulator with recorded callbacks (including
retries); sandbox used only for final runs"),
    ("R6", "Requirements unclear (late fee, subsidy rule)", 3, 4, "Business analyst signs the rule table in
week 1; disputed rule raised to the Chief Engineer within 48 h"),
    ("R7", "Defect fix time stays at 4 days and reopen rate at 18 %", 4, 4, "Fix-time target 2 days for
critical; regression test with every fix; root-cause review of every reopen"),
    ("R8", "Real consumer data copied into the test environment", 2, 5, "Masked extract only; access log; data
owner sign-off"),
    ("R9", "Load of 3 lakh consumers near due date not tested", 3, 5, "Load test of the bill-view and payment
path in weeks 7–8 with JMeter on the second VM"),
    ("R10", "Tool budget overspent", 2, 2, "Open-source tools first (TestLink/JIRA free tier, JMeter); licence
only for what is used"),
]
RISKS_SORTED = sorted(RISKS, key=lambda r: -r[2] * r[3])


# ---------------------------------------------------------------- Task 6 situation simulation --------------
-------------
# A decision path is a list of (role, hours waiting for that role). Waiting times are assumptions shown in
Annexure; delay = sum.
WAIT = {"tester": 0.5, "dev": 2, "team lead": 2, "module lead": 1, "test manager": 4, "analyst": 2, "billing
coord": 4, "vendor PM": 8,
        "vendor tester": 4, "contract change": 40, "chief engineer": 24, "release manager": 4}
SITUATIONS = {
    "S1 Defect dispute (tester says defect, developer says 'as designed')": {
        "A": ["tester", "team lead", "test manager", "dev", "analyst", "test manager"],
        "B": ["tester", "dev", "analyst"],
        "C": ["vendor tester", "vendor PM", "test manager", "dev", "analyst", "test manager"]},
    "S2 Urgent tariff change (regulator order, 3 days to go live)": {
        "A": ["billing coord", "analyst", "test manager", "team lead", "tester", "team lead", "test manager"],
        "B": ["billing coord", "analyst", "module lead", "tester", "dev"],
        "C": ["billing coord", "analyst", "test manager", "vendor PM", "contract change", "vendor tester",
"test manager"]},
    "S3 Tester absent for a week": {
        "A": ["team lead", "test manager", "tester"],
        "B": ["module lead", "test manager", "tester"],
        "C": ["vendor PM", "vendor tester"]},
}
SIM = {s: {k: {"path": " → ".join(v), "hops": len(v), "hours": sum(WAIT[x] for x in v)} for k, v in d.items()}
for s, d in SITUATIONS.items()}


# communication paths n(n-1)/2 inside the structure that must coordinate a release decision
def links(n): return n * (n - 1) // 2



# ---------------------------------------------------------------- Task 7 skills matrix ----------------------
---
SKILLS = ["Test design techniques", "Tool / automation", "Billing domain", "Analytical / defect analysis",
"Communication and reporting", "Planning and estimation"]
TESTERS = {  # scores 1–5 from the profiles: two fresh graduates, one from billing operations, one former
developer
```

```
    "T1 Fresh graduate (CSE)": [2, 2, 1, 2, 3, 1],
    "T2 Fresh graduate (ECE)": [2, 1, 1, 2, 2, 1],
    "T3 Ex-billing operations": [2, 1, 5, 3, 4, 2],
    "T4 Former developer": [3, 5, 2, 4, 2, 3],
}
TARGET = [4, 3, 3, 4, 3, 2]
# expected gain in 4 weeks per strategy for the two graduates (assumed from training content, checked by the
assessments)
GAIN = {"A Classroom + ISTQB": [1.5, 0.5, 0.0, 0.5, 0.5, 0.5], "B Mentoring + pairing": [1.0, 1.0, 1.5, 1.0,
1.0, 0.5],
        "C Skills matrix + rotation": [1.0, 1.0, 0.5, 1.0, 1.0, 1.0], "A+B combined": [2.0, 1.0, 1.5, 1.0,
1.0, 1.0]}
COST = {  # rupees and hours of the existing team in 4 weeks (assumptions in Task 7)
    "A Classroom + ISTQB": {"rupees": 2 * 12000 + 2 * 3500, "trainee_h": 2 * 40, "senior_h": 6},
    "B Mentoring + pairing": {"rupees": 0, "trainee_h": 2 * 40, "senior_h": 2 * 30},
    "C Skills matrix + rotation": {"rupees": 0, "trainee_h": 2 * 24, "senior_h": 24},
    "A+B combined": {"rupees": 2 * 12000 + 2 * 3500, "trainee_h": 2 * 60, "senior_h": 2 * 30},
}

# ---------------------------------------------------------------- Task 8 documentation effort --------------
------
DOCS = ["Test plan", "Test design spec", "Test case spec", "Test procedure", "Test log", "Incident reports",
"Summary report"]
# hours per document for the bill-generation feature (estimate) and the number of features of that weight (6
modules)
DOC_H = {"A": [8, 4, 6, 3, 3, 3, 4], "B": [6, 0, 2, 0, 0.5, 1, 2], "C": [8, 4, 6, 3, 3, 3, 4]}
FULL_MODULES = {"A": 6, "B": 0, "C": 2}               # modules documented in full



def doc_total(k):
    full = sum(DOC_H["A"][1:]); lean = 2 + 0 + 0 + 0.5 + 1 + 0     # per module lean set (cases, log,
incidents in tool)
    plan = 8 if k != "B" else 6; summ = 4 if k != "B" else 2
    if k == "A": return plan + 6 * (full - DOC_H["A"][-1]) + 3 * summ
    if k == "B": return plan + 6 * lean + 3 * summ
    return plan + 2 * (full - DOC_H["A"][-1]) + 4 * lean + 3 * summ



DOC_TOTAL = {k: doc_total(k) for k in "ABC"}

# ---------------------------------------------------------------- Task 9 -----------------------------
E9 = effort(400); CAP9 = 3 * 4 * 30
STRAT9 = {  # schedule feasibility by remaining effort after the strategy's levers (estimates)
    "A": {"conds": 160, "note": "cut to the 40 % highest-risk conditions to fit 360 h"},
    "B": {"conds": 400, "design": 0.3, "exe": 0.25, "cycles": 2, "auto": 0.5, "note": "re-use 180 old designs
(0.1 h), 2 cycles, 50 % of execution automated"},
    "C": {"conds": 400, "design": 0.3, "exe": 0.25, "cycles": 2, "auto": 0.6, "note": "as B + central group,
generated documents, automated evidence capture"},
}



def e9(s):
    if s == "A": return effort(160)
    d = STRAT9[s]; reuse = 180
    design = reuse * 0.1 + (d["conds"] - reuse) * 0.4
    exe = d["conds"] * d["exe"] * d["cycles"] * (1 - d["auto"]) + 40   # 40 h to build/maintain automation
scripts
    rt = 0.2 * exe; sub = design + exe + rt
    extra = 24 if s == "C" else 0                                  # set-up of evidence capture
    return {"design": design, "execution": exe, "retest": rt, "subtotal": sub, "management": 0.12 * sub,
"total": sub * 1.12 + extra}



E9S = {s: e9(s) for s in "ABC"}

if __name__ == "__main__":
    print(E1, E1b, E1_ALL, CAP); print(PCT); print([(r[0], r[2] * r[3]) for r in RISKS_SORTED])
    for s, d in SIM.items(): print(s, {k: (v["hops"], v["hours"]) for k, v in d.items()})
```

```
    print(DOC_TOTAL); print(E9, CAP9); print({s: round(v["total"]) for s, v in E9S.items()})
```

F7. tests/ – pytest regression suite

tests/conftest.py

```
import os, sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import powerpay as pp
import pytest



@pytest.fixture(autouse=True)
def faults():
    pp.ACTIVE = set(f for f in os.environ.get("POWERPAY_FAULTS", "").split(",") if f)
    yield
    pp.ACTIVE = set()
```

tests/test_regression.py

```
"""Regression tests added with each fix (Task 1, approach C). Each fails on its faulty build and passes after
the fix."""
import powerpay as pp



def test_reg_f1_exactly_500_units_keeps_subsidy():
    # REQ-06: subsidy is withdrawn only ABOVE 500 units: 300 + 1000 + 1400 + 40 = 2740
    assert pp.bill_amount(1200, 1700) == 2740
    assert pp.bill_amount(1200, 1701) == 450 + 1000 + 1400 + 9 + 40



def test_reg_f2_slab_two_starts_at_101():
    assert pp.bill_amount(0, 101) == 300 + 5 + 40
    assert pp.bill_amount(0, 150) == 300 + 250 + 40



def test_reg_f3_half_rupee_rounds_up():
    # 20 units: 60 + 40 = 100; late fee 1.5 % -> 101.50 -> 102
    assert pp.bill_amount(0, 20, late=True) == 102
    assert pp.bill_amount(0, 172, late=True) == 711     # 700 * 1.015 = 710.50 -> 711 (even rupee part)



def test_reg_f4_meter_rollover():
    # previous 99950, current 50 -> 100 units -> 300 + 40
    assert pp.bill_amount(99950, 50) == 340
```

ANNEXURE G – Viva Questions and Model Answers

Five likely viva questions per task, answered from the evidence of this assignment.

Task 1 – Testing and debugging

#	Question	Model answer
1	What is the difference between testing and debugging?	Testing reveals failures by comparing actual and expected results; debugging locates and removes the fault that causes a failure. Testing ends with an incident report; debugging ends with a fix and a regression test.
2	Why did F1 change the bill only at exactly 500 units?	REQ-06 withdraws the subsidy only above 500. The seeded `>= 500` also withdrew it at 500 (₹150 more); at 501 and above both conditions agree.
3	How many bisect runs are needed for 16 commits?	$\lceil \log_2 16 \rceil = 4$; the measured runs were 4 for F1–F3 and 3 for F4, which was introduced early.
4	Why is round() a bad fix for F3?	Python's Decimal round() uses half-to-even, so ₹710.50 becomes ₹710; REQ-07 requires half-up (₹711).
5	What explains an 18 % reopen rate?	Fixes checked only on the reported case, no regression tests, symptom patches (special cases) and pressure from a 4-day fix time.

Task 2 – Test planning

#	Question	Model answer
1	List the IEEE 829 test plan components.	Identifier, introduction, test items, features to be tested, features not to be tested, approach, item pass/fail criteria, suspension and resumption criteria, deliverables, testing tasks, environment, responsibilities, staffing and training, schedule, risks and contingencies, approvals.
2	Calculate the effort for 180 conditions.	Design 72 h + execution 135 h + retest 27 h = 234 h; + 15 % management = 269.1 h.
3	Is the plan feasible?	Yes: with 450 h of other planned work the total is about 719 h of 1,200 h (60 %); the rest is a reserve for the top risks.
4	What are your suspension criteria?	Smoke test fails, server down > 4 h, > 30 % of a day's tests blocked, or a critical defect blocks a module; resume after a smoke pass and checksum audit.
5	Why the risk-based plan?	It covers the pilot problems by risk on 9 pages; with a 2-page IEEE appendix it scores 100 % on the checklist.

Task 3 – Test items

#	Question	Model answer
1	Why did the build number not detect the wrong tariff file?	The tariff file is a separate configuration item; the build number of the code did not change.
2	What does an item transmittal report contain?	The items handed over (build, files and versions), their location, status (smoke result) and approval by the build engineer.
3	Which requirement had no test?	REQ-14 (load); TC-UI-07 had no requirement.
4	How did the checksum detect the wrong file even with the same version text?	SHA-256 of the content differed from the value in the release note.
5	Which attachments are compulsory?	TDS and TCS for billing and payment, a procedure per module and an item transmittal report for every build.

Task 4 – Reporting

#	Question	Model answer
1	What was the pass rate of your 18 tests?	13 of 18 = 72 %; 5 failures in 3 incidents (INC-01, INC-02, INC-03).
2	Why can a 95 % pass rate be dangerous?	The 500-unit fault fails only the 500-unit rows; the module can look healthy while every 500-unit consumer is over-charged ₹150.

#	Question	Model answer
3	What makes an incident report reproducible?	Build, inputs, steps, expected and actual results, and evidence; the structured reports replayed 3 of 3.
4	Which report answers ‘is it ready?’	The test summary report, through its evaluation section.
5	Which reports did you choose for whom?	Incidents for developers; dashboard and weekly status for the TM and Chief Engineer; summary reports for the Chief Engineer and auditors.

Task 5 – Test policy

#	Question	Model answer
1	What is the difference between a policy and a strategy?	The policy says why the organisation tests and what rules apply (release rule, targets); the strategy says how (levels, techniques, tools).
2	Give two measurable policy statements.	No release with an open critical defect (tracker query = 0); DRE ≥ 90 % (found before ÷ all found in 90 days).
3	What was the pilot DRE?	52 ÷ 73 = 71.2 %.
4	How is the policy tailored for a feedback form?	A tailoring sheet drops P2, P4, P7; P1 and P3 can never be dropped.
5	What are TMMi levels 2 and 3?	Level 2 managed: policy and strategy, planning, monitoring and control, design and execution, environment. Level 3 defined: organisation, training, life-cycle integration, non-functional testing, peer reviews.

Task 6 – Team organisation

#	Question	Model answer
1	Which structure decided fastest?	Module-aligned (B): 4.5 h, 9.5 h and 5.5 h for the three situations.
2	Why was the vendor structure slow for the tariff change?	The change needed a contract change request (40 h) and the vendor PM (8 h): 66 h in total.
3	How do you keep independence in B?	The test manager owns the summary report, the release recommendation and a cross-module regression run.
4	What is the single point of failure in B?	The build engineer serving three teams; T4 is trained as backup.
5	Should the vendor approve release?	No – release is an acceptance decision for the Corporation; the vendor supplies evidence only.

Task 7 – Skills

#	Question	Model answer
1	Name the six skill areas of your matrix.	Test design techniques, tools/automation, billing domain, analytical/defect analysis, communication and reporting, planning and estimation.
2	Which strategy builds domain knowledge?	Mentoring and pair testing with T3 (ex-billing).
3	How much does certification cost?	About ₹31,000 for two graduates (assumed fees), 21 % of the tool budget.
4	What did you recommend?	B for 4 weeks, then C; ISTQB after the project from the training budget.
5	Which skills remain essential with AI tools?	Domain knowledge, oracles and expected results, critical review of generated output, judgement for go/no-go.

Task 8 – Documentation sets

#	Question	Model answer
1	How much effort does each set need?	Full IEEE about 134 h, lean about 33 h, hybrid about 72 h for the whole process (estimate).
2	Which pilot problem does the lean set miss?	Tested ≠ released build – the tool’s build field alone does not identify configuration files.

#	Question	Model answer
3	What is your skip rule?	Skip only if no critical requirement, the information is exportable from the tool with its build id, and the skip is recorded in the tailoring sheet.
4	Who owns the summary report?	The test manager; the Chief Engineer signs the go/no-go.
5	Can AI-generated documents be audit evidence?	Only if every number comes from execution records, a named tester reviews them, and the build id and checksum are attached.

Task 9 – Redesign

#	Question	Model answer
1	Compute the effort for 400 conditions.	160 + 300 + 60 = 520 h; + 15 % = 598 h; capacity 3 × 4 × 30 = 360 h; shortfall 238 h.
2	What is the smart-meter peak load?	12 lakh meters in 60 s per slot = 20,000 readings/s; average 1,333/s.
3	What did the masking scan find?	0 field-level collisions; a naive substring scan flagged one masked consumer number containing a real mobile's 10 digits by coincidence; the boundary-aware scan found 0.
4	Why not retain the process?	It fits 360 h only by dropping 60 % of conditions, including most smart-meter tests, and has no automated audit evidence.
5	When would the redesign fail?	A late tariff regulation in week 3; mitigated by table-driven tariff tests whose expected values come from the oracle.

ANNEXURE H – Approved Test Plan PP-TP-01 v1.0 (risk-based plan with IEEE appendix)

1 Test plan identifier

PP-TP-01 v1.0, dated 02-11-2026. Owner: Test Manager, PowerPay project. Status: approved by the Chief Engineer (IT) on 04-11-2026. Changes are recorded in the revision table and re-approved when a product risk with exposure ≥ 15 changes.

2 Introduction

PowerPay is the Corporation's consumer portal for 3 lakh consumers. This plan covers system and acceptance testing and the developers' unit testing obligations. References: SRS v1.2 (REQ-01–14), pilot-year report, test policy TP-2026, IEEE 829-2008 templates, this answer's Tasks 1–8.

3 Test items

Build 1.3.x from the main branch; baseline tag PP-REL-1.3.x; configuration items tariff.json (TF-2026.2), database schema v7, SMS templates v3, deployment scripts, test scripts and masked data extracts. Each delivery is accompanied by an item transmittal report with SHA-256 checksums.

4 Features to be tested

All 14 requirements. Priority by risk: billing (REQ-04–08) and payment (REQ-09–11) critical; login (REQ-01–02) and load (REQ-14) high; meter upload (REQ-03) high; complaints and history (REQ-12–13) medium.

5 Features not to be tested

Payment gateway internals (bank-certified); SMS operator delivery after hand-off; legacy counter billing (unchanged); Internet Explorer 11 (0.4 % of visits, assumption). Each exclusion is accepted by the Chief Engineer in the approval meeting.

6 Approach

Risk-based. Billing: boundary value analysis on every slab edge, decision table for subsidy and late fee, unit-by-unit oracle. Payment: state-based tests of the callback life cycle including retries, timeouts and duplicates. Regression: automated pytest suite for billing and payment run on every build. Load: JMeter, 5,000 virtual users, 90th percentile ≤ 3 s. Debugging rule: every defect is localised systematically (minimal case, bisect) and fixed with a regression test (Task 1).

7 Item pass/fail criteria

A test case passes if every expected value is met exactly (bills to the rupee). An item passes when all its critical and major test cases pass and no open critical defect exists against it.

8 Suspension and resumption

Suspend on: smoke test failure; test server unavailable for more than 4 hours; more than 30 % of a day's tests blocked; a critical defect blocking a whole module; a checksum mismatch. Resume when a new build passes the 20-case smoke suite and the baseline audit.

9 Test deliverables

This plan; traceability matrix; TDS and TCS for billing and payment; test procedures per module; test logs and incident reports in the tool; daily dashboard; weekly status; summary report per level; automated regression suite; load-test scripts and results.

10 Testing tasks

Plan and risk analysis (week 1); environment and tools (weeks 1–2); test design (weeks 2–3); cycle 1 (weeks 4–5); tariff change re-test (week 6); cycle 2 and automation (weeks 6–7); load and security (weeks 7–8); cycle 3 regression and acceptance (weeks 9–10); summary report and go/no-go (week 10).

11 Environmental needs

Test server (8 vCPU, 16 GB RAM) reserved for testing; second VM for load (6 weeks); masked data extract refreshed weekly; payment gateway sandbox with configurable retries; SMS stub that records messages; TestLink and JIRA; JMeter 5.6.

12 Responsibilities

Test manager: plan, reports, release recommendation. Testers: design, execution, incidents, automation. Developers: unit tests, fixes with regression tests. Build engineer: builds, baselines, transmittal reports. Business analyst: rule clarification within 48 h. Billing coordinator: acceptance tests and sign-off.

13 Staffing and training

Four testers (two graduates mentored for 4 weeks, Task 7); JMeter training 1 day for T4; all testers trained on the masking procedure.

14 Schedule

As in the Gantt chart of Task 2 with milestones M1 design complete (end of week 3), M2 cycle 1 complete (week 5), M3 cycle 2 complete (week 8), M4 go/no-go (week 10).

15 Risks and contingencies

Risk register R1–R10 (Task 2) with owners; reviewed every Monday; any risk reaching exposure 16 is escalated to the Chief Engineer.

16 Approvals

Chief Engineer (IT) – approves plan and release; Head of Billing – approves billing acceptance criteria; Audit officer – reviews deliverables list and evidence rules.

Revision	Date	Change	Approved by
1.0	04-11-2026	First approved version	Chief Engineer (IT)
1.1	(week 6)	Tariff revision: REQ-05 rates; risk R3 re-rated; 40 h reserve used	Chief Engineer (IT)

ANNEXURE I – Catalogue of the 180 Functional Test Conditions

Generated from the test design (module, condition, technique, requirement, expected result). Bill expected values come from the oracle; the catalogue is regenerated when the tariff changes.

ID	Module	Condition	Technique	Requirement	Expected
C001	Billing	0 units	BVA	REQ-05/06/07	₹40
C002	Billing	0 units , late	BVA	REQ-05/06/07/08	₹41
C003	Billing	1 units	BVA	REQ-05/06/07	₹43
C004	Billing	1 units , late	BVA	REQ-05/06/07/08	₹44
C005	Billing	50 units	EP	REQ-05/06/07	₹190
C006	Billing	50 units , late	EP	REQ-05/06/07/08	₹193
C007	Billing	99 units	BVA	REQ-05/06/07	₹337
C008	Billing	99 units , late	BVA	REQ-05/06/07/08	₹342
C009	Billing	100 units	BVA	REQ-05/06/07	₹340
C010	Billing	100 units , late	BVA	REQ-05/06/07/08	₹345
C011	Billing	101 units	BVA	REQ-05/06/07	₹345
C012	Billing	101 units , late	BVA	REQ-05/06/07/08	₹350
C013	Billing	102 units	EP	REQ-05/06/07	₹350
C014	Billing	102 units , late	EP	REQ-05/06/07/08	₹355
C015	Billing	150 units	EP	REQ-05/06/07	₹590
C016	Billing	150 units , late	EP	REQ-05/06/07/08	₹599
C017	Billing	200 units	EP	REQ-05/06/07	₹840
C018	Billing	200 units , late	EP	REQ-05/06/07/08	₹853
C019	Billing	299 units	BVA	REQ-05/06/07	₹1,335
C020	Billing	299 units , late	BVA	REQ-05/06/07/08	₹1,355
C021	Billing	300 units	BVA	REQ-05/06/07	₹1,340
C022	Billing	300 units , late	BVA	REQ-05/06/07/08	₹1,360
C023	Billing	301 units	BVA	REQ-05/06/07	₹1,347
C024	Billing	301 units , late	BVA	REQ-05/06/07/08	₹1,367
C025	Billing	302 units	EP	REQ-05/06/07	₹1,354
C026	Billing	302 units , late	EP	REQ-05/06/07/08	₹1,374
C027	Billing	350 units	EP	REQ-05/06/07	₹1,690
C028	Billing	350 units , late	EP	REQ-05/06/07/08	₹1,715
C029	Billing	450 units	EP	REQ-05/06/07	₹2,390
C030	Billing	450 units , late	EP	REQ-05/06/07/08	₹2,426
C031	Billing	499 units	BVA	REQ-05/06/07	₹2,733
C032	Billing	499 units , late	BVA	REQ-05/06/07/08	₹2,774
C033	Billing	500 units	BVA	REQ-05/06/07	₹2,740
C034	Billing	500 units , late	BVA	REQ-05/06/07/08	₹2,781
C035	Billing	501 units	BVA	REQ-05/06/07	₹2,899
C036	Billing	501 units , late	BVA	REQ-05/06/07/08	₹2,942
C037	Billing	502 units	EP	REQ-05/06/07	₹2,908
C038	Billing	502 units , late	EP	REQ-05/06/07/08	₹2,952
C039	Billing	600 units	EP	REQ-05/06/07	₹3,790
C040	Billing	600 units , late	EP	REQ-05/06/07/08	₹3,847
C041	Billing	750 units	EP	REQ-05/06/07	₹5,140
C042	Billing	750 units , late	EP	REQ-05/06/07/08	₹5,217
C043	Billing	800 units	EP	REQ-05/06/07	₹5,590
C044	Billing	800 units , late	EP	REQ-05/06/07/08	₹5,674
C045	Billing	900 units	EP	REQ-05/06/07	₹6,490
C046	Billing	900 units , late	EP	REQ-05/06/07/08	₹6,587
C047	Meter	99950→50	BVA	REQ-03/04	₹340
C048	Meter	99999→0	BVA	REQ-03/04	₹43
C049	Meter	99800→120	BVA	REQ-03/04	₹1,480
C050	Meter	99990→260	BVA	REQ-03/04	₹1,190
C051	Meter	5→5	BVA	REQ-03/04	₹40
C052	Meter	100→99	BVA	REQ-03/04	reject (lower reading)

ID	Module	Condition	Technique	Requirement	Expected
C053	Meter	Reading with letters	EP invalid	REQ-03	reject
C054	Meter	Reading 6 digits	EP invalid	REQ-03	reject
C055	Meter	Upload twice same month	state	REQ-03	second rejected
C056	Billing	Due date = bill date + 15	BVA	REQ-08	late fee from day 16
C057	Billing	Paid on day 15	BVA	REQ-08	no late fee
C058	Billing	Paid on day 16	BVA	REQ-08	late fee 1.5 %
C059	Payment	Pay exact amount	EP	REQ-10	recorded once
C060	Payment	Pay less than bill	EP	REQ-10	refused
C061	Payment	Pay more than bill	EP	REQ-10	refused
C062	Payment	Callback retried once	state	REQ-09	recorded once
C063	Payment	Callback retried 3 times	state	REQ-09	recorded once
C064	Payment	Two transactions same consumer	EP	REQ-09	both recorded
C065	Payment	Callback after session expiry	EP	REQ-09	recorded once
C066	Payment	Gateway timeout then success	state	REQ-09	recorded once
C067	Payment	Gateway failure	EP	REQ-09	not recorded; bill outstanding
C068	Payment	Amount in paise to gateway	EP	REQ-09	bill × 100
C069	Payment	UPI payment	EP	REQ-09	recorded once
C070	Payment	Card payment	EP	REQ-09	recorded once
C071	Payment	Net banking	EP	REQ-09	recorded once
C072	Payment	Double click on Pay	EP	REQ-09	one transaction
C073	Payment	Refresh after payment	EP	REQ-09	no second charge
C074	Payment	Pay old bill after new bill	EP	REQ-10	oldest outstanding first
C075	Payment	Pay exact amount (data set 2)	EP	REQ-10	recorded once
C076	Payment	Pay less than bill (data set 2)	EP	REQ-10	refused
C077	Payment	Pay more than bill (data set 2)	EP	REQ-10	refused
C078	Payment	Callback retried once (data set 2)	state	REQ-09	recorded once
C079	Payment	Callback retried 3 times (data set 2)	state	REQ-09	recorded once
C080	Payment	Two transactions same consumer (data set 2)	EP	REQ-09	both recorded
C081	Payment	Callback after session expiry (data set 2)	EP	REQ-09	recorded once
C082	Payment	Gateway timeout then success (data set 2)	state	REQ-09	recorded once
C083	Payment	Gateway failure (data set 2)	EP	REQ-09	not recorded; bill outstanding
C084	Payment	Amount in paise to gateway (data set 2)	EP	REQ-09	bill × 100
C085	Payment	UPI payment (data set 2)	EP	REQ-09	recorded once
C086	Payment	Card payment (data set 2)	EP	REQ-09	recorded once
C087	Payment	Net banking (data set 2)	EP	REQ-09	recorded once
C088	Payment	Double click on Pay (data set 2)	EP	REQ-09	one transaction
C089	Payment	Refresh after payment (data set 2)	EP	REQ-09	no second charge
C090	Payment	Pay old bill after new bill (data set 2)	EP	REQ-10	oldest outstanding first
C091	Payment	Pay exact amount (data set 3)	EP	REQ-10	recorded once
C092	Payment	Pay less than bill (data set 3)	EP	REQ-10	refused
C093	Payment	Pay more than bill (data set 3)	EP	REQ-10	refused
C094	Payment	Callback retried once (data set 3)	state	REQ-09	recorded once
C095	Payment	Callback retried 3 times (data set 3)	state	REQ-09	recorded once
C096	Payment	Two transactions same consumer (data set 3)	EP	REQ-09	both recorded
C097	Payment	Callback after session expiry (data set 3)	EP	REQ-09	recorded once
C098	Payment	Gateway timeout then success (data set 3)	state	REQ-09	recorded once
C099	Receipt	SMS amount ₹2,740	EP	REQ-11	Rs 2740.00
C100	Receipt	SMS amount ₹40	EP	REQ-11	Rs 40.00
C101	Receipt	SMS txn id	EP	REQ-11	id present
C102	Receipt	SMS within 2 min	EP	REQ-11	≤ 120 s
C103	Receipt	SMS on retried callback	EP	REQ-11	one SMS
C104	Receipt	SMS Tamil template	EP	REQ-11	Tamil text
C105	Receipt	SMS consumer number masked	EP	REQ-11	last 4 digits
C106	Receipt	SMS amount ₹2,740	EP	REQ-11	Rs 2740.00

ID	Module	Condition	Technique	Requirement	Expected
C107	Receipt	SMS amount ₹40	EP	REQ-11	Rs 40.00
C108	Receipt	SMS txn id	EP	REQ-11	id present
C109	Receipt	SMS within 2 min	EP	REQ-11	≤ 120 s
C110	Receipt	SMS on retried callback	EP	REQ-11	one SMS
C111	Receipt	SMS Tamil template	EP	REQ-11	Tamil text
C112	Receipt	SMS consumer number masked	EP	REQ-11	last 4 digits
C113	Receipt	SMS amount ₹2,740	EP	REQ-11	Rs 2740.00
C114	Receipt	SMS amount ₹40	EP	REQ-11	Rs 40.00
C115	Receipt	SMS txn id	EP	REQ-11	id present
C116	Receipt	SMS within 2 min	EP	REQ-11	≤ 120 s
C117	Receipt	SMS on retried callback	EP	REQ-11	one SMS
C118	Receipt	SMS Tamil template	EP	REQ-11	Tamil text
C119	Consumer	Register valid 12-digit number	EP/BVA	REQ-01	OTP sent
C120	Consumer	Register 11 digits	EP/BVA	REQ-01	rejected
C121	Consumer	Wrong OTP 3 times	EP/BVA	REQ-01	blocked 10 min
C122	Consumer	Login 3 wrong passwords	EP/BVA	REQ-02	locked 30 min
C123	Consumer	Login after 30 min	EP/BVA	REQ-02	allowed
C124	Consumer	View 12 bills	EP/BVA	REQ-13	12 listed
C125	Consumer	View with 3 bills only	EP/BVA	REQ-13	3 listed
C126	Complaints	Report outage	EP/BVA	REQ-12	ticket number
C127	Complaints	Track ticket	EP/BVA	REQ-12	status shown
C128	Complaints	Duplicate outage same area	EP/BVA	REQ-12	linked to first ticket
C129	Consumer	Register valid 12-digit number (set 2)	EP/BVA	REQ-01	OTP sent
C130	Consumer	Register 11 digits (set 2)	EP/BVA	REQ-01	rejected
C131	Consumer	Wrong OTP 3 times (set 2)	EP/BVA	REQ-01	blocked 10 min
C132	Consumer	Login 3 wrong passwords (set 2)	EP/BVA	REQ-02	locked 30 min
C133	Consumer	Login after 30 min (set 2)	EP/BVA	REQ-02	allowed
C134	Consumer	View 12 bills (set 2)	EP/BVA	REQ-13	12 listed
C135	Consumer	View with 3 bills only (set 2)	EP/BVA	REQ-13	3 listed
C136	Complaints	Report outage (set 2)	EP/BVA	REQ-12	ticket number
C137	Complaints	Track ticket (set 2)	EP/BVA	REQ-12	status shown
C138	Complaints	Duplicate outage same area (set 2)	EP/BVA	REQ-12	linked to first ticket
C139	Consumer	Register valid 12-digit number (set 3)	EP/BVA	REQ-01	OTP sent
C140	Consumer	Register 11 digits (set 3)	EP/BVA	REQ-01	rejected
C141	Consumer	Wrong OTP 3 times (set 3)	EP/BVA	REQ-01	blocked 10 min
C142	Consumer	Login 3 wrong passwords (set 3)	EP/BVA	REQ-02	locked 30 min
C143	Consumer	Login after 30 min (set 3)	EP/BVA	REQ-02	allowed
C144	Consumer	View 12 bills (set 3)	EP/BVA	REQ-13	12 listed
C145	Consumer	View with 3 bills only (set 3)	EP/BVA	REQ-13	3 listed
C146	Complaints	Report outage (set 3)	EP/BVA	REQ-12	ticket number
C147	Complaints	Track ticket (set 3)	EP/BVA	REQ-12	status shown
C148	Complaints	Duplicate outage same area (set 3)	EP/BVA	REQ-12	linked to first ticket
C149	Consumer	Register valid 12-digit number (set 4)	EP/BVA	REQ-01	OTP sent
C150	Consumer	Register 11 digits (set 4)	EP/BVA	REQ-01	rejected
C151	Consumer	Wrong OTP 3 times (set 4)	EP/BVA	REQ-01	blocked 10 min
C152	Consumer	Login 3 wrong passwords (set 4)	EP/BVA	REQ-02	locked 30 min
C153	Consumer	Login after 30 min (set 4)	EP/BVA	REQ-02	allowed
C154	Consumer	View 12 bills (set 4)	EP/BVA	REQ-13	12 listed
C155	Consumer	View with 3 bills only (set 4)	EP/BVA	REQ-13	3 listed
C156	Complaints	Report outage (set 4)	EP/BVA	REQ-12	ticket number
C157	Complaints	Track ticket (set 4)	EP/BVA	REQ-12	status shown
C158	Complaints	Duplicate outage same area (set 4)	EP/BVA	REQ-12	linked to first ticket
C159	Consumer	Register valid 12-digit number (set 5)	EP/BVA	REQ-01	OTP sent
C160	Consumer	Register 11 digits (set 5)	EP/BVA	REQ-01	rejected
C161	Consumer	Wrong OTP 3 times (set 5)	EP/BVA	REQ-01	blocked 10 min
C162	Consumer	Login 3 wrong passwords (set 5)	EP/BVA	REQ-02	locked 30 min
C163	Consumer	Login after 30 min (set 5)	EP/BVA	REQ-02	allowed
C164	Consumer	View 12 bills (set 5)	EP/BVA	REQ-13	12 listed

ID	Module	Condition	Technique	Requirement	Expected
C165	Consumer	View with 3 bills only (set 5)	EP/BVA	REQ-13	3 listed
C166	Complaints	Report outage (set 5)	EP/BVA	REQ-12	ticket number
C167	Complaints	Track ticket (set 5)	EP/BVA	REQ-12	status shown
C168	Complaints	Duplicate outage same area (set 5)	EP/BVA	REQ-12	linked to first ticket
C169	Consumer	Register valid 12-digit number (set 6)	EP/BVA	REQ-01	OTP sent
C170	Consumer	Register 11 digits (set 6)	EP/BVA	REQ-01	rejected
C171	Consumer	Wrong OTP 3 times (set 6)	EP/BVA	REQ-01	blocked 10 min
C172	Consumer	Login 3 wrong passwords (set 6)	EP/BVA	REQ-02	locked 30 min
C173	Consumer	Login after 30 min (set 6)	EP/BVA	REQ-02	allowed
C174	Consumer	View 12 bills (set 6)	EP/BVA	REQ-13	12 listed
C175	Consumer	View with 3 bills only (set 6)	EP/BVA	REQ-13	3 listed
C176	Complaints	Report outage (set 6)	EP/BVA	REQ-12	ticket number
C177	Complaints	Track ticket (set 6)	EP/BVA	REQ-12	status shown
C178	Complaints	Duplicate outage same area (set 6)	EP/BVA	REQ-12	linked to first ticket
C179	Consumer	Register valid 12-digit number (set 7)	EP/BVA	REQ-01	OTP sent
C180	Consumer	Register 11 digits (set 7)	EP/BVA	REQ-01	rejected

ANNEXURE J – Execution Record of the Week (166 executed conditions)

Every condition executed in the simulated week of Task 4, in order, with its result on the build of that day (b214 Monday–Wednesday, b215 Thursday–Friday).

#	Day	Kind	Input	Result	#	Day	Kind	Input	Result
1	Mon	bill	(0, 501, False)	PASS	84	Wed	bill	(0, 100, False)	PASS
2	Mon	pay	1	PASS	85	Wed	sms	None	FAIL
3	Mon	bill	(0, 100, True)	PASS	86	Wed	pay	1	PASS
4	Mon	bill	(0, 300, False)	PASS	87	Wed	bill	(0, 500, False)	FAIL
5	Mon	sms	None	FAIL	88	Wed	bill	(0, 100, True)	PASS
6	Mon	bill	(0, 300, False)	PASS	89	Wed	bill	(0, 300, False)	PASS
7	Mon	bill	(0, 500, True)	FAIL	90	Wed	bill	(0, 712, True)	PASS
8	Mon	bill	(0, 101, True)	PASS	91	Wed	pay	2	FAIL
9	Mon	pay	1	PASS	92	Wed	bill	(0, 501, True)	PASS
10	Mon	sms	None	FAIL	93	Wed	pay	1	PASS
11	Mon	pay	1	PASS	94	Wed	bill	(0, 100, False)	PASS
12	Mon	bill	(0, 122, False)	PASS	95	Wed	bill	(0, 501, True)	PASS
13	Mon	bill	(0, 625, False)	PASS	96	Wed	pay	1	PASS
14	Mon	pay	1	PASS	97	Thu	bill	(0, 300, True)	PASS
15	Mon	bill	(0, 100, False)	PASS	98	Thu	sms	None	PASS
16	Mon	sms	None	FAIL	99	Thu	bill	(0, 501, False)	PASS
17	Mon	bill	(0, 101, True)	PASS	100	Thu	bill	(0, 100, True)	PASS
18	Mon	pay	1	PASS	101	Thu	bill	(0, 101, True)	PASS
19	Mon	bill	(0, 300, True)	PASS	102	Thu	bill	(0, 300, False)	PASS
20	Mon	bill	(0, 100, False)	PASS	103	Thu	sms	None	PASS
21	Mon	bill	(0, 236, False)	PASS	104	Thu	pay	2	FAIL
22	Mon	sms	None	FAIL	105	Thu	bill	(0, 501, True)	PASS
23	Mon	bill	(0, 300, False)	PASS	106	Thu	pay	1	PASS
24	Mon	pay	1	PASS	107	Thu	bill	(0, 501, False)	PASS
25	Mon	bill	(0, 501, False)	PASS	108	Thu	sms	None	PASS
26	Mon	bill	(0, 168, False)	PASS	109	Thu	bill	(0, 500, False)	FAIL
27	Mon	bill	(0, 501, False)	PASS	110	Thu	bill	(0, 341, False)	PASS
28	Mon	sms	None	FAIL	111	Thu	bill	(0, 500, True)	FAIL
29	Mon	pay	2	FAIL	112	Thu	pay	1	PASS
30	Mon	sms	None	FAIL	113	Thu	bill	(0, 300, False)	PASS
31	Tue	bill	(0, 501, True)	PASS	114	Thu	pay	2	FAIL
32	Tue	sms	None	FAIL	115	Thu	bill	(0, 301, False)	PASS
33	Tue	bill	(0, 501, False)	PASS	116	Thu	bill	(0, 619, False)	PASS
34	Tue	bill	(0, 100, False)	PASS	117	Thu	pay	2	FAIL
35	Tue	bill	(0, 35, False)	PASS	118	Thu	bill	(0, 500, False)	FAIL
36	Tue	bill	(0, 301, False)	PASS	119	Thu	sms	None	PASS
37	Tue	bill	(0, 501, True)	PASS	120	Thu	bill	(0, 500, False)	FAIL
38	Tue	bill	(0, 101, False)	PASS	121	Thu	bill	(0, 301, True)	PASS
39	Tue	sms	None	FAIL	122	Thu	pay	1	PASS
40	Tue	bill	(0, 604, False)	PASS	123	Thu	bill	(0, 101, False)	PASS
41	Tue	bill	(0, 101, True)	PASS	124	Thu	bill	(0, 617, False)	PASS
42	Tue	bill	(0, 3, False)	PASS	125	Thu	bill	(0, 301, False)	PASS
43	Tue	bill	(0, 501, True)	PASS	126	Thu	sms	None	PASS
44	Tue	bill	(0, 307, False)	PASS	127	Thu	bill	(0, 443, True)	PASS
45	Tue	bill	(0, 101, False)	PASS	128	Thu	pay	1	PASS
46	Tue	bill	(0, 828, True)	PASS	129	Thu	bill	(0, 626, False)	PASS
47	Tue	bill	(0, 755, False)	PASS	130	Thu	pay	2	FAIL
48	Tue	bill	(0, 500, True)	FAIL	131	Thu	bill	(0, 500, False)	FAIL
49	Tue	pay	1	PASS	132	Thu	bill	(0, 501, False)	PASS
50	Tue	pay	2	FAIL	133	Thu	sms	None	PASS
51	Tue	bill	(0, 871, False)	PASS	134	Thu	pay	1	PASS
52	Tue	bill	(0, 140, True)	PASS	135	Thu	sms	None	PASS
53	Tue	bill	(0, 301, False)	PASS	136	Thu	bill	(0, 301, False)	PASS
54	Tue	bill	(0, 500, False)	FAIL	137	Fri	pay	2	FAIL
55	Tue	pay	2	FAIL	138	Fri	pay	1	PASS
56	Tue	pay	2	FAIL	139	Fri	bill	(0, 100, False)	PASS

#	Day	Kind	Input	Result	#	Day	Kind	Input	Result
57	Tue	bill	(0, 101, True)	PASS	140	Fri	bill	(0, 100, False)	PASS
58	Tue	bill	(0, 100, True)	PASS	141	Fri	bill	(0, 300, False)	PASS
59	Tue	bill	(0, 301, False)	PASS	142	Fri	sms	None	PASS
60	Tue	pay	1	PASS	143	Fri	bill	(0, 101, False)	PASS
61	Tue	bill	(0, 101, True)	PASS	144	Fri	bill	(0, 301, False)	PASS
62	Tue	bill	(0, 66, False)	PASS	145	Fri	bill	(0, 300, False)	PASS
63	Tue	bill	(0, 666, False)	PASS	146	Fri	bill	(0, 300, False)	PASS
64	Tue	bill	(0, 500, False)	FAIL	147	Fri	sms	None	PASS
65	Tue	pay	1	PASS	148	Fri	bill	(0, 301, False)	PASS
66	Tue	bill	(0, 8, True)	PASS	149	Fri	bill	(0, 101, False)	PASS
67	Tue	bill	(0, 500, False)	FAIL	150	Fri	pay	2	FAIL
68	Tue	pay	1	PASS	151	Fri	bill	(0, 300, False)	PASS
69	Tue	bill	(0, 501, True)	PASS	152	Fri	pay	1	PASS
70	Tue	pay	1	PASS	153	Fri	bill	(0, 517, False)	PASS
71	Wed	bill	(0, 300, False)	PASS	154	Fri	pay	1	PASS
72	Wed	bill	(0, 501, False)	PASS	155	Fri	pay	1	PASS
73	Wed	bill	(0, 101, True)	PASS	156	Fri	bill	(0, 100, False)	PASS
74	Wed	bill	(0, 500, False)	FAIL	157	Fri	bill	(0, 500, True)	FAIL
75	Wed	bill	(0, 100, True)	PASS	158	Fri	bill	(0, 226, True)	PASS
76	Wed	bill	(0, 611, False)	PASS	159	Fri	pay	1	PASS
77	Wed	sms	None	FAIL	160	Fri	bill	(0, 301, False)	PASS
78	Wed	bill	(0, 301, False)	PASS	161	Fri	bill	(0, 301, False)	PASS
79	Wed	bill	(0, 501, True)	PASS	162	Fri	bill	(0, 300, True)	PASS
80	Wed	bill	(0, 100, True)	PASS	163	Fri	bill	(0, 101, False)	PASS
81	Wed	bill	(0, 70, True)	PASS	164	Fri	bill	(0, 301, True)	PASS
82	Wed	pay	2	FAIL	165	Fri	bill	(0, 501, False)	PASS
83	Wed	sms	None	FAIL	166	Fri	bill	(0, 815, False)	PASS

ANNEXURE K – Debugging Records of Task 1 (real output)

Fault F1

Print session output (approach A):

```
print: u=500 energy=2850.00 fixed=40 total=2890 expected=2740
print: units=500 withdrawn=True first=4.50
```

pdb transcript (approach B):

```
> _drv.py(2)<module>()
-> import sys; sys.path.insert(0, '.'); import powerpay as pp
(Pdb) Breakpoint 1 at powerpay.py:36
(Pdb) > powerpay.py(36)energy_charge()
-> withdrawn = units >= 500 if "F1" in ACTIVE else units > 500
(Pdb) 500
(Pdb) > powerpay.py(37)energy_charge()
-> first = Decimal("4.50") if withdrawn else Decimal("3.00")
(Pdb) True
(Pdb)
```

git bisect log (approach C), minimal case 7713→8213:

```
running '/usr/bin/python3' 't.py'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[f08b1fc6340ccbe99998e54ad476c45ae3f74b73] docstrings
running '/usr/bin/python3' 't.py'
Bisecting: 1 revision left to test after this (roughly 1 step)
[b4fb522ae8c4fca22234dd05a9111234ef8a59b8] due-date helper
running '/usr/bin/python3' 't.py'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[ddd46db7b5addb8e29e38c0aac094e6863030163] subsidy rule for >500 units refactored
running '/usr/bin/python3' 't.py'
ddd46db7b5addb8e29e38c0aac094e6863030163 is the first bad commit
commit ddd46db7b5addb8e29e38c0aac094e6863030163
Author: dev <t@pp>
Date:   Wed Nov 4 08:35:41 2026 +0000
    subsidy rule for >500 units refactored
 powerpay.py | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
bisect found first bad commit
```

Fault F2

Print session output (approach A):

```
print: u=150 energy=545.00 fixed=40 total=585 expected=590
print: branch=101-300 base=49 (expected 50)
print: slab start constant 101 should be 100
```

pdb transcript (approach B):

```
> _drv.py(2)<module>()
-> import sys; sys.path.insert(0, '.'); import powerpay as pp
(Pdb) Breakpoint 1 at powerpay.py:41
(Pdb) > powerpay.py(41)energy_charge()
-> base = units - 101 if "F2" in ACTIVE else units - 100
(Pdb) 150
(Pdb) > powerpay.py(42)energy_charge()
-> return first * 100 + 5 * base
(Pdb) 49
(Pdb)
```

git bisect log (approach C), minimal case 39991→40256:

```
running '/usr/bin/python3' 't.py'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[b6d98d3f1ca0b56d11c7131ab5481998f06d06e9] meter reading parser
```

```
running '/usr/bin/python3' 't.py'
Bisecting: 1 revision left to test after this (roughly 1 step)
[61b5e70d505cbcce2eb23045735526c0c56469e3] receipt sms text
running '/usr/bin/python3' 't.py'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[5330ccf7aaf2b22769e711b8e5b461a38db4cda4] tariff constants moved
running '/usr/bin/python3' 't.py'
5330ccf7aaf2b22769e711b8e5b461a38db4cda4 is the first bad commit
commit 5330ccf7aaf2b22769e711b8e5b461a38db4cda4
Author: dev <t@pp>
Date:   Wed Nov 4 08:35:40 2026 +0000
    tariff constants moved
 powerpay.py | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
bisect found first bad commit
```

Fault F3

Print session output (approach A):

```
print: u=20 energy=60.00 fixed=40 total=101 expected=102
print: after late fee=101.50000
print: int(x)=101 half-up=102
```

pdb transcript (approach B):

```
> _drv.py(2)<module>()
-> import sys; sys.path.insert(0, '.'); import powerpay as pp
(Pdb) Breakpoint 1 at powerpay.py:49
(Pdb) > powerpay.py(49)round_rupee()
-> x = Decimal(x)
(Pdb) Decimal('101.50000')
(Pdb) > powerpay.py(50)round_rupee()
-> if "F3" in ACTIVE:
(Pdb) > powerpay.py(51)round_rupee()
-> return int(x)
(Pdb) 101
(Pdb)
```

git bisect log (approach C), minimal case 23303→23323:

```
running '/usr/bin/python3' 't.py'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[f08b1fc6340ccbe99998e54ad476c45ae3f74b73] docstrings
running '/usr/bin/python3' 't.py'
Bisecting: 1 revision left to test after this (roughly 1 step)
[a10b6844faa590a2f756eaad24a9689c927c77da] rounding helper simplified
running '/usr/bin/python3' 't.py'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[29cd5f424a913acffb1df0e9328d3026cf01f27d] receipt template
running '/usr/bin/python3' 't.py'
a10b6844faa590a2f756eaad24a9689c927c77da is the first bad commit
commit a10b6844faa590a2f756eaad24a9689c927c77da
Author: dev <t@pp>
Date:   Wed Nov 4 08:35:41 2026 +0000
    rounding helper simplified
 powerpay.py | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
bisect found first bad commit
```

Fault F4

Print session output (approach A):

```
print: u=99900 energy=897450.00 fixed=40 total=897490 expected=340
print: previous=99950 current=50 d=-99900 -> abs used = 99900
```

pdb transcript (approach B):

```
> _drv.py(2)<module>()
-> import sys; sys.path.insert(0, '.'); import powerpay as pp
(Pdb) Breakpoint 1 at powerpay.py:28
(Pdb) > powerpay.py(28)units_consumed()
-> return abs(current - previous)
(Pdb) (99950, 50)
(Pdb) -99900
(Pdb)
```

git bisect log (approach C), minimal case 99950→50:

```
running '/usr/bin/python3' 't.py'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[b6d98d3f1ca0b56d11c7131ab5481998f06d06e9] meter reading parser
running '/usr/bin/python3' 't.py'
Bisecting: 0 revisions left to test after this (roughly 1 step)
[0b8cd2524029bb4347a86d82f6b4e71a2a838e23] add late fee
running '/usr/bin/python3' 't.py'
b6d98d3f1ca0b56d11c7131ab5481998f06d06e9 is the first bad commit
commit b6d98d3f1ca0b56d11c7131ab5481998f06d06e9
Author: dev <t@pp>
Date:   Wed Nov 4 08:35:40 2026 +0000
    meter reading parser
 powerpay.py | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
bisect found first bad commit
```

ANNEXURE L – Incident Report Template and Unit-Level Incident Reports

Template fields (IEEE 829 test incident report, as used in the tool): identifier; summary; item and build with checksums; test case and log reference; inputs; expected and actual results; steps to reproduce; environment; severity and priority; evidence attached; reproducibility (n of n); status history; linked fix commit and regression test.

Field	Content
Identifier	INC-U1
Summary	Bill for 101 units ₹340 instead of ₹345
Item / build	Billing module, unit build; seeded fault F2
Inputs	0→101
Expected / actual	₹345 / ₹340
Severity	Major (₹5 under-charge for 101–300 units)
Cause located by bisect	rev 7 “tariff constants moved”
Regression test	test_reg_f2_… (fails before, passes after)
Status	Fixed → verified → closed

Field	Content
Identifier	INC-U2
Summary	Late bill ₹101.50 shown as ₹101
Item / build	Billing module, unit build; seeded fault F3
Inputs	0→20, late
Expected / actual	₹102 / ₹101
Severity	Major (rounding against the rule)
Cause located by bisect	rev 14 “rounding helper simplified”
Regression test	test_reg_f3_… (fails before, passes after)
Status	Fixed → verified → closed

Field	Content
Identifier	INC-U3
Summary	Rolled-over meter billed 99,900 units
Item / build	Billing module, unit build; seeded fault F4
Inputs	99950→50
Expected / actual	₹340 / very large bill
Severity	Critical (huge wrong bill)
Cause located by bisect	rev 4 “meter reading parser”
Regression test	test_reg_f4_… (fails before, passes after)
Status	Fixed → verified → closed

ANNEXURE M – Test Policy TP-2026 (full text of policy B)

1 Purpose and scope

This policy states why and how the State Power Distribution Corporation tests software. It applies to all software that computes, collects or reports consumer money or consumer data, and to every system used by more than 1,000 consumers, whether developed in-house or by a vendor.

2 Mission of testing

Testing provides independent, documented evidence of whether a system meets its requirements and is fit to be released to consumers. Testing is not a phase at the end; it starts with the review of requirements.

3 Definitions

Test levels: unit, integration, system, acceptance. Severity: critical (wrong money, loss of data, security breach, unavailable service), major (wrong function with workaround), minor (cosmetic). Test item: a uniquely identified build or configuration item. Baseline: a tagged, approved set of items with checksums.

4 Principles

(a) Testing effort follows product risk. (b) System and acceptance testing are independent of the developers. (c) The released item must be identical to the tested item. (d) Every fix carries a regression test. (e) Consumer data used in testing is masked.

5 Documentation

Each project keeps a test plan, a traceability matrix, incident reports and a test summary report per level. IEEE 829 or an equivalent tool-generated record may be used. Documents for money-related modules are never skipped.

6 Metrics

Each project reports defect removal efficiency, open defects by severity and age, requirement coverage, reopen rate and fix time to the test process group at release and 90 days after release.

7 Quality targets

DRE ≥ 90 %; reopen rate ≤ 5 %; critical fix time ≤ 2 working days; zero open critical defects at release.

8 Roles

The Head of IT owns this policy. The test process group maintains the strategy and measures compliance. Each project’s test manager is responsible for applying it. The Chief Engineer (IT) signs every release to consumers.

9 Release rule

A system is released to consumers only when statements P1–P8 are met, or when a waiver naming the unmet statement, the reason, the risk and the owner is signed by the Chief Engineer (IT).

10 Tailoring

A project may relax statements through a tailoring sheet approved by the test process group. P1 and P3 cannot be relaxed.

11 Review

The policy is reviewed every year using the metrics of clause 6 and the results of the TMMi assessment.

ANNEXURE N – Organisational Test Strategy (summary of policy C)

Risk class	Example	Levels	Techniques	Automation	Documents
A Money	Billing, payment, refund	Unit, integration, system, acceptance	BVA, decision tables, state models, oracle	Mandatory regression on every build	Full TDS/TCS, TSR per level
B Consumer data	Registration, login, history	Unit, system	EP, BVA, security checks	Smoke + login regression	Cases in tool, TSR
C Information	Notices, FAQ pages	System	Checklist, exploratory	Optional	Checklist record
Non-functional	Load near due date	System	Load profile from logs	JMeter scripts kept	Load report

Standard environments: development, test (reserved for testers), staging (production-like). Standard tools: TestLink or equivalent, JIRA, pytest/Selenium, JMeter. Defect workflow: New → Accepted → Fixed (with regression test) → Verified → Closed; Reopened returns to Accepted and triggers a root-cause note.

ANNEXURE O – Formula Sheet and Worked Calculations

Quantity	Formula	Worked value
Energy charge, ≤ 500 units	3·min(u,100) + 5·max(0,min(u,300)−100) + 7·max(0,min(u,500)−300)	u = 500 → 300 + 1,000 + 1,400 = ₹2,700
Energy charge, > 500 units	4.5·100 + 5·200 + 7·200 + 9·(u − 500)	u = 501 → 450 + 1,000 + 1,400 + 9 = ₹2,859
Bill	round_half_up(energy + 40) (× 1.015 if late)	u = 500 → ₹2,740; u = 20 late → ₹102
Rollover units	current − previous + 100000 if negative	99950 → 50 = 100 units
Effort	C·0.4 + C·0.25·3 + 0.2·(execution); × 1.15	C = 180 → 269.1 h; C = 400 → 598 h
Capacity	testers × weeks × 30 h	4 × 10 × 30 = 1,200 h; 3 × 4 × 30 = 360 h
Risk exposure	probability × impact (1–5 each)	R1: 4 × 4 = 16
DRE	found before release ÷ (before + after)	52 ÷ 73 = 71.2 %
Bisect runs	⌈ $\log_2 n$ ⌉ for n commits	n = 16 → 4
Communication paths	n(n − 1)/2	n = 5 → 10
Smart-meter rate	meters × 96 ÷ 86,400; burst = meters ÷ 60	12 lakh → 1,333/s; 20,000/s

ANNEXURE P – Risk Register with Owners, Triggers and Status

ID	Risk	P×I	Owner	Trigger	Status (week 5)
R1	Test server unavailable or shared with development	16	Build engineer	Server down > 4 h in a week	Open – watched
R2	Builds delivered late to test	16	Development lead	Build not ready Monday 10:00	Open – watched
R7	Defect fix time stays at 4 days and reopen rate at 18 %	16	Development lead	Median critical fix > 2 days	Open – watched
R3	Tariff slabs revised by the regulator during testing	15	Business analyst	Regulator notice received	Open – watched
R9	Load of 3 lakh consumers near due date not tested	15	T4	Load scripts not ready by week 7	Open – watched
R5	Payment-gateway sandbox unstable	12	T4	Sandbox error rate > 10 %	Open
R6	Requirements unclear (late fee, subsidy rule)	12	Business analyst	Rule query open > 48 h	Open
R8	Real consumer data copied into the test environment	10	Build engineer	Unmasked field found in scan	Open
R4	Tester absent for a week or more	9	Test manager	Leave > 3 days	Open
R10	Tool budget overspent	4	Test manager	Spend > ₹1,20,000	Open

ANNEXURE Q – Masking Specification and Sample Records

Field	Original (synthetic)	Masked	Rule
consumer_no	394096041694	930355703226	HMAC → 12 digits
name	Senthil Nair	NAME-02645C18DE	token
mobile	9253115961	9000081738	range 90000xxxxx
email	user0@example.in	E-17238DEF78@masked.invalid	token @masked.invalid
address	98, Anna Salai, Ward 24	Ward 24	ward only
meter_no	SM74765263	SM43DFA5AD	token
units	280	280	kept
bill	0	0	kept
upi_vpa	divya0@okbank	VPA-8C5588E464@test	token @test

All originals here are synthetic records generated by `exp_t9.py`; no real consumer data was used. The key is stored outside the repository and rotated per release; without it the tokens cannot be reversed.

ANNEXURE R – Skills Assessment Used for the Matrix

#	Assessment item	Skill area	Scoring
1	List the boundary values for REQ-05 and REQ-06 (10 min)	Test design	1 per two correct values, max 5
2	Compute the bill for 500 and 501 units	Billing domain	Both correct 5; one 3; none 1
3	Write an incident report for a given failure	Communication	Checklist of 9 fields
4	Find the cause from a 20-line log	Analytical	Time and correctness
5	Run a pytest test and read a JIRA filter	Tools	Unaided 5; with help 3
6	Estimate effort for 20 conditions	Planning	Within 20 % of reference 5

ANNEXURE S – Go / No-Go Checklist for the Release Meeting

#	Check (policy statement)	Evidence	Week-5 status
1	No open critical defect (P1)	Tracker query	✗ INC-01, INC-02 open
2	Billing and payment regression automated and passing (P2)	Pipeline report	Partly (billing yes)
3	Deployed items equal baseline (P3)	Checksum audit log	✓
4	Requirement coverage 100 % or waiver (P7)	Traceability matrix	✗ REQ-14
5	Summary report signed (P8)	TSR in repository	Draft
6	Masked data only	Leak scans	✓ 0 field / boundary hits
Decision			**NO-GO** at week 5; re-check at week 10

ANNEXURE T – Procedure to Reproduce the Results

- Install Python 3.10+, git and graphviz; `pip install pytest matplotlib`.
- Run `python exp_t1.py` – Task 1 (creates `_repo` with 16 commits and runs bisect; about 20 s).
- Run `python exp_t34.py` – Tasks 3 and 4 (baseline, wrong tariff, log, week, replay).
- Run `python exp_t9.py` – Task 9 (masking 20,000 synthetic records, scan, meter validator).
- Run `python calc_pp.py` – prints the effort, checklist, risk, simulation and documentation figures.
- Run `python figs4.py` – regenerates all figures in `figs/`.
- Run `POWERPAY_FAULTS=F1 pytest tests` to see a regression test fail, and `pytest tests` to see all pass.

ANNEXURE U – Self-Evaluation Against the Evidence Expected

Task	Evidence expected	Where	Self-check
1	Time/runs/tools table; testing vs debugging; choice	Table 3, Table 5, Table 7	Complete
2	Three plans; checklist; effort; risks; choice	Table 9–Table 14	Complete
3	Item lists; attachments; RTM; choice	Table 17–Table 23	Complete
4	Log + incidents; three reports; reproducibility; choice	Table 25–Table 32	Classmate column open
5	Three policies; statements; problem table; choice	Table 35–Table 39	Complete
6	Three charts; situation table; choice	Figure 11–Figure 13, Table 44	Complete
7	Matrix; three plans; cost/gap; choice	Table 46–Table 49	Complete
8	Documents; flowchart with owners; effort; choice	Table 51, Figure 18, Table 52	Complete
9	Impact + effort; evaluation; comparison + flowchart; defence	Table 56–Table 62	Complete

ANNEXURE V – Test Design Specification for the Payment Feature (full set, billing and payment are ‘full’ modules)

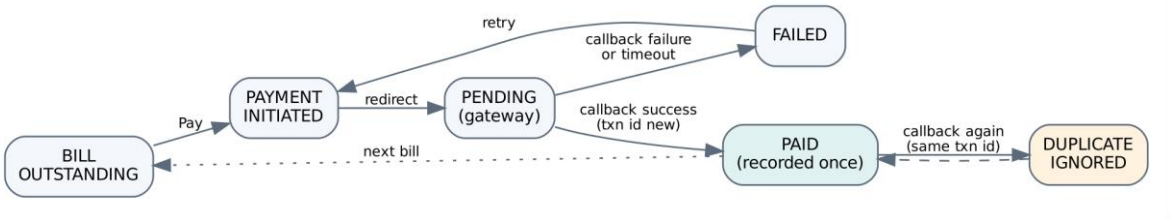


Figure 22: Payment state model used to design the payment tests (REQ-09, REQ-10)

Field	Content
Identifier	TDS-PAY-01 v1.0
Features	REQ-09 record once per transaction id; REQ-10 exact amount; REQ-11 receipt
Approach	State transition testing (every transition and the invalid ‘callback again’ event), equivalence classes of payment mode, boundary of amount (bill − ₹1, bill, bill + ₹1)
Pass/fail	Ledger total equals the bill; exactly one receipt SMS per transaction id

TC	Transition / class	Steps	Expected
TC-PAY-01	OUTSTANDING → PAID	Pay ₹2,740 once	Ledger ₹2,740; 1 SMS
TC-PAY-02	PAID → DUPLICATE IGNORED	Gateway repeats callback T3	Ledger unchanged; no second SMS
TC-PAY-03	Two different txn ids	T5 and T6 for two bills	Both recorded
TC-PAY-04	PENDING → FAILED → retry	Timeout, then pay again	Recorded once after retry
TC-PAY-05	Amount boundary	Bill − ₹1, bill, bill + ₹1	Refuse, accept, refuse
TC-PAY-06	Payment modes	UPI, card, net banking	Each recorded once
TC-PAY-07	Double click on Pay	Two clicks within 1 s	One transaction id
TC-PAY-08	Session expiry	Callback after logout	Recorded once; consumer sees it on next login

ANNEXURE W – Pilot-Year Root-Cause Analysis and Lessons Built into the Process

Pilot problem	Root cause	Process element that prevents it	Task
Bill wrong at exactly 500 units	Boundary of REQ-06 not designed as a test; no oracle	BVA on every slab edge; unit-by-unit oracle; regression test with every fix	1, 3
Payment recorded twice	Gateway retries not modelled; no idempotency test	State model with ‘callback again’; TC-PAY-02	3, 4, V
Tested build ≠ released build	Configuration files outside the build number	Baseline tag + SHA-256 + item transmittal report; deployment audit	3
Nobody knew how much was tested	No plan, no traceability, no summary report	Risk-based plan, traceability, dashboard, signed TSR	2, 4
21 field defects (6 critical)	No release rule	Policy P1–P8, go/no-go checklist	5, S
18 % reopen, 4-day fixes	Fixes confirmed only on the reported case	Systematic localisation, boundary regression suite, 2-day critical target	1, 5
Testers missed the boundary	No domain mentoring; no skills matrix	Mentoring with ex-billing tester; matrix re-scored	7

Weekly status summaries, weeks 1–4 (plan view)

Week	Planned	Status line for the Chief Engineer
1	Plan, SRS review, risk analysis	Plan PP-TP-01 approved; 3 SRS ambiguities (late fee day 15, subsidy at 500, rollover) resolved by the BA
2	Environment, design starts	Test server reserved; gateway sandbox delayed to week 3 (R5); 60 of 180 conditions designed
3	Design complete (M1)	180 of 180 designed; traceability 13 of 14 (REQ-14 planned for week 7); smoke suite automated
4	Cycle 1 starts	Build b214 delivered 2 days late (R2); first 30 conditions run; INC-01 raised

ANNEXURE X – Tool Budget, Effort per Module and Condition Counts

Item	Cost	Reason
Second VM for load and testing, 6 weeks (State Data Centre cloud)	₹18,000	Risk R1
JIRA / test management, 10 users, 3 months (cloud standard tier, assumed price)	₹36,000	Single shared tool (Tasks 4, 8)
JMeter, pytest, Selenium, TestLink	₹0	Open source (R10)
Payment gateway sandbox	₹0	Provided by the bank
SMS stub and test SIM pack	₹6,000	REQ-11 checks
Reserve for tariff change tooling / extra VM time	₹30,000	Risk R3
Total committed	**₹90,000**	₹60,000 unspent (certification is funded from the training budget, Task 7)

Module	Requirements	Conditions	Risk	Design h (×0.4)	Execution h (×0.75)	Total with 20 % + 15 %
Consumer & Login	REQ-01, 02, 13	22	High	8.8	16.5	32.9
Meter Reading	REQ-03, 04	15	High	6.0	11.2	22.4
Billing	REQ-05–08	58	Critical	23.2	43.5	86.7
Payment	REQ-09, 10	40	Critical	16.0	30.0	59.8
Receipt / SMS	REQ-11	20	High	8.0	15.0	29.9
Complaints & Outage	REQ-12	25	Medium	10.0	18.8	37.4
Total		180		72	135	269.1

ANNEXURE Y – Record of the Plan Approval Meeting (template filled for PP-TP-01)

Item	Record
Meeting	PowerPay test plan approval, 04-11-2026, 11:00–11:45
Present	Chief Engineer (IT), Head of Billing, Audit officer, Test manager, Development lead, Business analyst
Presented	1-page risk table (Plan C core), effort and capacity chart, schedule with M1–M4, features not tested, suspension criteria
Questions	Audit: how is tested = released proved? → baseline + checksum audit (Task 3). Billing: who signs the 500-unit rule? → BA by week 1.
Decisions	Plan approved as v1.0; second VM approved (₹18,000); IE11 exclusion accepted; tariff change reserve of 40 h accepted
Actions	BA: rule table signed by 06-11; Build engineer: release-note template by 06-11; TM: weekly status from week 2
Signatures	Chief Engineer (IT) _____ Head of Billing _____ Audit officer _____

Test summary report template for the acceptance level (to be completed in week 10)

Section	To record
Identifier	TSR-ACC-01
Variances	Changes to items, plan or procedures since TSR-SYS-01
Comprehensiveness	Acceptance cases run by the billing section against the traceability matrix; anything not run and why
Summary of results	Incidents found in acceptance, by severity; status of INC-01, INC-02
Evaluation	Go / no-go against policy P1–P8 and the go/no-go checklist (Annexure S)
Approvals	Billing coordinator, Head of Billing, Chief Engineer (IT)

ANNEXURE Z – SRS Review Record (testability of the 14 requirements)

Each requirement was reviewed for testability before test design (week 1). A requirement is testable when its expected result can be stated exactly for any input.

Requirement	Verdict	Question raised	Answer recorded
REQ-01	Testable	OTP expiry time not stated	BA: 5 minutes
REQ-02	Testable	Does the 30-minute lock reset on success?	Yes
REQ-03	Ambiguous	Lower reading allowed after meter replacement?	Only with a replacement flag
REQ-04	Testable	–	–
REQ-05	Testable	–	–
REQ-06	Ambiguous	“Above 500” – is 500 included?	No: strictly above 500 (pilot defect)
REQ-07	Ambiguous	Rounding of ₹x.50	Half up
REQ-08	Ambiguous	Is day 15 late?	No: late from day 16
REQ-09	Testable	–	–
REQ-10	Testable	Several outstanding bills?	Oldest first
REQ-11	Testable	Language of SMS	Tamil and English
REQ-12	Untestable as written	‘Status tracking’ undefined	States: Open, Assigned, Resolved, Closed
REQ-13	Testable	Fewer than 12 bills?	Show all available
REQ-14	Ambiguous	Which pages? which mix?	Bill view 60 %, payment 30 %, login 10 %

Seven of fourteen requirements needed a clarification before tests could have exact expected values; REQ-06 is the one whose ambiguity caused the pilot defect. Reviewing requirements is the cheapest test activity in this project.